GREEN POLICY INTELLIGENCE ENGINE (GPIE)

Index

Project Overview

The Green Policy Intelligence Engine (GPIE) is a global geospatial decision intelligence framework designed to independently evaluate the real-world environmental effectiveness and economic efficiency of environmental policies using Earth Observation, Geographic Information Systems (GIS), remote sensing, spatial data science, and environmental economics.

Conventional policy assessments primarily rely on government-reported statistics, which may be delayed, inconsistent, or collected using different methodologies across countries. GPIE addresses this limitation by integrating independently observed satellite-derived environmental indicators with socioeconomic, economic, and policy datasets to create an objective and reproducible evaluation framework.

Rather than focusing on a single country or policy, the framework is designed as a globally transferable methodology capable of evaluating environmental policies implemented in different geographic, economic, and political contexts.

The European Green Deal will serve as the first demonstration case because of its comprehensive environmental policy framework, high-quality open datasets, and global relevance. However, the methodology is intentionally designed so that it can later be applied to any country or region worldwide.

Problem Statement

Governments invest billions of dollars annually in environmental and climate policies. Despite these investments, there is currently no unified geospatial framework capable of independently verifying whether implemented policies have produced measurable environmental improvements while simultaneously evaluating their economic efficiency and spatial impacts.

Existing evaluations frequently depend on administrative reports and fragmented datasets, making comparisons across countries and policies difficult. There remains a need for a transparent, reproducible, and satellite-driven framework capable of integrating environmental observations, economic indicators, spatial analysis, and policy information into a single decision-support system.

Aim

To develop a globally applicable geospatial framework that objectively measures, compares, and visualizes the environmental performance and economic effectiveness of environmental policies using satellite observations, GIS, Python automation, and spatial analytical techniques.

Objectives

- Develop an automated workflow for collecting policy, environmental, and socioeconomic datasets.
- Integrate multi-source Earth Observation datasets into a unified spatial database.
- Quantify measurable environmental changes associated with environmental policy implementation.
- Compare environmental outcomes across different geographic regions.
- Evaluate the economic effectiveness of environmental interventions using environmental economics indicators.
- Produce reproducible geospatial analyses, thematic maps, and decision-support outputs.
- Build a modular framework that can be reused for evaluating future environmental policies worldwide.

Research Question

How can Earth Observation, GIS, remote sensing, spatial data science, and environmental economics be integrated into a reproducible geospatial framework capable of objectively evaluating the environmental effectiveness and economic efficiency of environmental policies at regional, national, and global scales?

Expected Outputs

The completed project will generate:

- A fully automated Python-based geospatial processing workflow.
- A standardized environmental policy evaluation framework.
- Satellite-derived environmental assessment datasets.
- GIS-ready spatial databases.
- Thematic maps and interactive geospatial visualizations.
- Environmental policy performance indicators.
- Economic efficiency assessments.
- Comparative regional analyses.
- A reproducible methodology suitable for adaptation to other countries and policy domains.
- Complete technical documentation and open-source implementation.

Demonstration Case

European Green Deal

The European Green Deal will be used exclusively as the first implementation and validation case study for demonstrating the capabilities of the Green Policy Intelligence Engine. Following validation, the framework can be extended to evaluate environmental policies implemented anywhere in the world.

Current Status

Project Concept Finalized

Version 1.0

==========

GPIE — Module Architecture (Updated)

GPIE — Module Architecture (Final)

MODULE 1 — Policy Database Acquisition ✅ EXECUTED

Automated scraping of European Green Deal policy records from EUR-Lex. Extracts and structures policy metadata (title, type, year, status, CELEX ID, summary, word count, thematic tags) into `documents.json` / `documents.csv`, followed by exploratory statistical analysis and visualization. This forms the "policy" half of the project's core comparison — what governments claim to have done.

MODULE 2 — Earth Observation & Auxiliary Data Acquisition ✅ EXECUTED (Population partial by

design)

Acquisition of all independent datasets needed to verify policy outcomes and support spatial/economic analysis, across two scopes: the original EU-27 dataset, and an expanded 30-country dataset (EU-27 + UK, Norway, Switzerland) built later to support a genuine control group for causal inference. Sentinel-5P NO_2 and Sentinel-2/CGLS NDVI fully acquired for both scopes via the Sentinel Hub Statistical API (December data-gap bug identified and fixed in both variables' acquisition scripts). ERA5 Climate fully acquired and processed for both scopes (a duplicate-timestamp processing bug affecting the temperature variable was identified and fixed; EFTA-country double-counting in the 30-country boundary set was also identified and fixed). Copernicus DEM and ESA WorldCover fully acquired for the EU-27 scope only (not required for the control-group countries, since these are static variables fully absorbed by country fixed effects in the causal model). Eurostat GDP acquired for EU-27; a second GDP source (World Bank API, with documented approximate EUR/USD currency conversion) acquired for the three control-group countries, since Eurostat does not cover non-EU countries. NUTS Boundaries (EU-27) and GADM Level-0 boundaries (UK, Norway, Switzerland) both acquired, combined via a shared boundary-loading utility (`country_boundaries.py`). WorldPop Population remains partially executed by design (2019–2020 only), formally a supporting/descriptive dataset excluded from causal modeling.

MODULE 3 — Preprocessing & Standardization ✅ EXECUTED

Converts raw acquired data into standardized, analysis-ready form, for both the EU-27-only and 30-country (EU-27 + control group) dataset scopes.

- **NO_2 and NDVI**: processed via Sentinel Hub server-side statistical aggregation (minQa=75 quality filtering applied server-side for NO_2); flattened from nested API-response structure into flat per-country-year-month format via shared flattening logic, applied consistently to both the EU-27-only and 30-country datasets.
- **ERA5 Climate**: fully processed — unit conversion (Kelvin→Celsius, meters→millimeters), merging of split temperature/precipitation source files with explicit time-coordinate alignment (correcting a duplicate-timestamp bug), and aggregation into country-level monthly statistics for both dataset scopes, using a unified NUTS+GADM boundary loader for the 30-country version.
- **ESA WorldCover (Land Cover)**: fully processed for EU-27 — mosaicked via VRT, resampled to 500m using nearest-neighbor (categorical-safe) resampling, aggregated into country-level land cover class percentages.
- **Copernicus DEM**: fully processed for EU-27 — mosaicked via VRT (860 tiles), resampled to 500m using bilinear resampling, aggregated into country-level elevation statistics via `process_dem.py`.
- **Eurostat + World Bank GDP**: both sources fully processed into a combined, consistently-unioned lookup table covering all 30 countries.
- **WorldPop Population**: partially processed (28 of 54 country-year files; supporting/descriptive only, not blocking core pipeline).

Two master datasets were assembled: `master_dataset.csv` (EU-27 only, 1,944 rows, includes DEM and Land Cover) for exploratory work, and `master_dataset_control.csv` (30 countries, 2,160 rows, includes a `treatment_group` indicator) as the dataset actually used for the project's final causal inference model.

MODULE 4 — Temporal Aggregation ✅ EFFECTIVELY COMPLETE (via per-dataset scripts, not a shared engine)

Monthly aggregation achieved independently within each dataset's own processing script. No dedicated, generalized temporal-aggregation engine was built as a separate module; this remains a possible future refactor but has not blocked progress, since all monthly-resolution datasets share a consistent `(country, year, month)` structure suitable for merging across both dataset scopes.

MODULE 5 — Export & Output Standardization ✅ EFFECTIVELY COMPLETE (ad hoc, not a dedicated engine)

All processed datasets exist in practical analysis-ready form (JSON/CSV), produced ad hoc within each dataset's own processing or flattening script. This approach proved sufficient to reach two fully merged, verified master datasets; a dedicated export module was not built as a separate component.

MODULE 6 — Validation & Quality Control 🟡 PARTIALLY EXECUTED (ad hoc, proven highly effective)

No dedicated, standalone validation engine was built. Systematic ad hoc verification was applied consistently throughout — record-count checks, missing-value audits, diagnostic investigation of illogical patterns, and, critically, **statistical robustness testing of the causal model itself** (placebo test, event-study disaggregation). This approach identified and resolved multiple hidden defects across the project (the NO_2/NDVI December gap, the ERA5 duplicate-timestamp bug, EFTA double-counting in the 30-country boundary set) and, most significantly, identified a fundamental research-design flaw in the original single-cohort causal model via placebo testing — arguably this module's most important contribution to the project's overall scientific integrity.

MODULE 7 — Pipeline Orchestration & Execution Management 🟡 PARTIALLY EXECUTED (superseded in practice)

The original month-wise orchestrator (`run_pipeline.py`) remains implemented but is no longer the primary execution path. All datasets were acquired and processed via independent, dataset-specific scripts, coordinated manually. This has proven adequate for the project's actual scale.

MODULE 8 — Causal Inference & Policy Verification ✅ COMPLETE

Used Difference-in-Differences methodology to test whether the European Green Deal / European Climate Law (effective 30 June 2021) produced a statistically distinguishable reduction in satellite-observed NO_2 pollution across EU-27 countries.

Development sequence: An initial single-cohort model (all 27 EU countries, no control group, country and seasonal fixed effects) found a statistically significant negative effect (p=0.026). A placebo test — rerunning the identical model with a fake treatment date — found an equally or more significant "effect" at a date with no relevant policy event, revealing that the original result was capturing a general multi-year pollution-decline trend rather than a policy-specific effect. This is a well-documented structural limitation of single-cohort designs applied to policies affecting an entire study population simultaneously, with no available control group.

Correction: A genuine external control group (United Kingdom, Norway, Switzerland — geographically and economically comparable non-EU European countries) was constructed, requiring new boundary data (GADM), extended Earth Observation acquisition (NO_2, NDVI, Climate for 30 countries), and a second GDP data source (World Bank API). A proper two-group DiD model (EU-27 vs. control group, treatment × post interaction term) was then estimated.

Final result: The DiD interaction coefficient was not statistically significant (coefficient -1.40×10^{-6}, p=0.632, 95% CI spanning zero). An event-study extension, testing the effect separately across all 23 individual quarters from 2019Q1 to 2024Q4, found no significant effect in any single quarter — providing both support for the model's parallel-trends assumption (no significant pre-treatment divergence) and confirmation that the null result is not an artifact of averaging across time (no delayed effect emerged in any later quarter either).

Conclusion: No statistically distinguishable EU-specific reduction in NO_2 attributable to the European Climate Law was found, once genuinely compared against a non-EU control group; the pollution decline observed within the EU-27 over this period appears to reflect a broader trend shared with comparable non-EU European countries. This is reported as a rigorously validated, honest scientific finding consistent with the project's "Trust, But Verify" design — the validation process itself (placebo test → control group → event study) is as significant a project output as the substantive result.

Environment note: A significant native numerical-computing failure (corrupted Intel MKL backend causing silent crashes across `linearmodels`, `statsmodels`, and raw NumPy operations) was diagnosed and resolved mid-module via reinstalling NumPy/SciPy with an OpenBLAS backend (`nomkl`), unrelated to any project code logic.

MODULE 9 — Economic Efficiency Ranking ⚠️ SCOPED OUT (reasoned decision, not abandoned)

Originally planned to combine causal inference results with policy cost data to rank environmental interventions by cost-per-unit-environmental-improvement. **This module has been deliberately scoped out following Module 8's completion**, for a specific, documented reason: Module 8's rigorously validated finding is that no statistically significant EU-specific causal effect on NO_2 was detected. Constructing a "cost-per-unit-environmental-improvement" ranking presupposes the existence of a measurable improvement to rank against cost — proceeding with this module as originally conceived would require either manufacturing statistical significance that the data does not support, or ranking interventions against an effect size that is not distinguishable from zero, neither of which is scientifically defensible.

This decision is itself treated as a finding consistent with the project's core design principle: GPIE exists to independently verify policy claims rather than to assume their effectiveness, and a module whose premise depends on an effect that verification did not confirm should not be forced to completion.

MODULE 10 — Geospatial Output Generation ✅ COMPLETE

Seven publication-quality geospatial and statistical visualizations were produced using a Python-based mapping pipeline (`geopandas` + `matplotlib`), covering: NO_2 distribution (2019–2024 average, 30 countries), a 2019-vs-2024 before/after comparison, the treatment/control study-design map, dominant land cover class (EU-27), NDVI distribution, GDP (log-scale, 30 countries), and the event-study robustness-check plot. Each map was independently verified for correctness via a structured, checklist-based verification workflow before finalization. QGIS was deliberately not used for map production, in favor of a fully scripted, reproducible Python pipeline consistent with the project's automation-first design.

MODULE 11 — Dashboard & Deployment ✅ COMPLETE

The complete project was packaged into an eight-page interactive Streamlit dashboard (Home, Study Design, Environmental Data, Before/After, Economic Context, Causal Results, Methodology & Limitations, Explore Trends, About & Data), featuring an interactive country-level time-series explorer (Plotly), downloadable raw master dataset, and a full narrative walkthrough of the project's validation journey (placebo test → control group → event study → honest null result). The complete codebase was published as a public, open-source GitHub repository, and the dashboard was deployed to Streamlit Community Cloud, producing a permanent public URL. A path-resolution bug (relative image/data paths resolving incorrectly under Streamlit Cloud's different working-directory structure compared to local execution) was identified and fixed across all dashboard pages during deployment.

PROJECT STATUS: COMPLETE

All eleven planned modules have been executed, with Module 9 formally and transparently scoped out for a specific, documented scientific reason rather than left incomplete. GPIE is deployed as a public GitHub repository and a live, interactive Streamlit dashboard, ready for academic and portfolio presentation.

==========

Policy Database Construction (Days 1–2)

Before any Earth Observation data could be collected, the first working component of GPIE needed to be a structured database of the policies themselves. The following two days document the process of building an automated web-scraping and data-processing pipeline to extract, structure, and analyze European Green Deal

policy records directly from EUR-Lex — starting from a single test request and progressing to a fully modular, reusable Python system producing a clean, analysis-ready dataset (`documents.json` / `documents.csv`).

Day 1

- Verified EUR-Lex API availability.
- Tested Python requests library.
- Successfully sent first HTTP request.
- Received HTML response from EUR-Lex.
- Installed and tested BeautifulSoup.
- Parsed HTML successfully.
- Extracted title, h1 tag and href attribute.
- Started understanding HTML structure for future web scraping.

Day 2

- Continued exploration of the EUR-Lex website HTML structure using BeautifulSoup.
- Identified and extracted multiple policy document containers from the search results page.
- Implemented automated iteration through all search results using Python `for` loops.
- Successfully extracted policy document titles from each search result.
- Extracted relative document URLs (`href`) for every policy document.
- Extracted legal status information (e.g., "In force") for each document.
- Learned to navigate nested HTML elements using chained BeautifulSoup methods.
- Organized extracted information into structured Python dictionaries.
- Stored all extracted policy records in a Python list (`documents`).
- Successfully built the first structured in-memory dataset containing multiple European Green Deal policy documents.
- Verified successful extraction of the first ten policy documents from the EUR-Lex search results.
- Used Python's `pprint` module to inspect the structured dataset in a readable format.
- Completed the first end-to-end multi-record web scraping workflow for the Green Policy Intelligence Engine.
- Exported the structured policy dataset to a reusable JSON file (`documents.json`).
- Successfully generated the first machine-readable dataset for the Green Policy Intelligence Engine.
- Verified that all extracted records were correctly serialized into JSON format.
- Established the initial data persistence layer for future analysis and automation workflows.
- Followed extracted policy URLs to access individual EUR-Lex policy pages automatically.
- Successfully sent HTTP requests to retrieve complete HTML content of individual policy documents.
- Saved the first policy page locally as `policy_page.html` for detailed HTML inspection and debugging.
- Explored the HTML structure of a full policy document using Visual Studio Code.
- Identified the `eli-main-title` container holding the official policy title and metadata.
- Successfully extracted the complete policy title directly from the individual policy page.
- Verified the transition from search-result scraping to detailed document-level scraping.
- Completed the first successful extraction from an individual European Green Deal policy document.
- Identified the HTML elements (`<p class="oj-doc-ti">`) containing the official policy heading, publication date, subtitle, and legal notes.
- Used `find_all()` to extract multiple related text elements from the policy document.
- Learned to iterate through extracted HTML elements using Python `for` loops.

- Combined multiple policy text fragments into a single structured string using string concatenation.
- Preserved line breaks in the extracted policy text using the newline character (`\n`).
- Created a reusable `policy_text` variable to store the complete introductory section of each policy document.
- Successfully extracted the introductory content for every policy page automatically.
- Extended the scraping workflow from metadata extraction to actual policy content extraction.
- Integrated the extracted `policy_text` into the structured policy dictionary for each record.
- Expanded the JSON dataset by adding a new `policy_text` field for every policy document.
- Verified that the extracted policy text was correctly stored alongside the title, link, and status.
- Removed duplicate debugging outputs to produce clean and readable terminal output.
- Validated that the final dataset contained both policy metadata and document text in a structured JSON format.
- Completed the first full document-level data extraction pipeline for the Green Policy Intelligence System.
- Successfully transformed raw EUR-Lex policy pages into a machine-readable structured dataset ready for downstream analysis and automation.
- Computed the character length of each extracted policy using Python's built-in `len()` function.
- Created a new `policy_length` metadata field representing the size of every policy introduction.
- Successfully integrated `policy_length` into the structured JSON dataset.
- Identified the policy category (e.g., REGULATION, DECISION) directly from the extracted policy text.
- Automatically extracted the policy type using Python string processing (`split()`).
- Added a new `policy_type` field to every policy record in the dataset.
- Parsed the publication year from the official policy identifier embedded in the document heading.
- Successfully extracted and stored the policy year as a separate `policy_year` metadata field.
- Enriched every policy record with structured metadata derived from the document content rather than HTML attributes.
- Expanded each policy record to include seven structured attributes: title, link, status, policy type, policy year, policy text, and policy length.
- Improved the semantic richness of the Green Policy Intelligence dataset through automatic metadata extraction.
- Validated that all extracted metadata fields were correctly written to `documents.json`.
- Verified consistency of the structured dataset across all extracted policy documents.
- Produced the first analysis-ready policy intelligence dataset with standardized metadata and document content.
- Established the initial metadata engineering pipeline for downstream policy analytics, filtering, search, and AI-driven policy intelligence workflows.
- Calculated the total word count for each extracted policy using Python string processing.
- Added a dedicated `word_count` metadata field to every policy record.
- Parsed the unique CELEX identifier directly from the policy URL.
- Successfully extracted and stored the `policy_id` for every policy document.
- Implemented automatic extraction of the policy summary from the first line of the document heading.
- Added a `policy_summary` field to provide a concise preview of each policy.
- Standardized the dataset by introducing a `source` field identifying the data origin as EUR-Lex.
- Expanded the structured dataset to include policy identifier, summary, source, and word count metadata.
- Sorted all policy records in descending order based on publication year to prioritize the latest policies.

- Refactored the scraping workflow by removing temporary debugging outputs after successful verification.
- Cleaned the execution pipeline to produce structured datasets without unnecessary terminal logs.
- Verified the successful generation of all enriched metadata fields across every extracted policy record.
- Expanded the final dataset to include title, link, status, policy type, policy year, policy ID, policy summary, source, policy text, policy length, and word count.
- Prepared the enriched policy dataset for downstream export, visualization, analytics, and AI-driven policy intelligence applications.
- Imported Python's built-in CSV module to prepare the data export pipeline for future spreadsheet-based analysis and interoperability with GIS, analytics, and dashboarding tools.
- Successfully configured Python's `csv.DictWriter()` to automatically export the structured policy dataset into CSV format.
- Generated the first spreadsheet-compatible version of the European Green Deal policy dataset (`documents.csv`).
- Verified successful conversion of the JSON-based policy dataset into tabular CSV format.
- Established a reusable multi-format export pipeline supporting both JSON and CSV outputs.
- Added an automatically generated policy summary field extracted from the introductory section of each policy document.
- Successfully extracted concise policy descriptions suitable for previews, dashboards, and search results.
- Added a standardized source field (`EUR-Lex`) to every policy record for dataset provenance and traceability.
- Computed the total word count for each extracted policy introduction using Python string processing.
- Expanded the structured metadata by adding policy summary, source, and word count fields for every policy document.
- Implemented automatic keyword-based policy tagging using a rule-based classification approach.
- Normalized policy summaries to lowercase to enable case-insensitive keyword matching.
- Built the first rule-based policy intelligence engine capable of assigning thematic tags based on policy content.
- Successfully identified and tagged climate-related policies using automated keyword detection.
- Integrated the generated tags into the structured dataset as a new metadata field.
- Verified the correctness of the rule-based tagging workflow through terminal-based debugging and dataset inspection.
- Enhanced the dataset with semantic metadata in addition to descriptive metadata.
- Completed the first end-to-end metadata enrichment and semantic classification pipeline for the Green Policy Intelligence System.
- Produced the first analysis-ready policy intelligence dataset combining document metadata, textual content, structured attributes, and automatically generated thematic classifications.
- Successfully exported the enriched policy dataset into CSV format to enable compatibility with spreadsheet software, GIS platforms, and data analytics tools.
- Built the first structured CSV export pipeline using Python's built-in `csv` module and `DictWriter`.
- Automatically generated CSV column headers from the dataset structure to ensure consistent data export.
- Exported all policy records from the in-memory dataset into a tabular CSV file without manual formatting.
- Verified the successful creation and integrity of the generated `documents.csv` file.
- Imported the exported CSV dataset into a Pandas DataFrame for analytical processing.

- Successfully established the first Pandas-based data analysis workflow for the Green Policy Intelligence System.
- Verified the DataFrame structure by inspecting sample records using `head()`.
- Confirmed successful loading of all policy records and metadata fields into the DataFrame.
- Inspected the dataset schema using `info()` to validate column names, data types, non-null values, and memory usage.
- Performed the first descriptive statistical analysis of the policy dataset using Pandas `describe()`.
- Computed summary statistics including record count, mean, standard deviation, minimum, maximum, and quartile values for numerical policy attributes.
- Executed the first conditional DataFrame filtering operation by selecting policies published in the year 2024.
- Successfully retrieved a subset of the dataset using Boolean indexing based on publication year.
- Performed categorical filtering to isolate all policies of type `REGULATION`.
- Validated the transition from data collection to exploratory data analysis using structured Pandas operations.
- Established the foundational data exploration pipeline required for downstream policy analytics, visualization, machine learning, and geospatial integration.
- Displayed individual dataset columns to inspect complete categorical values stored within the structured policy dataset.
- Identified all unique policy categories using Pandas' `unique()` function to understand categorical diversity within the dataset.
- Computed the frequency distribution of policy types using `value_counts()` for quantitative category analysis.
- Calculated the average policy word count grouped by publication year using Pandas' `groupby()` aggregation.
- Performed grouped statistical analysis to compare average policy lengths across different policy categories.
- Applied multi-condition filtering to retrieve policies satisfying multiple metadata criteria simultaneously.
- Successfully transitioned from basic filtering to grouped analytical operations using the Pandas DataFrame.
- Generated the first statistical summaries from the structured Green Policy Intelligence dataset without manual calculations.
- Created the first visualization of the Green Policy Intelligence dataset using a bar chart representing policy type distribution.
- Verified the successful integration of Pandas with Matplotlib for automated data visualization.
- Established the initial exploratory data visualization workflow for identifying policy distribution patterns.
- Completed the first end-to-end exploratory data analysis (EDA) pipeline combining filtering, aggregation, statistical summarization, and visualization.
- Saved the first policy distribution visualization as a high-resolution PNG image for documentation and future reporting.
- Implemented automated figure export using Matplotlib's `savefig()` function.
- Organized generated visualizations into a dedicated project output directory (`outputs/plots`) following a structured project architecture.
- Generated a chronological policy publication distribution chart to visualize the temporal spread of the collected policy dataset.

- Applied index-based sorting to ensure year-wise visualizations followed the correct chronological sequence.
- Explored multi-dimensional relationships between policy publication year and policy type using Pandas `groupby()` operations.
- Computed grouped statistics to compare policy categories across different publication years.
- Constructed a cross-tabulation (pivot table) summarizing policy counts by publication year and policy type.
- Extended the analytical workflow from one-dimensional frequency analysis to multi-dimensional categorical analysis.
- Created the first grouped bar chart comparing policy types across publication years.
- Saved the grouped comparative visualization as a high-resolution research-quality PNG image.
- Established a reusable visualization pipeline integrating Pandas aggregation, Matplotlib plotting, automated figure export, and organized project outputs.
- Successfully completed the first comparative policy intelligence visualization workflow for the Green Policy Intelligence System.
- Generated aggregated summary statistics comparing average policy length and average word count across different policy types.
- Identified the longest and shortest policy documents through automated sorting based on textual word count.
- Exported the top three longest policy documents as a separate analysis-ready CSV dataset for downstream policy intelligence analysis.
- Refactored the web scraping workflow by introducing the reusable `fetch_page()` function, eliminating duplicate page retrieval logic.
- Successfully completed the first production-level code refactoring while preserving identical analytical outputs, establishing the foundation for a modular, maintainable, and scalable Green Policy Intelligence System architecture.

Day 3

- Continued production-level refactoring of the Green Policy Intelligence System to improve code modularity and maintainability.
- Extracted policy metadata generation into a dedicated reusable function (`extract_metadata()`), separating data extraction logic from the main scraping workflow.
- Centralized policy metadata generation including policy type, publication year, CELEX identifier, policy summary, policy length, and word count into a single reusable component.
- Implemented a dedicated `generate_tags()` function to encapsulate all rule-based thematic classification logic.
- Isolated automatic keyword-based policy tagging from the main workflow, improving readability and future extensibility.
- Refactored policy status extraction into a reusable `extract_status()` function to eliminate repeated HTML parsing logic.
- Introduced a dedicated `get_source()` function to standardize dataset provenance across all extracted policy records.
- Replaced duplicated metadata extraction code with reusable function calls throughout the policy scraping pipeline.
- Updated the document construction workflow to retrieve structured metadata directly from reusable dictionaries instead of recalculating individual attributes.

- Reduced code duplication by replacing repeated string-processing operations with centralized reusable functions.
- Improved separation of concerns by assigning individual responsibilities to dedicated functions rather than embedding all logic inside the main scraping loop.
- Preserved identical analytical outputs while significantly improving code organization and long-term maintainability.
- Established the first modular architecture for the Green Policy Intelligence System, preparing the codebase for future expansion into satellite data processing, geospatial analysis, and decision intelligence workflows.
- Encapsulated JSON export functionality into a dedicated reusable `export_json()` function.
- Encapsulated CSV export functionality into a dedicated reusable `export_csv()` function.
- Centralized dataset export operations to eliminate repeated file-writing logic from the main execution workflow.
- Introduced a reusable `load_dataframe()` function to standardize loading of structured policy datasets into Pandas.
- Refactored the complete policy scraping pipeline into a dedicated `scrape_policies()` function, making the data acquisition workflow fully reusable and self-contained.
- Consolidated website access, HTML parsing, metadata extraction, semantic tagging, and dataset construction into a single modular scraping component.
- Implemented a centralized `main()` function to orchestrate the complete execution workflow, including data collection, dataset export, analytical processing, and visualization generation.
- Adopted Python's standard program entry-point pattern using `if __name__ == "__main__":` to improve execution control and project structure.
- Removed remaining duplicated execution logic by routing the complete workflow through the centralized `main()` function.
- Improved the overall software architecture by separating data acquisition, data export, analytical processing, and application execution into clearly defined functional modules.
- Completed the first fully modular implementation of the Green Policy Intelligence System while preserving identical research outputs, datasets, statistical analyses, and visualizations.
- Successfully transformed the project from a sequential scripting workflow into a reusable, maintainable, and production-oriented software architecture suitable for future integration with Earth Observation datasets, GIS workflows, and environmental intelligence modules.
- Successfully executed the fully modular Green Policy Intelligence System through a centralized application entry point, validating the complete end-to-end policy intelligence workflow.
- Verified successful integration of automated policy acquisition, metadata extraction, semantic classification, structured data export, statistical analysis, and visualization generation within a unified software architecture.
- Confirmed reproducibility of analytical outputs by generating identical structured datasets, summary statistics, visualizations, and analysis-ready exports after complete architectural refactoring.
- Established a reusable software architecture capable of supporting future integration of Earth Observation datasets, GIS workflows, spatial analysis, environmental indicators, and decision intelligence modules without altering the existing policy intelligence pipeline.

Why This Phase Was Necessary (Days 1–2-3)

Before any satellite or geospatial processing could begin, GPIE needed a **policy-side dataset** — a structured record of what environmental policies actually exist, when they were introduced, and what they cover. Without

this, the Earth Observation data collected in later phases would have nothing concrete to be evaluated against.

- **EUR-Lex was chosen as the source** because it is the official, authoritative repository of EU legal and policy documents — any policy evaluation framework needs to trace back to a primary source, not a secondary summary, to remain scientifically defensible.
- **Web scraping (rather than manual collection) was necessary** because EUR-Lex does not offer a clean structured export for bulk policy metadata, and manual collection would not scale or be reproducible across hundreds of documents.
- **Structured metadata extraction (policy type, year, ID, tags, word count, etc.) was prioritized early** because these fields are what will later allow policies to be filtered, grouped, and matched against specific time windows and regions during the causal inference phase (Phase 4) — without this metadata, satellite data collected later would have no policy events to test against.
- **Exploratory Data Analysis (EDA) and visualization at this stage** served as a sanity check — confirming the dataset was complete, consistent, and free of structural errors before any time or resources were invested in the much heavier satellite data pipeline.
- **Code refactoring into reusable functions (`fetch_page()`, `main()`, etc.) was done before moving to Phase 3** because the same production-quality software architecture (modular, fault-tolerant, reusable) would be required for the satellite pipeline — establishing this pattern early meant Phase 3 could build on a proven foundation rather than repeating architectural decisions later.

In short: **Days 1–2-3 built the "policy" half of GPIE's core comparison** — government policy records — so that when the "independent verification" half (satellite data) was added in Phase 3 onward, there would already be a reliable, structured dataset of *what* to verify *against*.

==========

Earth Observation Database Development Methodology

Objective

To construct a standardized, research-grade Earth Observation database that will provide independent environmental evidence for evaluating the effectiveness of environmental policies. This phase establishes the complete geospatial data foundation required for all subsequent spatial analysis, environmental assessment, and policy evaluation workflows within the Green Policy Intelligence Engine (GPIE).

Study Area

European Union

The European Union serves as the demonstration region for validating the Green Policy Intelligence Engine using the European Green Deal as the first implementation case.

Temporal Extent

01 January 2019 – 31 December 2024

A standardized six-year study period will be adopted for all time-dependent datasets to ensure methodological consistency across the project. This temporal framework provides a baseline preceding the

European Green Deal announcement together with multiple years covering its implementation and early outcomes.

Dataset Inventory

The Earth Observation database will integrate the following datasets:

1. Sentinel-5P TROPOMI NO_2
2. Sentinel-2 NDVI
3. ESA WorldCover
4. Copernicus DEM
5. ERA5 Climate Reanalysis
6. WorldPop Population
7. Eurostat Regional Statistics
8. Eurostat GISCO Administrative Boundaries
9. EUR-Lex Green Policy Database (Completed in Phase 2)

Data Acquisition Strategy

All datasets will be collected directly from their respective official providers to ensure scientific reliability, reproducibility, and long-term accessibility.

Dynamic datasets (e.g., NO_2, NDVI, ERA5, Population, GDP) will be acquired for the complete study period.

Static datasets (e.g., DEM, Administrative Boundaries, Land Cover where appropriate) will be collected once and reused throughout the project.

Data Organization

All downloaded datasets will be organized using a standardized directory structure separating raw, processed, intermediate, and final outputs.

Each dataset will follow consistent naming conventions to facilitate automated processing, reproducibility, and future scalability.

Data Verification

Every acquired dataset will undergo quality verification before further processing.

- Verification will include:

1. Coordinate Reference System (CRS)
2. Spatial extent
3. Spatial resolution
4. Temporal coverage
5. Data completeness
6. Metadata validation
7. File integrity Only validated datasets will proceed to preprocessing.

Data Standardization

Following acquisition, all datasets will be standardized to ensure interoperability within a unified geospatial database.

Standardization procedures may include:

1. Coordinate reference system harmonization
2. Raster and vector format validation
3. Temporal organization
4. Metadata standardization
5. Consistent file naming
6. Dataset indexing
7. Preparation for automated processing

- No scientific analysis will be performed during this stage.

Automated Preprocessing

After successful validation, Python-based workflows will automate repetitive preprocessing tasks including dataset conversion, extraction, mosaicking, temporal aggregation, and preparation of GIS-ready outputs.

Automation will ensure reproducibility while minimizing manual intervention throughout the project.

Phase Deliverables

At the completion of Phase 3, the project will contain:

1. A validated multi-source Earth Observation database
2. A standardized geospatial database
3. GIS-ready environmental datasets
4. Policy database integrated with supporting environmental datasets
5. Fully organized and reproducible project data architecture
6. Analysis-ready datasets prepared for subsequent geospatial and statistical workflows

Expected Transition to Phase 4

Upon successful completion of Phase 3, the project will transition from data acquisition and preparation to geospatial analysis and environmental policy evaluation.

Phase 4 will focus on integrating policy information with Earth Observation datasets to quantify environmental change, identify spatial patterns, evaluate policy effectiveness, and generate reproducible geospatial decision-support outputs.

Methodology Lock

The methodology described above shall serve as the fixed implementation framework for Phase 3 of the Green Policy Intelligence Engine. Any future modifications shall be made only if they improve scientific validity,

reproducibility, or project objectives, and not merely due to implementation convenience. This ensures architectural consistency throughout the development of GPIE.

COPERNICUS AUTHENTICATION MODULE

Objectives

- Began implementation of the authentication workflow for the Copernicus Data Space Ecosystem (CDSE).
- Established the official authentication endpoint for secure API communication.
- Configured the project to use the CDSE OAuth2 password grant authentication mechanism.

Work Completed

- Integrated the official CDSE authentication endpoint into the project configuration.
- Defined the required authentication parameters:
  - grant_type
  - client_id
  - username
  - password
- Implemented a reusable authentication function to submit secure POST requests to the CDSE Identity Service.
- Configured request timeout handling to improve reliability during network communication.
- Designed the authentication workflow as an independent module to enable reuse across future components.
- Developed an isolated authentication testing workflow to validate server responses before integrating product search and download functionality.
- Verified the latest CDSE API documentation and aligned the implementation with the current authentication specifications.

Current Status

- Authentication request workflow implemented.
- Response validation mechanism prepared.
- Access Token extraction and validation will be completed in the next development stage.
- Successfully validated the authentication workflow by obtaining a valid CDSE Access Token from the Identity Service.
- Verified successful authentication through HTTP Status Code 200 responses.
- Confirmed secure communication between the application and the Copernicus Data Space authentication server.
- Integrated the authentication module with the product search workflow to enable authenticated API requests.
- Implemented automatic extraction of the Access Token from the JSON authentication response.
- Configured Authorization headers using the Bearer Access Token for authenticated communication with protected CDSE services.
- Established secure communication with the Copernicus OData Product Catalogue endpoint.
- Successfully executed the first authenticated product catalogue request.
- Verified successful retrieval of product metadata from the Copernicus catalogue.

- Confirmed correct parsing of API responses received in JSON format.
- Validated that the application can retrieve and display product metadata from the catalogue.
- Added configurable search parameters to support future implementation of filtered product searches.
- Defined project constants for satellite collection, product type, and temporal search range to improve code maintainability.
- Implemented initial API request parameter handling using query parameters for scalable catalogue searches.
- Configured result pagination through the OData `$top` parameter to limit the number of returned products during testing.
- Established the foundation for implementing advanced catalogue filtering based on satellite collection, atmospheric product type, acquisition date, geographic region, and processing level.
- Successfully completed the initial integration between Authentication, Authorization, and Product Catalogue communication modules.
- Prepared the product search module for implementation of advanced OData filtering and automated Sentinel-5P NO_2 product discovery in the next development stage.
- Implemented advanced OData filtering to retrieve Sentinel-5P products from the Copernicus Data Space Catalogue based on predefined search criteria.
- Configured collection-based filtering to restrict catalogue searches exclusively to the Sentinel-5P mission.
- Integrated temporal filtering using standardized study period boundaries (01 January 2019 – 31 December 2024) to ensure methodological consistency with the project framework.
- Implemented product name filtering to automatically retrieve atmospheric NO_2 datasets while excluding unrelated Sentinel-5P products.
- Successfully executed authenticated catalogue searches and verified correct communication between the authentication module and the Copernicus Catalogue API.
- Validated HTTP response status codes to confirm successful product retrieval before processing catalogue metadata.
- Parsed JSON catalogue responses and implemented automated extraction of returned product records for downstream processing.
- Implemented automated counting of retrieved products to verify catalogue query results during testing.
- Developed an iterative product metadata extraction workflow capable of processing multiple catalogue records without manual intervention.
- Extracted and validated essential metadata including Product ID, Product Name, Publication Date, Origin Date, Content Type, Content Length, and Geospatial Footprint.
- Verified successful retrieval of geospatial footprint polygons representing the spatial coverage of individual Sentinel-5P observations.
- Confirmed that the implemented filtering strategy successfully reduced catalogue responses to relevant NO_2 atmospheric products required for the Earth Observation database.
- Established a reusable metadata extraction workflow that will support automated product selection, validation, and download operations during subsequent stages of database development.
- Implemented the initial authenticated product download workflow using unique Product IDs returned by the Copernicus Catalogue API.
- Constructed dynamic download URLs by integrating catalogue-derived Product IDs with the official Copernicus download endpoint.
- Configured streaming-based file download requests to enable efficient handling of large Earth Observation datasets while minimizing memory usage.

- Implemented download status verification using HTTP response validation prior to initiating file transfer.
- Successfully verified secure communication with the Copernicus Download Service, confirming readiness for automated dataset acquisition in subsequent development stages.

Sentinel-5P TROPOMI NO_2 Acquisition Protocol (DS02) — FINAL LOCKED VERSION (v1.0)

Product Selection

- Dataset: Sentinel-5P TROPOMI NO_2 Level-2
- Product Version: RPRO (Reprocessed)
- Fallback Product: OFFL (Offline) only if RPRO is unavailable.
- NRTI products will not be used.

Reason: RPRO products provide the highest-quality historical retrievals using improved processing algorithms and are specifically designed for long-term environmental trend analysis. Since GPIE evaluates environmental policy effectiveness between 2019–2024 rather than real-time air quality, RPRO represents the scientifically preferred dataset.

Study Area

- European Union (Demonstration Case)

Reason: The European Green Deal serves as the first validation case for the globally transferable Green Policy Intelligence Engine (GPIE). The framework architecture remains applicable to future policy evaluations in any country or region.

Temporal Extent

- 01 January 2019 – 31 December 2024

Reason: A standardized six-year study period captures both pre-policy baseline conditions and multiple years following Green Deal implementation, enabling robust long-term environmental assessment.

Coordinate Reference System (CRS)

Raw Sentinel-5P Products

- EPSG:4326 (WGS84)

Processing

- EPSG:4326 (WGS84)

Final Outputs

- EPSG:4326 (WGS84)

Policy: No reprojection shall be performed unless a future analysis explicitly requires another projection (for example, equal-area calculations).

Reason: Sentinel-5P products are natively referenced in WGS84. Maintaining a single CRS throughout the processing pipeline eliminates unnecessary reprojection errors and preserves scientific reproducibility.

Spatial Discovery Strategy

Catalogue discovery shall use the standardized European Bounding Box (BBOX).

Bounding Box

MIN_LON = -31.5

MIN_LAT = 27.5

MAX_LON = 35.0

MAX_LAT = 71.5

Reason: Bounding Box discovery is recommended by ESA/CDSE because it provides efficient catalogue searches, minimizes API processing overhead, avoids complex polygon timeouts, and retrieves all orbital swaths intersecting the study area.

Download Strategy

- Download only Sentinel-5P Level-2 RPRO orbital swaths intersecting the European Bounding Box.
- Global Sentinel-5P products will not be downloaded.
- Products outside the study region will be excluded during downstream processing.

Reason: Restricting downloads to intersecting orbital swaths substantially reduces storage requirements and download time while preserving all observations required for environmental policy assessment.

Spatial Processing Strategy

European Bounding Box Search ↓ Download Level-2 RPRO Products ↓ HARP Level-2 → Level-3 Conversion ↓ Clip Level-3 Products using the Official European Union Boundary ↓ Generate Analysis-Ready Datasets

Reason: Searching with a Bounding Box maximizes API efficiency, while clipping after Level-3 generation preserves pixel integrity, avoids boundary artifacts, and produces datasets containing only the official European Union study area.

Spatial Resolution

Raw Level-2

≈ 5.5 × 3.5 km (native TROPOMI resolution)

Processing Grid

0.05°

Final Products

0.05°

Reason: A standardized 0.05° grid provides consistent spatial analysis across Europe while preserving appropriate scientific resolution for long-term policy evaluation.

Raw Data Acquisition Strategy

- Acquire Sentinel-5P Level-2 RPRO products throughout the complete study period.
- Products shall be collected as orbital swaths preserving the original satellite observations.

Reason: Monthly Sentinel-5P products do not exist. Level-2 orbital products preserve the complete observational record and maximize scientific reproducibility for downstream processing.

Batch Execution Strategy

-Data acquisition shall be executed using monthly batches, aligned with the project's month-wise pipeline lifecycle (download → process → raw-delete → next month). Each monthly batch retrieves all intersecting Sentinel-5P products for that calendar month.

Reason: Monthly execution matches the orchestration granularity implemented in run_pipeline.py and date_utils.generate_monthly_ranges(), allowing fault isolation and raw-data cleanup to occur once per month rather than requiring finer-grained weekly checkpointing.

Query Strategy

- Catalogue searches shall be executed sequentially throughout the study period using small temporal batches.
- Large multi-year catalogue requests will never be used.

Reason: Small temporal queries improve API stability, simplify failure recovery, reduce timeout risk, and align with Copernicus Data Space best practices for automated Earth Observation workflows.

Quality Control

Apply

qa_value ≥ 0.75

during preprocessing.

Reason: This is the official ESA-recommended quality threshold for tropospheric NO_2 analyses. It removes low-quality retrievals affected by clouds, snow/ice, and retrieval uncertainties, ensuring scientifically reliable atmospheric observations.

Processing Workflow

Downloaded Level-2 Products ↓ Quality Filtering (qa_value ≥ 0.75) ↓ Level-2 → Level-3 Conversion (HARP) ↓ Daily Level-3 Mosaics ↓ Monthly Composites ↓ Annual Composites ↓ Multi-Year Composite (when required)

Reason: This workflow preserves the complete observation record while producing standardized multi-temporal datasets suitable for long-term environmental policy evaluation.

Processing Platform

Primary Processing Engine

Python

Libraries

- requests
- HARP
- xarray
- rioxarray
- rasterio
- geopandas
- numpy
- pandas

Role of QGIS

- Visualization
- Quality Inspection
- Manual Validation
- Cartographic Outputs

QGIS shall not be used for the operational production pipeline.

Reason: Python enables fully automated, reproducible, scalable, and globally transferable processing workflows, whereas QGIS is reserved for visualization and validation.

Failure Recovery Strategy

Implement

State-Verified Differential Downloading

The pipeline shall

- verify existing files
- verify file integrity
- skip valid files
- automatically redownload missing files
- automatically replace corrupted files

Reason: Differential downloading minimizes unnecessary network usage, improves robustness against interrupted downloads, prevents duplicate downloads, and represents the industry standard for large-scale Earth Observation systems.

Raw Data Lifecycle

Download ↓ Verify File Integrity ↓ Preprocess ↓ Generate Level-3 Outputs ↓ Validate Outputs ↓ Archive or Remove Temporary Raw Products (when storage optimization is required)

Reason: The processing pipeline preserves reproducibility while minimizing long-term storage requirements.

Missing Data Policy

- Invalid pixels shall be stored as NaN.
- No interpolation shall be performed during preprocessing.
- Missing observations shall remain explicitly represented.

Reason: Preserving missing values maintains scientific integrity and prevents introduction of artificial atmospheric signals.

Output Products

Raw

NetCDF (.nc)

Intermediate

NetCDF (.nc)

Spatial Products

GeoTIFF (.tif)

Dashboard Products

Cloud Optimized GeoTIFF (COG)

Statistical Products

Parquet (.parquet)

Reason: Each format is optimized for its intended downstream application while maintaining interoperability.

Directory Structure

data/ └── earth_observation/ └── no2/ ├── raw/ ├── processed/ │ ├── daily/ │ ├── monthly/ │ ├── annual/ │ └── multiyear/ ├── statistics/ ├── metadata/ ├── logs/ └── final/

Reason: The hierarchy supports scalable automation, efficient storage management, and reproducible processing.

File Naming Convention

Raw ---> Raw files are stored using the original Copernicus product name as returned by the CDSE catalogue (e.g., S5P_RPRO_L2__NO2____...nc), preserving full traceability to the source product rather than applying a custom renaming scheme. Reason: Retaining the official product name avoids ambiguity, simplifies cross-referencing against the Copernicus catalogue, and eliminates the need for a separate naming-parity check between local files and source metadata.

Daily

NO2_DAILY_YYYY_MM_DD.tif

Monthly

NO2_MONTHLY_YYYY_MM.tif

Annual

NO2_ANNUAL_YYYY.tif

Multi-Year

NO2_MULTIYEAR_2019_2024.tif

Reason: A standardized naming convention simplifies automation and chronological indexing.

Metadata Policy

Every processed output shall contain metadata including

- Satellite
- Sensor
- Product Version
- Acquisition Date
- Processing Date
- CRS
- Spatial Resolution
- Bounding Box

- QA Threshold
- Pipeline Version
- Software Versions

Reason: Complete metadata ensures reproducibility, transparency, and future auditability.

Logging Policy

The pipeline shall automatically record

- Download Started
- Download Completed
- Processing Started
- Processing Completed
- Processing Duration
- Failed Downloads
- Retry Attempts
- Errors

Reason: Comprehensive logs simplify debugging, monitoring, and recovery.

Version Control Policy

Pipeline versions shall be documented.

Example

v1.0

↓

v1.1

↓

v2.0

Reason: Version tracking guarantees reproducibility across future framework improvements.

Framework Design Principle

- All acquisition parameters shall remain configurable rather than hard-coded.
- Study area, temporal extent, CRS, processing settings, output structure, and quality thresholds shall be controlled through centralized configuration files.
- The framework shall remain globally transferable without requiring modifications to the underlying software architecture.

Reason: GPIE is designed as a reusable global environmental policy evaluation framework. The European Green Deal represents only the first demonstration case, while the same architecture should support environmental policy assessment anywhere in the world through configuration changes alone.

Scientific Design Principle

Raw satellite observations shall never be modified.

All analyses shall be performed on processed derivative products while preserving the original Level-2 observations.

Reason: Preservation of original observations ensures scientific integrity, reproducibility, and future reprocessing capability.

Methodology Lock

The Sentinel-5P NO_2 acquisition protocol described above shall serve as the fixed implementation framework for DS02 within the Green Policy Intelligence Engine (GPIE).

Future modifications shall be introduced only when they demonstrably improve

- Scientific validity
- Reproducibility
- Scalability
- Software architecture

and never merely for implementation convenience.

Day 4

• Implemented the finalized European Bounding Box (WKT) within the Sentinel-5P OData catalogue search, enabling automated retrieval of orbital swaths intersecting the study area.

• Successfully validated the spatial query against the Copernicus Data Space Ecosystem API, confirming that the finalized Bounding Box returns Sentinel-5P Level-2 RPRO NO_2 products intersecting the European study region.

• Verified successful API communication (HTTP Status Code 200) following integration of the finalized spatial query.

• Retrieved and validated Sentinel-5P product metadata including Product ID, Product Name, Acquisition Time, ContentLength, and GeoFootprint information required for downstream automated processing.

• Examined the returned GeoFootprint metadata and confirmed that complete orbital footprint geometries are available for every retrieved Sentinel-5P product, providing the spatial information required for future processing and validation.

• Implemented a centralized configuration architecture (config.py) containing project metadata, study area parameters, temporal extent, Sentinel-5P product configuration, standardized European Bounding Box, API endpoints, download settings, and directory paths.

• Refactored the catalogue search module to consume centralized configuration parameters, eliminating duplicated configuration values across the acquisition workflow.

• Refactored the download module to consume centralized configuration parameters, establishing a unified configuration-driven acquisition pipeline.

• Verified successful integration between the configuration module, authentication module, catalogue search module, and download module.

• Implemented dynamic local file path generation using official Copernicus product names while preserving original dataset naming conventions for complete metadata traceability.

• Integrated the standardized Earth Observation directory structure into the automated download workflow, enabling consistent storage of Sentinel-5P products within the GPIE database architecture.

• Implemented automatic creation of required download directories prior to data acquisition, preventing filesystem errors during automated execution.

• Developed a reusable download utility module to support file verification and download management throughout the acquisition pipeline.

• Implemented automatic file existence verification before initiating downloads, enabling the workflow to detect previously downloaded Sentinel-5P products.

• Implemented file integrity verification using the official Copernicus ContentLength metadata, allowing comparison between expected remote file size and locally stored datasets.

• Implemented state-verified differential download logic, ensuring that verified products are automatically skipped while only missing or incomplete products proceed to the download stage.

• Implemented automatic corrupted-file handling by identifying incomplete datasets through file-size verification and preparing them for clean re-download during subsequent execution.

• Successfully established the complete modular software pipeline linking Authentication, Configuration Management, Dynamic Catalogue Search, Spatial Filtering, Metadata Retrieval, File Verification, Download Management, and Differential Download Logic into a unified Earth Observation acquisition framework.

• Completed the production-ready software foundation for scalable Sentinel-5P NO_2 acquisition, preparing the Green Policy Intelligence Engine (GPIE) for robust long-term automated Earth Observation data collection.

• Finalized the complete automated month-wise acquisition strategy for Sentinel-5P NO_2, where each monthly batch is independently downloaded, processed, validated, and cleared before proceeding to the next month, ensuring efficient storage utilization and uninterrupted long-term processing.

• Standardized the sequential monthly execution workflow consisting of catalogue discovery, differential downloading, preprocessing, validation, and controlled raw-data removal, enabling fully automated batch execution across the entire 2019–2024 study period.

• Finalized the temporary raw-data lifecycle strategy in which original Level-2 orbital products are retained only until successful processing and validation, after which temporary raw files are safely removed to minimize storage requirements while preserving processed analytical outputs.

• Verified the complete month-wise processing architecture to support scalable long-term Earth Observation acquisition under limited local storage constraints without compromising scientific reproducibility.

• Validated the complete operational workflow linking automated monthly acquisition, preprocessing, validation, cleanup, and continuation into a continuous production-ready processing pipeline suitable for multi-year Sentinel-5P datasets.

• Confirmed that the download architecture supports interruption-safe execution through differential downloading, allowing previously verified files to be skipped automatically while processing continues seamlessly from the last completed state.

• Finalized the overall execution strategy separating Data Acquisition and Data Processing into independent modular stages, allowing both components to be executed, tested, and maintained independently within the Green Policy Intelligence Engine (GPIE) software architecture.

• Locked the operational software design for scalable month-wise Earth Observation processing, establishing the implementation blueprint for the forthcoming automated Sentinel-5P NO_2 preprocessing pipeline.

• Finalized the production processing methodology for Sentinel-5P NO_2 based exclusively on ESA Reprocessed (RPRO) Level-2 products, ensuring scientific consistency throughout the complete 2019–2024 study period.

• Locked EPSG:4326 (WGS84) as the standardized coordinate reference system for raw products, intermediate processing, and final outputs, eliminating unnecessary reprojection during the preprocessing workflow.

• Locked a standardized 0.05° × 0.05° spatial processing grid for Level-2 to Level-3 conversion to provide consistent spatial resolution across all temporal analyses.

• Adopted the official ESA-recommended quality assurance threshold (qa_value ≥ 0.75) as the fixed quality filtering criterion for all preprocessing operations.

• Finalized the HARP-based processing workflow for automated Level-2 quality filtering, spatial binning, and Level-3 generation while preserving scientific reproducibility.

• Standardized the complete temporal aggregation workflow consisting of Level-2 orbital products, daily Level-3 mosaics, monthly composites, annual composites, and multi-temporal analytical products.

• Locked the official European Union study boundary as the final clipping geometry while retaining Bounding Box discovery exclusively for efficient catalogue searches.

• Standardized NaN as the universal representation for missing observations throughout the complete processing chain, avoiding artificial interpolation or replacement of missing values.

• Locked the processing platform architecture based on Python automation, with QGIS reserved exclusively for visualization, validation, and cartographic quality inspection.

• Standardized reproducible output generation using NetCDF for intermediate products and GeoTIFF for analysis-ready raster datasets, supported by comprehensive metadata and execution logging.

• Finalized the modular processing architecture separating validation, preprocessing, temporal aggregation, export, and quality-control components into independent software modules to maximize scalability, maintainability, and reproducibility.

• Locked the Sentinel-5P NO_2 processing methodology as the fixed implementation framework for DS02 within the Green Policy Intelligence Engine (GPIE), with future modifications permitted only when they demonstrably improve scientific validity, reproducibility, scalability, or software architecture.

• Successfully configured and validated the HARP scientific processing environment within the dedicated GPIE Python environment, resolving all dependency, interpreter, and dynamic library loading issues required for Sentinel-5P processing.

• Verified successful installation and execution of the HARP Python API by importing the library, validating the underlying HARP runtime, and confirming compatibility between the Python interface and HARP processing engine.

• Successfully inspected a representative Sentinel-5P ESA Reprocessed (RPRO) Level-2 NO_2 NetCDF product to validate dataset accessibility before initiating large-scale automated processing.

• Identified and documented the complete variable inventory contained within the selected Sentinel-5P Level-2 product, confirming the availability of all required atmospheric, geolocation, quality assurance, and auxiliary variables.

• Verified the presence of the primary scientific variable `tropospheric_NO2_column_number_density` together with its associated latitude and longitude coordinates required for subsequent spatial processing.

• Successfully validated the HARP `keep()` operation by extracting only the required variables (`latitude`, `longitude`, and `tropospheric_NO2_column_number_density`) from the original Level-2 product while preserving the native scientific measurements.

• Confirmed successful execution of the HARP import and extraction workflow using a single representative Sentinel-5P orbital product prior to batch automation.

• Successfully exported the extracted variables to CSV format as an intermediate validation output, confirming correct data accessibility and attribute extraction from the original NetCDF product.

• Verified that the extracted dataset retained the full spatial observation set from the original satellite orbit, demonstrating that no unintended data loss occurred during the inspection workflow.

• Confirmed that scientific measurement values were preserved in their native floating-point representation, including scientific notation and missing-value handling, without applying any preprocessing, filtering, aggregation, interpolation, or spatial clipping.

• Validated the end-to-end inspection workflow using a single Sentinel-5P Level-2 product, establishing a fully functional prototype that will serve as the foundation for automated multi-year processing across the complete 2019–2024 dataset.

Day 5 — Security Hardening & Pipeline Robustness

- Identified that authentication credentials were hardcoded in plaintext within `auth.py`, posing a security and reproducibility risk.
- Migrated credential management to a `.env` file using `python-dotenv`, removing all hardcoded secrets from source code.
- Added `.gitignore` to exclude `.env`, `__pycache__/`, raw `.nc` files, and raw data directories from version control.
- Refactored `get_access_token()` to return a clean access token string directly, rather than a full response object, simplifying downstream integration.
- Updated `search_products.py` and `download-no2.py` to consume the refactored authentication function, resolving `AttributeError` failures caused by inconsistent return types.
- Verified successful token retrieval end-to-end via `test_auth.py`, confirming HTTP 200 authentication against the CDSE Identity Service.
- Verified `search_products.py` independently, confirming correct retrieval of Sentinel-5P RPRO NO_2 product metadata for the January 2019 test window.

Day 5 — Download Pipeline Hardening

- Rebuilt `download_no2.py` to eliminate the single-point-of-failure behavior in which one failed product download halted the entire batch.
- Implemented per-file retry logic (3 attempts) with delay-based backoff to handle transient network failures.
- Implemented post-download file-size verification against Copernicus `ContentLength` metadata, in addition to the existing pre-download differential-download check.
- Refactored `download_product()` to return a list of successfully downloaded filepaths, enabling downstream modules to consume verified outputs directly.
- Conducted a live download test; confirmed correct differential-download behavior (skipping already-verified files) and correct retry behavior under manual interruption.
- Identified and removed one incomplete (interrupted) raw NO_2 file from the local dataset to prevent downstream corruption during processing.

Day 5 — (HARP Preprocessing)

- Converted the exploratory `extract_no2.py` script into a reusable, production-oriented function: `preprocess_file()`.
- Implemented the ESA-recommended quality assurance filter (`qa_value ≥ 0.75`) directly within the HARP operations string, aligning implementation with the locked DS02 methodology.
- Implemented 0.05° spatial binning within the same HARP operation chain, converting raw orbital swaths into standardized Level-3 grid products.
- Function returns the output filepath on success and `None` on failure, without raising unhandled exceptions, to support fault-tolerant batch execution.

Day 5 — Temporal Automation Layer

- Removed a duplicate import statement in `date_utils.py`.
- Implemented `generate_monthly_ranges()` in `date_utils.py`, producing ISO-formatted start/end date pairs for each calendar month across an arbitrary year range.
- Extended `config.py` with `STUDY_START_YEAR`, `STUDY_START_MONTH`, `STUDY_END_YEAR`, and `STUDY_END_MONTH` to define the full 2019–2024 study period, while preserving legacy

`START_DATE`/`END_DATE` variables for backward compatibility.

Day 5 — Module 6 Implementation (Execution, Cleanup & Orchestration)

- Rebuilt `run_pipeline.py` as a full month-wise orchestrator, implementing the locked lifecycle: download → preprocess → raw-file deletion → progression to next month.
- Implemented structured logging via Python's `logging` module, writing timestamped execution logs to a dedicated `logs/` directory in addition to console output, satisfying the project's Logging Policy requirement.
- Implemented fault isolation at the month level: a failed download stage logs an error and skips to the next month rather than halting the full pipeline.
- Implemented fault isolation at the file level: if preprocessing fails for a given file, the raw file is preserved (not deleted) to allow reprocessing, consistent with the Raw Data Lifecycle principle that raw observations are never deleted before successful validation.
- Created `test_pipeline_one_month.py` as an isolated single-month test harness to validate the full download-process-cleanup cycle prior to full-scale 72-month execution.

Day 5 — DS05 Implementation (Copernicus DEM GLO-30)

- Verified current data access pathway for Copernicus DEM GLO-30 via the public, authentication-free AWS Open Data S3 bucket (managed by Sinergise), confirming the dataset remains actively maintained.
- Implemented `download_dem.py`, including tile-name generation logic matching the official Copernicus Product Package naming convention.
- Implemented remote file-size verification via HTTP HEAD requests prior to download, enabling accurate skip-if-complete and corrupted-file-redownload logic consistent with the DS02 differential-download standard.
- Implemented per-tile retry logic (3 attempts) and graceful handling of non-existent (ocean) tiles via HTTP 404 detection.
- Conducted a live single-tile test download (Netherlands region, tile N50_E003); confirmed successful retrieval, correct file size (10.2 MB), and correct placement within the standardized `data/earth_observation/dem/raw/` directory structure, maintaining EPSG:4326 and bounding-box consistency with DS02.

Day 5 — DS09 Implementation (NUTS Administrative Boundaries)

- Implemented `download_nuts.py`, retrieving country-level (NUTS LEVL_0) boundaries as a single GeoJSON file from the official Eurostat GISCO distribution API.
- Verified successful download of the 2024 NUTS release at 1:20M resolution in EPSG:4326.
- Implemented `get_eu_country_list.py` to parse the downloaded NUTS GeoJSON and extract country identifiers.
- Identified that the raw NUTS dataset includes non-EU entities (EFTA members, candidate countries, and non-member states); implemented an explicit EU-27 ISO2-to-ISO3 mapping to filter the dataset to official EU member states only, including correct handling of Eurostat's non-standard "EL" designation for Greece.
- Verified successful extraction of exactly 27 EU member state ISO3 codes, establishing a reusable country-code list for cross-dataset use.

Day 5 — DS07 Implementation (WorldPop Population, Partial)

- Implemented `download_population.py`, consuming the EU-27 country list generated from DS09 to drive per-country data acquisition, establishing a cross-dataset dependency between boundary and demographic data layers.
- Implemented retrieval via the WorldPop REST API (`wpgp` project alias) for the verified 2000–2020 "Global 1" dataset, with FTP-to-HTTPS URL normalization.
- Implemented per-country differential-download logic and error handling.
- Scoped initial execution to the 2019–2020 period only, as these years are confirmed available in the verified dataset.
- Identified that full study-period coverage (2021–2024) requires integration with WorldPop's newer "Global 2" dataset (distributed via HDX/STAC), whose exact programmatic access pattern has not yet been verified and remains an open task for a future session.

Current Status — End of Day 5

- DS01 (Policy Database): Complete.
- DS02 (Sentinel-5P NO_2): Pipeline complete and hardened; full 72-month execution pending.
- DS05 (Copernicus DEM): Pipeline complete; single-tile verified; full bounding-box execution pending.
- DS07 (WorldPop Population): Partially complete (2019–2020 verified); 2021–2024 requires further API research.
- DS09 (NUTS Boundaries): Complete.
- Causal inference and economic analysis modules (DiD/Synthetic Control, Green ROI ranking): Not yet started.

Project Journal — DS05 Population Module

Methodology & Data Source

- **Dataset**: WorldPop "Global 1" gridded population estimates
- **Provider**: WorldPop (University of Southampton)
- **Access Method**: WorldPop REST API (`wpgp` project endpoint) — `https://www.worldpop.org/rest/data/pop/wpgp`
- **Format**: GeoTIFF (.tif), consistent with project-wide raster standardization
- **Spatial Resolution**: 100m (native WorldPop grid resolution)
- **CRS**: WGS84 (EPSG:4326), consistent with DS02/DS05 CRS Lock policy
- **Organizational Unit**: Per-country (not bounding-box based, unlike DEM) — data is distributed as one file per country per year

Temporal Scope

- Full study period requirement: 2019–2024 (per project's standardized six-year study window)
- Verified data availability via the WorldPop REST API is confirmed only for **2019 and 2020**, as these fall within the "Global 1" dataset (2000–2020 coverage)
- 2021–2024 falls under WorldPop's newer "Global 2" dataset (2015–2030), distributed through a separate system (HDX/STAC API) whose exact programmatic access pattern has not yet been verified
- **Decision**: Execution scope restricted explicitly to verified years (`years = [2019, 2020]`) to ensure the module runs reliably without introducing unverified or unrepresented data gaps

- 2021–2024 population acquisition remains an open task for a future session, to be resolved via direct Global 2/HDX integration

Cross-Dataset Dependency

- Population acquisition is driven by the EU-27 country list generated in the DS09 (NUTS Boundaries) module, establishing an explicit dependency between boundary data and demographic data layers
- Ensures consistent country coverage across all per-country datasets in the project

Implementation Details

- Implemented `download_population.py` with per-country, per-year download logic via `download_country_population()`
- Implemented differential-download logic: existing non-empty local files are skipped to avoid redundant downloads
- Implemented FTP-to-HTTPS URL normalization, converting WorldPop's legacy FTP file references (`ftp://ftp.worldpop.org.uk`) to HTTPS mirror URLs (`https://data.worldpop.org`) for compatibility with standard HTTP request handling
- Implemented graceful error handling at both the metadata-query stage and the file-download stage, ensuring failures for a single country/year do not halt batch execution across the full 27-country loop
- `download_all_eu_population()` iterates over all EU-27 countries for each verified year, logging progress as `[done/total]` for execution visibility

Design Principle

- No silent or implicit data substitution is performed for unavailable years; only years with verified, physically-downloadable source data are included in the execution scope, consistent with the project's broader emphasis on reproducibility and data provenance transparency

---

Project Journal — DS05 Copernicus DEM Module

Methodology & Data Source

- **Dataset**: Copernicus DEM GLO-30 (Digital Elevation Model)
- **Provider**: Copernicus Programme, distributed via AWS Open Data Registry (managed by Sinergise)
- **Access Method**: Public, authentication-free access via AWS S3 bucket — `https://copernicus-dem-30m.s3.amazonaws.com`
- **Format**: Cloud Optimized GeoTIFF (COG) (.tif)
- **Spatial Resolution**: 30m
- **CRS**: WGS84 (EPSG:4326), consistent with DS02 CRS Lock policy
- **Temporal Resolution**: Static (single acquisition, no repeat downloads required)
- **Organizational Unit**: 1°×1° tiles, following the official Copernicus DSM Product Package naming convention

Data Source Verification

- Verified via web search that the AWS Open Data S3 bucket remains actively maintained and publicly accessible, with no authentication required
- Noted that this AWS mirror reflects an earlier dataset version (last confirmed update ~March 2023), as certain third-party platforms (e.g., OpenTopography) have since migrated to sourcing directly from ESA for the most current version
- **Decision**: AWS mirror deemed acceptable for project use, as DEM is a static, slowly-changing dataset where version currency is less critical than for time-series data (e.g., DS02 NO_2); version/access-date to be noted in documentation for reproducibility

Spatial Extent Strategy

- Tile discovery performed by iterating over integer latitude/longitude steps spanning the project's standardized European Bounding Box (`MIN_LON`, `MIN_LAT`, `MAX_LON`, `MAX_LAT` from `config.py`), ensuring consistency with DS02's spatial extent
- Since the bounding box spans both land and ocean area, a significant number of tile requests are expected to return HTTP 404 (no tile exists over open ocean) — handled explicitly as an expected, non-error condition

Implementation Details

- Implemented `download_dem.py` with tile-name generation logic (`generate_tile_name()`) matching the official Copernicus naming convention (e.g., `Copernicus_DSM_COG_10_N50_00_E003_00_DEM`)
- Implemented remote file-size verification via HTTP HEAD requests (`get_remote_size()`) prior to download, enabling accurate differential-download logic without downloading files solely to check completeness
- Implemented local file-completeness verification (`is_complete_local_file()`) via byte-size comparison against the verified remote size
- Implemented per-tile retry logic (3 attempts, with delay) to handle transient network failures, consistent with the retry standard established in the DS02 download module
- Implemented explicit differentiation between download outcomes (`downloaded`, `skipped`, `not_found`, `failed`) with a summary report printed at the end of batch execution
- Corrupted or incomplete local files are automatically deleted and re-downloaded rather than silently skipped

Live Verification

- Conducted a single-tile live test download (Netherlands region, tile `N50_E003`)
- Confirmed successful retrieval: correct file size (10.2 MB), valid TIF file (thumbnail-readable), and correct placement within the standardized `data/earth_observation/dem/raw/` directory structure
- Full bounding-box execution (~1,500–3,000 tile checks across the full European extent) deferred to next high-bandwidth session, given the static, one-time nature of this dataset

---

Project Journal — DS03 NDVI: Planned Approach (Not Yet Implemented)

Status

Not yet started. Documented here as a planned methodology decision for a future session.

Original Plan vs. Revised Plan

- **Original plan**: Download raw Sentinel-2 tiles and calculate NDVI locally, or use the Sentinel Hub Statistical/Process API to compute NDVI on-the-fly from raw bands.
- **Problem identified**: Raw Sentinel-2 tiles are extremely large (500MB–1GB per tile), making full EU/6-year coverage impractical in terms of storage. The cloud-computation alternative (Sentinel Hub API) requires a separate OAuth client setup (distinct from the existing CDSE credentials), introduces monthly Processing Unit (PU) quota management, and adds a new layer of authentication complexity beyond what the rest of the pipeline uses.
- **Revised plan**: Use a **pre-computed NDVI product** instead of computing NDVI from raw bands.

Rationale for Using Pre-Computed NDVI

- NDVI is a standardized, formula-based index — (NIR − Red) / (NIR + Red) — so a pre-computed NDVI product from an official source is scientifically equivalent to computing it manually from raw bands. There is no loss of methodological control, unlike with the project's primary variable.
- This decision was evaluated specifically against the project's core NO_2 methodology, where raw-to-processed control is essential (QA thresholds, custom spatial binning) because NO_2 is the primary variable used for causal inference and government-outcome verification. NDVI, by contrast, functions as a supporting/control variable in the broader analysis, so using a standardized, pre-validated product does not weaken the project's core scientific contribution.
- Conclusion: pre-computed NDVI is an acceptable and defensible substitution; it does not compromise the "Trust but Verify" research design, which centers on NO_2.

Selected Data Source

- **Dataset**: Copernicus Global Land Service (CGLS) NDVI, 300m resolution, 10-daily composites, Version 3 (covering 2014–present; earlier versions 1 and 2 are superseded and should not be used)
- **Provider**: Copernicus Land Monitoring Service (CLMS), distributed via the dedicated portal at `land.copernicus.eu`
- **Format**: NetCDF/GeoTIFF, consistent with the project's raster format standards
- **CRS**: WGS84 (EPSG:4326), consistent with the project-wide CRS Lock policy

Access Path Investigated and Rejected

- An alternative access route was identified: the same CGLS NDVI collection (`CLMS_NDVI_GLOBAL_300M_10DAILY_V3`) has recently become available through the Copernicus Data Space Ecosystem (CDSE), as part of an ongoing CLMS-to-CDSE migration.
- This route was rejected for immediate use because the collection is currently exposed only via the **openEO API**, not the standard OData Products catalogue already used for DS02 (NO_2). openEO uses a fundamentally different request structure (JSON-based process graphs rather than simple filtered queries) and would require building a separate client integration.
- Critically, this pathway would not fail loudly if used incorrectly — a mismatched implementation would return empty results silently for every query rather than raising an error, risking undetected data loss during time-constrained field execution. This route is deferred until it can be properly researched and tested, since a new API architecture should not be first tested on a bulk production run.

Planned Access Method

- The CGLS direct distribution portal will be used instead, as it is a long-standing, independently documented distribution path that does not depend on the newer CDSE/openEO migration.
- This path requires a one-time manual step: free account registration at `land.copernicus.eu/global`, followed by generation of an API token via the account dashboard (a comparable manual step to the OAuth client creation that would otherwise have been required for Sentinel Hub).
- Following registration and token generation, programmatic bulk downloading is expected to be possible via CLMS's Machine-to-Machine (M2M) API and/or its published manifest-file system (text files listing available product files per time period, intended for automated batch downloading).

Explicitly Deferred / Not Verified

- The exact M2M API request format, authentication header structure, and download URL pattern have not yet been verified against official documentation.
- A candidate implementation was drafted externally but was found to rely on an unverified, likely incorrect base URL and contained no functioning download logic; it was discarded rather than used, to avoid introducing an untested dependency into a live field session.
- Before implementation, the official "How to download data through the M2M API" guide (CLMS documentation) must be read in full and the request format tested against a small sample before scaling to the full 27-country, 6-year batch.

Next Steps (Future Session)

1. Register for a free account at the CGLS portal.
2. Generate an API token via the account dashboard.
3. Read and verify the M2M API documentation for exact request/authentication format.
4. Build and test `download_ndvi.py` against a small single-file sample before scaling to full batch execution.
5. Integrate into `run_pipeline.py` following the same differential-download and retry patterns established for DS02/DS05.

Project Journal — DS08 Eurostat Regional GDP Module

Methodology & Data Source

- **Dataset**: Gross Domestic Product (GDP) at current market prices, by NUTS 2 region
- **Official Dataset Code**: `nama_10r_2gdp`
- **Provider**: Eurostat
- **Access Method**: Eurostat REST Statistics API (JSON-stat dissemination format), publicly accessible without authentication
- **Format**: JSON (JSON-stat structure), saved locally as `.json`
- **Spatial Unit**: NUTS 2 administrative regions (basic regions for regional policy application), consistent with the DS09 NUTS Boundaries module
- **Temporal Resolution**: Annual

Verification Process

- Confirmed the current, active Eurostat REST API endpoint structure via official Eurostat API documentation: `https://ec.europa.eu/eurostat/api/dissemination/statistics/1.0/data/{DATASET_CODE}`
- Verified the exact dataset code (`nama_10r_2gdp`) against multiple independent sources (official Eurostat data browser, EU regional statistics publications, and third-party citations), rather than assuming a code from memory, to avoid the type of silent-failure risk identified during the DS03 NDVI investigation

Design Rationale — Single Bulk Request vs. Per-Country Looping

- Unlike DS07 (Population), which required per-country iteration due to WorldPop's per-country file distribution model, the Eurostat API returns data for **all NUTS regions in a single request**
- This eliminates the need for a country-code-driven loop (and therefore a dependency on the DS09 country list), simplifying the module to a single API call covering the full EU dataset
- This reflects a broader project pattern: acquisition strategy is adapted to each provider's native data structure rather than forcing a uniform per-country approach across all datasets

Temporal Scope

- Query filtered to the project's standardized study period (2019–2024) using the Eurostat API's built-in `sinceTimePeriod` and `untilTimePeriod` filter parameters, avoiding the need to download and locally filter the full historical time series
- This is consistent with the project's general principle of minimizing unnecessary data transfer where server-side filtering is available (as also applied in DS02's temporal batching and DS05's spatial bounding-box discovery)

Implementation Details

- Implemented `download_eurostat_gdp.py` with a single function, `download_regional_gdp()`, encapsulating the full request-and-save workflow
- Implemented basic differential-download logic: if a non-empty output file already exists for the given year range, the download is skipped rather than repeated
- Implemented graceful error handling for both non-200 HTTP responses and request exceptions, returning `None` on failure rather than raising an unhandled exception, consistent with the fault-tolerance pattern established across DS02, DS05, and DS07 modules
- Output is stored under `data/earth_observation/economy/raw/`, following the project's standardized per-dataset directory structure

Design Simplicity Note

- This module is intentionally the simplest acquisition script in the project to date: no authentication, no batching, no retry logic was deemed necessary, as a single well-scoped API request against a stable government statistics service carries substantially lower failure risk than large-scale satellite or per-country raster acquisition
- Retry logic and chunked/streaming download handling (as used in DS02, DS05, DS07) were deliberately omitted here, as the expected response size is small (tabular JSON, not large binary raster data) and does not require streaming

==========

Project Journal — DS04 ESA WorldCover Module

Methodology & Data Source

- **Dataset**: ESA WorldCover 10m 2021, Version 200 (v200)
- **Provider**: European Space Agency (ESA), produced by the WorldCover consortium; distributed via AWS Open Data (managed by VITO)
- **Parameter**: Global land cover classification (11 land cover classes), derived from Sentinel-1 and Sentinel-2 data
- **Access Method**: Public, authentication-free access via AWS S3 (`s3://esa-worldcover`, `eu-central-1` region), consistent with the auth-free access pattern already established for DS05 (DEM)
- **Format**: Cloud Optimized GeoTIFF (COG), consistent with project-wide raster format standards
- **CRS**: WGS84 (EPSG:4326), consistent with the project-wide CRS Lock policy
- **Native Tile Grid**: 3° × 3° tiles (distinct from DEM's 1° × 1° tile grid)

Version Selection Rationale

- Two versions exist: v100 (2020 map) and v200 (2021 map), generated using different underlying algorithms
- v200 (2021) was selected as it reflects an improved training methodology and higher validated overall accuracy (76.7% vs. 74.4% for v100), and represents the more current classification approach
- Noted for future documentation: because v100 and v200 differ algorithmically (not just temporally), any comparison between the two years would conflate real land cover change with algorithmic differences — this project uses a single version (v200) as its land cover baseline rather than attempting a direct 2020-vs-2021 change analysis from these two products

Verification Process

- Confirmed via official ESA WorldCover, AWS Open Data Registry, and Digital Earth Africa documentation that the dataset is hosted on a public, no-authentication-required S3 bucket
- Confirmed the tile naming convention includes a coordinate-based identifier (e.g., `N00E033`) corresponding to the 3°×3° grid cell, cross-referenced via independent third-party documentation (Microsoft Planetary Computer dataset examples)
- **Open verification item**: the exact full filename string format (constructed as `ESA_WorldCover_10m_2021_v200_{lat}{lon}_Map.tif`) was assembled from established ESA WorldCover naming conventions but has not yet been confirmed against a live server response, unlike DS05 where a single-tile download was already verified live. A single-tile test is planned as the first execution step before full bounding-box download, mirroring the verification approach used for DS05.

Spatial Extent Strategy

- Tile discovery adapted to WorldCover's native 3° grid (distinct from DEM's 1° grid), using floor-division logic to align arbitrary bounding-box coordinates to valid tile boundaries
- Reuses the project's standardized European Bounding Box (`MIN_LON`, `MIN_LAT`, `MAX_LON`, `MAX_LAT` from `config.py`), maintaining spatial extent consistency with DS02 and DS05
- As with DS05, the bounding box spans both land and ocean, so a subset of tile requests are expected to return HTTP 404 (no tile exists over open ocean); handled explicitly as an expected, non-error condition

rather than a failure

Implementation Details

- Implemented `download_worldcover.py`, structurally modeled on the DS05 (`download_dem.py`) module to maintain consistency in acquisition patterns across static/near-static datasets
- Implemented tile-name generation (`generate_tile_name()`) using floor-division to snap arbitrary coordinates to the nearest valid 3° tile origin
- Implemented differential-download logic via local file-existence and non-zero-size checks (simpler than DS05's remote-size-verification approach, as an initial baseline; byte-level completeness verification via HTTP HEAD requests noted as a possible future enhancement if partial-download corruption is observed in practice)
- Implemented per-tile retry logic (3 attempts) and explicit HTTP 404 handling for non-existent (ocean) tiles, consistent with the DS05 retry standard
- Implemented an execution summary (tiles checked, downloaded, skipped, not found, failed) printed at the end of batch execution, consistent with DS05's reporting format

Planned Verification Step (Not Yet Executed)

- Before full bounding-box execution, a single-tile test download is planned (e.g., the same Netherlands-region coordinates used to verify DS05) to confirm the filename pattern resolves correctly against the live S3 bucket
- If the single-tile test fails (e.g., HTTP 404 for a known-land tile, indicating an incorrect filename pattern), the naming convention will be corrected before attempting full-scale execution, following the same fail-fast, verify-before-scale principle applied throughout the project's acquisition modules

==========

Project Journal — DS06 ERA5 Climate Reanalysis Module

Methodology & Data Source

- **Dataset**: ERA5 Monthly Averaged Reanalysis on Single Levels
- **Provider**: Copernicus Climate Data Store (CDS), operated by ECMWF
- **Parameters**: 2m Temperature, Total Precipitation
- **Access Method**: CDS API (`cdsapi` Python client), a fundamentally different access architecture from all previously implemented datasets
- **Format**: NetCDF (.nc), consistent with the project's raster/scientific format standards
- **Spatial Resolution**: ~31 km (native ERA5 resolution)
- **Temporal Resolution**: Monthly averages (project's target aggregation level, avoiding the need for local hourly-to-monthly aggregation)

Architectural Distinction from Prior Modules

- Unlike DS02, DS05, DS07, DS08, and DS04 — all of which use direct HTTP GET requests against REST/OData/S3 endpoints — DS06 required migrating to an entirely separate system: the CDS API, which operates on an **asynchronous job-queue model** rather than direct file retrieval
- A request is submitted, queued server-side (`accepted` → `running` → `successful`), and the resulting file is only released once processing completes; wait times vary from seconds to several minutes depending on server load, in contrast to the immediate-response pattern of the project's other acquisition modules
- This required no custom polling logic to be written: the `cdsapi.Client.retrieve()` method handles the accept/run/complete lifecycle internally and blocks until the file is ready, simplifying the implementation despite the underlying architectural difference

Platform Migration Context

- Verified that the Copernicus Climate Data Store underwent a full infrastructure migration ("CDS-Beta") in 2024, meaning any pre-existing CDS account or credentials would not be valid; a new ECMWF account registration was required
- This was confirmed via official ECMWF/CDS migration documentation before implementation began, avoiding the risk of building against a deprecated system

Authentication Setup

- Unlike the project's other modules, which use either no authentication (DS05, DS08, initial DS04) or credentials read from a project-local `.env` file (DS02, DS07), CDS authentication requires a `.cdsapirc` file located in the user's home directory (`C:\Users\<username>\.cdsapirc`), a fixed location expected by the `cdsapi` library itself and not configurable from within the project
- This represents an intentional deviation from the project's otherwise centralized `.env`-based credential pattern, necessitated by the external library's hardcoded lookup behavior
- File contains two fields: the CDS API base URL and a personal access token, generated via the CDS account dashboard

Mandatory Terms of Use Acceptance

- Discovered that CDS enforces per-dataset licence acceptance as a precondition for API access; requests submitted without prior acceptance fail even with valid credentials
- Terms of Use (CC-BY licence) were accepted manually via the dataset's web interface (`Download` tab) before any programmatic request was attempted, as this step cannot be automated or bypassed via the API

Debugging Log

- Initial execution attempt failed with `Exception: Missing/incomplete configuration file: C:\Users\shobh/.cdsapirc`
- Root cause identified in two stages:
    1. The `.cdsapirc` file had initially been created in an incorrect directory (a display-name-based user folder rather than the canonical `C:\Users\<username>` path expected by the library)
    2. After relocating the file to the correct directory, the same error persisted; further diagnosis via direct terminal directory listing (`dir`) revealed the file had been saved with a hidden `.txt`

extension (`.cdsapirc.txt`) due to default text-editor save behavior, despite File Explorer not displaying the extension
- Resolved via terminal-based rename (`Rename-Item`) to strip the incorrect extension, after which authentication succeeded immediately on the next execution
- This debugging pattern (extension-hiding causing silent configuration failures) is consistent with an earlier incident in the same project (initial `.env` file creation), suggesting a recurring environment-specific risk worth noting for future file-based configuration steps

Implementation Details

- Implemented `download_era5.py` with `download_era5_year()`, requesting all 12 months of a given year in a single API call (rather than per-month requests), reducing the number of queued jobs relative to a more granular request pattern
- Implemented differential-download logic via local file-existence and non-zero-size checks, consistent with the pattern established across other acquisition modules
- Spatial extent constrained to the project's standardized European Bounding Box (converted to CDS's required `[North, West, South, East]` order), rather than requesting the "whole available region," to align with DS02/DS05 spatial consistency and avoid unnecessary global data transfer
- Output organized as one file per year (`era5_monthly_{year}.nc`) rather than one file per month, reflecting the dataset's low per-request data volume relative to DS02/DS05

Live Verification

- Conducted a live single-year test download (2019) prior to full-batch execution, consistent with the project's established fail-fast verification principle (as applied to DS05's single-tile test)
- Confirmed successful end-to-end execution: request accepted, processed server-side, and file downloaded to `data/earth_observation/climate/raw/era5_monthly_2019.nc`
- Noted that ERA5 monthly data volume is substantially smaller than DS02 (NO_2) or the anticipated DS03 (NDVI) volume, owing to its coarse spatial resolution (~31 km) and pre-aggregated monthly temporal resolution; full six-year batch execution (2020–2024, with 2019 skipped via differential-download) was assessed as low-risk for same-session execution.

==========

Project Journal — Day 6: DS04 Land Cover Processing (Complete Pipeline)

Status

Complete. Raw tiles successfully converted to analysis-ready, region-level land cover statistics.

Objective

To transform the 233 raw ESA WorldCover tiles (downloaded in the prior session) into a format usable for policy-evaluation analysis — specifically, land cover class percentages (forest, cropland, built-up, etc.) aggregated at the NUTS country level, rather than leaving the data as disconnected raw raster tiles.

Environment Setup — GDAL Installation

- Attempted to install GDAL via `pip install gdal`; this failed due to GDAL requiring compiled C++ binaries that pip cannot reliably build on Windows.
- Resolved by installing GDAL via `conda-forge` into the existing `gpie` conda environment (`conda install -c conda-forge gdal`), which provides pre-built binaries and avoids the compilation issue entirely.
- Discovered that VS Code's integrated PowerShell terminal does not automatically recognize `conda` commands or activate conda environments, even after setting the Python interpreter to the `gpie` environment via VS Code's interpreter selector.
- Adopted a reliable workaround: invoking the `gpie` environment's Python executable directly by full path (`& "$env:USERPROFILE\miniconda3\envs\gpie\python.exe" script.py`) for all subsequent script executions, rather than relying on `conda activate` or the `python` command working correctly in this terminal context.

Step 1 — Mosaic Construction (VRT)

- Implemented `process_landcover.py` to combine all 233 individual WorldCover tiles into a single logical mosaic using GDAL's `BuildVRT()` function.
- Deliberately used a **Virtual Raster (VRT)** rather than physically merging tiles into one large file: a VRT is a lightweight XML index file (confirmed at 128 KB) that references the original tile files and lets GDAL/QGIS/Python treat them as one continuous raster, without duplicating any pixel data or requiring additional disk space.
- This approach was chosen specifically to avoid the storage cost of physically merging 233 tiles of 10m-resolution data, which would have required a very large single file.

Step 2 — Rejected Approach: Full-Resolution Clipping

- Initially attempted to clip the VRT mosaic to the actual EU country boundary (from DS09 NUTS data) at full native 10m resolution, in order to remove the extra North Africa / ocean area included in the original satellite-orbit bounding box.
- This failed: GDAL reported that clipping at 10m resolution across the full European extent would require approximately 1.75 TB of disk space, exceeding available storage.
- The script did not correctly detect this failure — it proceeded to print a false success message despite the operation not producing an output file, due to a missing validation check on the `gdal.Warp()` return value.
- **Conclusion**: producing a full continent-scale clipped raster at native 10m resolution is not a practical goal, since country-level summary statistics do not require pixel-level precision at that scale. This step was abandoned in favor of a statistics-based approach (Step 3).

Step 3 — Rejected Approach: Zonal Statistics at Native/100m Resolution

- Attempted to compute per-NUTS-country land cover percentages directly via `rasterstats.zonal_stats()`, reading from the VRT without materializing a full clipped file — this is the correct general strategy (avoids the storage problem above).
- First attempt (at native 10m resolution) failed with a `TIFFReadEncodedTile` I/O error, traced to one specific corrupted tile (`ESA_WorldCover_10m_2021_v200_N45E012_Map.tif`) with incomplete pixel data, consistent with a prior known limitation: the WorldCover download module (DS04) does not

perform byte-level completeness verification the way DS02 and DS05 do. Resolved by deleting the corrupted tile and re-running the download script, which re-fetched only the missing file via its existing differential-download logic.
- Second attempt (still at native 10m resolution, after fixing the corrupted tile) failed with an out-of-memory error, because computing statistics for a large country still required loading a very large pixel array into RAM at full resolution.
- Attempted an intermediate fix: resampling the mosaic to 100m resolution before computing statistics. This reduced data volume substantially but still failed with an out-of-memory error on at least one large NUTS feature (likely a country with a wide bounding box, such as one including overseas territories), because 100m resolution over a very wide extent still produced an array too large for available memory.

Step 4 — Working Solution: 500m Resampling + Zonal Statistics

- Resampled the WorldCover mosaic to approximately 500m resolution using `gdal.Warp()` with **nearest-neighbor resampling** (`resampleAlg="near"`), explicitly not average or bilinear resampling.
- This resampling method was a deliberate scientific choice: WorldCover values are **categorical class codes** (e.g., 10 = Tree cover, 40 = Cropland, 50 = Built-up), not continuous measurements. Averaging or interpolating between class codes would produce meaningless intermediate values with no real-world interpretation; nearest-neighbor resampling preserves valid class codes at every pixel.
- 500m resolution was sufficient to keep memory requirements low enough for successful execution, while remaining far more than adequate for country-level percentage statistics (where sub-100m precision provides no additional analytical value).
- Applied LZW compression on output to further reduce file size.
- Computed zonal statistics via `rasterstats.zonal_stats()` with `categorical=True`, using the DS09 NUTS country-boundary GeoJSON as the zone geometry and the 500m resampled raster as the value raster.
- For each NUTS country, converted raw per-class pixel counts into percentages of total classified pixels, and mapped numeric WorldCover class codes to human-readable class names (Tree cover, Shrubland, Grassland, Cropland, Built-up, Bare/sparse vegetation, Snow and ice, Permanent water bodies, Herbaceous wetland, Mangroves, Moss and lichen) using the official WorldCover legend.
- Output saved as a single structured JSON file (`landcover_stats_by_country.json`) containing, for each NUTS country ID, a dictionary of land cover class percentages — a compact, directly analysis-ready format suitable for integration into the project's causal-inference workflow, in contrast to the raw raster tiles this pipeline started from.

Cleanup

- Deleted the intermediate, unsuccessful 100m resampled raster (`worldcover_2021_100m.tif`) after confirming the 500m version was the one actually used to produce the final statistics, to avoid retaining a non-functional intermediate artifact.

Final Pipeline Summary (DS04)

```
233 raw 10m tiles
    ↓ (VRT mosaic — no data duplication)
Single logical mosaic (128 KB index file)
    ↓ (resample, nearest-neighbor, 500m)
```

```
Compressed 500m raster
    ↓ (zonal statistics against NUTS boundaries)
landcover_stats_by_country.json
```

Design Principle Reinforced

- This session's repeated failures (disk space, corrupted tile, memory exhaustion) collectively reinforced a pattern already established elsewhere in the project (DS02, DS05): **resolution and processing scope should match the actual analytical requirement**, not the native resolution of the source data by default. Country-level policy analysis does not require pixel-perfect continental rasters; matching processing resolution to the analytical question (here, region-level percentages) is what made the pipeline computationally feasible on available hardware.

==========

Project Journal — Day 6: DS06 ERA5 Climate Processing (Complete Pipeline)

Status

Complete. Raw ERA5 files successfully converted to analysis-ready, monthly country-level temperature and precipitation statistics.

Objective

To transform the six raw ERA5 yearly download files (2019–2024, downloaded in the prior session) into a format usable for policy-evaluation analysis — specifically, monthly average temperature and precipitation aggregated at the NUTS country level — rather than leaving the data as raw gridded NetCDF files.

Step 1 — Discovering the Actual File Format (Zip-in-Disguise Issue)

- Attempted to open the first raw file (`era5_monthly_2019.nc`) directly with `xarray.open_dataset()`; this failed with an error indicating xarray could not identify a valid backend/format for the file.
- Diagnosed the root cause by testing the file with Python's `zipfile.is_zipfile()`, which confirmed the file was actually a **ZIP archive saved with a `.nc` extension**, not a genuine NetCDF file.
- Identified this as a known behavior of the newer CDS-Beta infrastructure (the same 2024 platform migration noted in the prior ERA5 acquisition session): even when `netcdf` output format is explicitly requested, CDS sometimes wraps the result in a ZIP container.
- Implemented `unzip_era5.py` to extract the actual NetCDF content from each yearly ZIP-disguised-as-`.nc` file into a corresponding `_extracted` subfolder.
- Discovered a second-layer complication upon extraction: each ZIP did not contain a single combined file, but **two separate NetCDF files**, split by internal GRIB `stepType`:
  - `data_stream-moda_stepType-avgua.nc` — containing the temperature variable (`t2m`)
  - `data_stream-moda_stepType-avgad.nc` — containing the precipitation variable (`tp`)
  - This split reflects a genuine technical distinction in the source GRIB data: temperature is an instantaneous-type field, while precipitation is an accumulated-type field, and CDS's GRIB-to-

NetCDF conversion process (`cfgrib`) apparently separates variables by this type when both are requested in a single request.

- Made `unzip_era5.py` idempotent and batch-capable: it checks whether an `_extracted` folder already exists before re-extracting (avoiding redundant work), and uses `glob` to automatically discover and process all six yearly files in one execution rather than requiring the year to be hardcoded, correcting an earlier version of the script that only ever processed 2019.

Step 2 — Merging and Unit Conversion

- Implemented `process_era5.py` to merge the two per-year variable files into a single combined dataset per year, and convert raw ERA5 units into human-interpretable units:
  - Temperature: converted from **Kelvin to Celsius** (ERA5's native unit is Kelvin, which is not directly interpretable for policy-relevant climate reporting)
  - Precipitation: converted from **meters to millimeters** (ERA5's native unit is meters of water equivalent, an unconventional unit for precipitation reporting; millimeters is the standard meteorological convention)
- Encountered a merge conflict (`MergeError: conflicting values for variable 'expver'`) when combining the temperature and precipitation datasets, caused by an internal CDS "experiment version" metadata marker (`expver`) that differed between the two source files.
- Resolved by explicitly dropping the `expver` coordinate from both datasets before merging, since this field is an internal CDS bookkeeping marker with no relevance to the project's climate analysis.
- Output: one combined, unit-converted NetCDF file per year (`era5_processed_{year}.nc`), each containing both `temperature_c` and `precipitation_mm` variables on the same spatial/temporal grid.
- Successfully processed all six years (2019–2024) in a single batch execution of `main()`.

Step 3 — Regional Aggregation (NUTS Country-Level Monthly Statistics)

- Implemented `era5_regional_stats.py` to aggregate the gridded climate data into per-NUTS-country, per-month summary statistics, following the same general strategy established in the DS04 (Land Cover) pipeline: raw gridded data is not analytically useful on its own for policy comparison, so it must be reduced to region-level statistics matching the spatial units used elsewhere in the project (NUTS boundaries, consistent with DS04, DS07, DS08, DS09).
- For each year and each of the 12 months within it, computed a country-level statistical mask using `rasterio.features.geometry_mask()` against each NUTS country polygon, applied directly to the ERA5 grid via a manually constructed affine transform derived from the dataset's latitude/longitude coordinate spacing.
- Computed **average temperature** (mean of masked grid cells) and **average precipitation** (masked-cell total divided by cell count) for each country-month combination.
- Skipped country polygons that produced an empty mask against the ERA5 grid (relevant for very small territories that may not intersect any grid cell at ERA5's coarse ~31km native resolution — an expected limitation of this dataset's resolution, not a processing error).
- Aggregated results across all six years into a single flat JSON structure, with one record per country-month combination, containing NUTS ID, month (YYYY-MM), average temperature (°C), and average precipitation (mm).
- Final output: `era5_stats_by_country_monthly.json`, containing 5,472 country-month records (consistent with approximately 27 countries × 12 months × 6 years, with some records naturally reduced by the small-territory grid-mask exclusion described above).

Why This Approach Was Chosen

- ERA5's coarse native resolution (~31km) made memory-related failures (of the kind encountered during DS04 Land Cover processing) unlikely, so no intermediate resampling step was required here — the full-resolution grid was small enough to process directly.
- Aggregating to monthly country-level statistics (rather than retaining daily or full-grid data) directly matches the temporal and spatial granularity needed for the project's planned causal-inference analysis, where climate variables are intended to serve as **control variables** — used to account for weather-driven variation in NO_2 or vegetation signals before attributing changes to policy effects, rather than as primary variables of interest requiring high spatial precision.

Final Pipeline Summary (DS06)

```
6 yearly raw files (ZIP-disguised-as-.nc)
    ↓ (unzip, split by stepType into t2m / tp files)
12 extracted NetCDF files (2 per year)
    ↓ (merge per year, drop conflicting metadata, convert units)
6 combined yearly files (era5_processed_{year}.nc)
    ↓ (zonal statistics against NUTS boundaries, per month)
era5_stats_by_country_monthly.json (5,472 records)
```

==========

Project Journal — Day 6: DS08 Eurostat GDP Processing (Complete Pipeline)

Status

Complete. Raw JSON-stat data successfully converted to a flat, analysis-ready CSV.

Objective

To transform the raw Eurostat regional GDP file (downloaded in an earlier session as a single JSON-stat response) into a simple, flat table — one row per region-year combination — usable for merging with the project's other datasets in future causal-inference analysis.

Step 1 — Understanding the Source Format

- Inspected the raw file structure via `inspect_eurostat.py` before writing any decoding logic, consistent with the project's established practice of verifying data structure rather than assuming it (as done previously for DS02, DS05, DS03).
- Confirmed the file follows the **JSON-stat 2.0** specification, a compact statistical data-exchange format used by Eurostat and other official statistics agencies.
- Identified the key structural components:

    - `dimension`: metadata describing each classification axis of the data (`freq`, `unit`, `geo`, `time`), each containing an ordered category index (e.g., mapping country/region codes to positions)
    - `id` and `size`: the fixed ordering of dimensions and the number of categories in each, which together define how multi-dimensional data is flattened
    - `value`: a dictionary of `{flat_index: value}` pairs, where `flat_index` is a single integer encoding the combination of all dimension positions (e.g., a specific region + a specific year), rather than nested per-dimension structures

Step 2 — Decoding the Flat Index Structure

- Recognized that JSON-stat's flat-index encoding is a space-efficient way of representing a multi-dimensional array as a 1D dictionary: for dimensions of sizes `(d1, d2, d3, d4)`, each valid combination of indices is mapped to a single integer via `((i1 × d2 + i2) × d3 + i3) × d4 + i4`, and this must be reversed to recover the original per-dimension indices.
- Implemented `process_eurostat.py` with `decode_jsonstat()`, which:
    1. Builds an ordered label list for each dimension by inverting the `category.index` mapping (label → position becomes position → label)
    2. For each entry in the `value` dictionary, decodes the flat integer index back into its per-dimension indices using successive modulo/floor-division operations, iterating dimensions in reverse order (matching the encoding convention)
    3. Uses the decoded `geo` and `time` indices to look up the actual region code and year label
- This approach avoids relying on any external JSON-stat parsing library, keeping the dependency footprint minimal and making the decoding logic fully transparent and auditable — relevant given the project's broader emphasis on reproducibility and understanding each processing step rather than treating any step as a black box.

Step 3 — Output Structure

- Flattened the decoded data into a list of records, each containing: `geo` (NUTS region code), `year`, and `gdp_million_eur` (the GDP value).
- Saved the result as a single CSV file (`gdp_by_country_year.csv`) using Python's built-in `csv` module, avoiding the need for additional dependencies (e.g., pandas) for this comparatively simple tabular-output task.
- Final output contained 18,470 records — substantially more than the number of EU-27 countries alone, because the source dataset spans **all NUTS levels (0, 1, and 2)** simultaneously (e.g., both country-level "DE" and sub-national regions like "DE11" appear as separate `geo` codes in the same flat structure), not country-level data only.

Why This Was the Simplest Module in the Pipeline

- Unlike DS02, DS04, DS05, and DS06 — all of which required raster or gridded-data handling, spatial masking, resampling, or memory-management strategies — DS08 required only tabular parsing and a lookup-table decoding operation, with no geospatial processing, no large data volumes, and no risk of the memory/storage failures encountered in the raster-based modules.
- No streaming, chunking, or resolution-reduction was necessary, consistent with the project's general principle (reinforced during DS04 processing) of matching processing complexity to the actual nature and scale of each dataset, rather than applying a uniform processing strategy across all modules.

Final Pipeline Summary (DS08)

```
Raw JSON-stat response (nama_10r_2gdp)
    ↓ (decode flat index using dimension metadata)
Per-record (geo, year, value) tuples
    ↓ (flatten to table)
gdp_by_country_year.csv (18,470 records)
```

==========

Project Journal — DS03 NDVI: Implementation Attempt — Complete Session Log

Status

Blocked. Authentication and dataset discovery fully functional; all download-request pathways tested against the CLMS M2M API have failed. Root cause identified as a structural limitation of the dataset itself, not a request-formatting error.

Prerequisite Steps Completed

- Registered a free account at the CGLS portal (`land.copernicus.eu/global`).
- Encountered a "Please fill in all required data" validation block on the user profile page; resolved by completing all required profile fields before proceeding.
- Generated a **service account key** via the profile's "API tokens" section — a JSON object containing `client_id`, `user_id`, `key_id`, a PEM-format private RSA key, and a `token_uri`, distinct in structure from the single-string bearer tokens used elsewhere in the project (e.g., DS06/CDS).
- Saved the service key as `clms_service_key.json` in the project root and added it to `.gitignore`, consistent with the project's established credential-handling practice.

Authentication Implementation — JWT-Based Flow

- Verified via official CLMS API documentation that access requires a four-step OAuth2 JWT-bearer flow, distinct from every other authentication method used elsewhere in the project (`.env`-based for DS02/DS07, home-directory `.cdsapirc` for DS06):
    1. Build a JWT containing claims `iss` (client_id), `sub` (user_id), `aud` (token_uri), `iat`, `exp`
    2. Sign the JWT using RS256 with the private RSA key from the service key
    3. Exchange the signed JWT for a short-lived access token via POST to `token_uri`
    4. Use the returned access token as a Bearer token for all further requests
- Installed `pyjwt` and `cryptography` to support JWT construction and signing.
- Implemented `auth_clms.py` with `get_clms_access_token()`, encapsulating this flow.
- **Confirmed working**: subsequent requests reached the server and returned data-validation errors rather than authentication errors, indicating the JWT exchange succeeded correctly.

Dataset Discovery

- Rather than guessing the dataset UID, implemented `find_ndvi_dataset.py` to search the CLMS catalogue programmatically via the `@search` endpoint.
- Confirmed two matching datasets; selected the non-superseded version:
  - **Selected**: "Normalised Difference Vegetation Index 2014-present (raster 300 m), global, 10-daily – version 3", UID `68a831a3eb7e4a568d3132ef71161387`
  - Rejected: Version 2, explicitly marked "SUPERSEDED" in its title
- Retrieved the associated `DatasetDownloadInformationID` (`e4662555-eb53-4e45-a3d2-45f6eb044d85`), required alongside the UID for any download request.

Download Request Debugging — Round 1: Request Format

- Implemented `download_ndvi.py` following the CLMS API's documented asynchronous task pattern (submit → poll → download), architecturally similar to DS06 but with additional required fields.
- **Error 1** (`HTTP 400: "BoundingBox is not valid"`): initial implementation submitted `BoundingBox` as a keyed object (`{west, south, east, north}`). Corrected against official documentation to the required flat array format `[North, East, South, West]`. Also corrected `TemporalFilter` dates from `YYYY-MM-DD` strings to milliseconds-since-epoch integers, per the same documentation.
- **Error 2** (`HTTP 400: "the requested BoundingBox is too big. The limit is 1600000000000"`): after fixing the format, the request reached a server-side area-size limit. The project's full European bounding box (sized for DS02's satellite-orbit discovery) exceeded this limit by a substantial margin.

Download Request Debugging — Round 2: Spatial Chunking

- Implemented `bbox_grid.py`, a reusable helper generating an evenly-spaced grid of sub-bounding-boxes across an arbitrary extent (`generate_bbox_grid()`), to split the oversized request into multiple smaller ones.
- Updated `download_ndvi.py` to loop over both temporal chunks (years) and spatial chunks (grid cells), submitting one request per combination, with per-cell output filenames to avoid collisions.
- Tested with a 5×5 grid (25 cells): still exceeded the area limit on the first cell, indicating the limit was substantially smaller than initially estimated.
- Tested with a 15×15 grid (225 cells): the area-limit error **no longer occurred**, confirming the limit had been correctly worked around through finer spatial chunking.

Download Request Debugging — Round 3: New Blocking Error

- With the area-limit issue resolved, a **new, different error** appeared: `HTTP 400: "this dataset is not downloadable"`.
- Hypothesized this might be specific to the `BoundingBox` restriction method, since CLMS documentation also describes an alternative, explicitly-supported restriction method using **NUTS country codes** (`"NUTS": "DE"` style parameter) rather than arbitrary bounding boxes.
- Pivoted `download_ndvi.py` to use NUTS-based requests instead of the bbox grid, iterating over the project's existing EU-27 country list (reusing `get_eu_country_codes()` from DS09/DS07, with an added ISO3→ISO2 mapping since CLMS's NUTS codes use the 2-letter convention).
- Tested against a single small country (Netherlands, `"NUTS": "NL"`): **identical error** (`"this dataset is not downloadable"`) was returned, ruling out the spatial-restriction method as the cause.

Root Cause Identified

- Since the error persisted regardless of spatial-restriction method (bounding box or NUTS code), inspected the dataset's download-information metadata directly via `inspect_ndvi_dataset.py`, querying the `@search` endpoint for the dataset's full `dataset_download_information` block.
- The response revealed two critical fields not previously visible: `"full_source": "CDSE"` and a `"byoc_collection"` identifier (`6303088f-3c19-4967-9038-119267c6d090`).
- **Conclusion**: this specific NDVI dataset is not physically hosted on CLMS's own infrastructure. It is a reference/proxy to a "Bring Your Own Collection" (BYOC) entry within the Copernicus Data Space Ecosystem / Sentinel Hub system. The CLMS website's `@datarequest_post` (M2M) endpoint — the pathway this entire session's implementation was built around — is fundamentally unable to serve this dataset, regardless of correctly-formatted requests, because CLMS itself does not hold the underlying data.
- This explains why the error was invariant across bounding-box format fixes, area-limit fixes, and the switch to NUTS-based restriction: none of these addressed the actual constraint, which is architectural rather than parameter-related.

Where This Leaves the Project

- The CGLS direct-portal M2M route — the access method selected in the prior session specifically to *avoid* Sentinel Hub/CDSE authentication complexity — has now been shown to not function for this dataset at all.
- The two remaining viable routes both lead back to the CDSE/Sentinel Hub ecosystem this project originally tried to route around:
    1. Sentinel Hub's own Process/Statistical API, using the `byoc_collection` ID directly — requires a separate Sentinel Hub OAuth Client (Client ID/Secret), distinct from both the CLMS service key and the CDSE credentials already in use for DS02, and introduces Processing Unit (PU) quota management.
    2. The openEO route identified (but deferred) in the prior session, which also resolves to CDSE infrastructure.
- Session ended at this diagnostic conclusion rather than proceeding into a fresh authentication setup, given the late hour and the fact that this represents a genuinely new sub-task (Sentinel Hub OAuth client creation) rather than a continuation of the current approach.

Required Methodology Update

The previously finalized DS03 methodology (prior journal entry) stated the CGLS direct-portal M2M route as the planned access method, explicitly chosen to avoid Sentinel Hub OAuth complexity. This is now confirmed **not viable** for the selected dataset (Version 3, 300m, 10-daily), and the methodology section should be revised to reflect:

1. **Access method must change** from "CGLS M2M API via `land.copernicus.eu`" to **Sentinel Hub Process/Statistical API**, using the `byoc_collection` ID discovered in this session (`6303088f-3c19-4967-9038-119267c6d090`).
2. **Authentication must change** from the JWT/service-key flow (`auth_clms.py`) to a **Sentinel Hub OAuth Client Credentials** flow (Client ID + Client Secret generated via the CDSE dashboard's Sentinel Hub OAuth Clients section) — this was the exact setup step the project deferred in an earlier session.
3. **A new constraint must be documented**: Sentinel Hub enforces monthly **Processing Unit (PU) quotas** (verified earlier as 40,000 PU + 10,000 openEO credits/month on the free tier), which does not apply to

the CGLS route and must now be budgeted for across the 27-country, 6-year request scope.
4. The `auth_clms.py`, `find_ndvi_dataset.py`, and NUTS/bbox-chunking logic in `download_ndvi.py` built during this session are **not wasted** — dataset UID/collection discovery remains valid — but the request-submission and authentication layers must be rebuilt against the Sentinel Hub API rather than the CLMS `@datarequest_post` endpoint.

Project Journal — DS03 NDVI: Sentinel Hub Implementation — Complete & Successful

Status

Complete. All 27 EU countries successfully acquired and processed for 2019–2024. This resolves the blocker documented in the prior session and fulfills the methodology update specified at that time.

Step 1 — Sentinel Hub OAuth Client Setup (Methodology Update, Point 2, Executed)

- Logged into the CDSE dashboard (`dataspace.copernicus.eu`) and located the Sentinel Hub OAuth Clients section under user account settings — a separate credential system from both the CDSE username/password (used for DS02) and the CLMS service key (used for the prior, unsuccessful NDVI attempt).
- Generated a new OAuth Client (Client ID + Client Secret), completing the exact setup step that the prior session's methodology update had flagged as required but not yet done.
- Added `SH_CLIENT_ID` and `SH_CLIENT_SECRET` to the project's existing `.env` file, keeping credential storage consistent with the project's established `.env`-based pattern rather than introducing a new storage mechanism.

Step 2 — Sentinel Hub Authentication Implementation

- Implemented `auth_sentinelhub.py` using the OAuth2 **Client Credentials** grant type (distinct from both the JWT-bearer flow used for CLMS and the password grant used for CDSE/DS02) — the simplest of the three authentication patterns encountered across the project, requiring only a single POST request with client ID and secret.
- Verified working via a standalone test execution, confirming successful token retrieval before building any request logic on top of it.

Step 3 — Choosing the Statistical API Over Raw Raster Download

- Rather than requesting raw NDVI GeoTIFF tiles (which would require a separate mosaicking/zonal-statistics pipeline, as was necessary for DS04 Land Cover), used Sentinel Hub's **Statistical API**, which computes zonal statistics (mean, min, max, standard deviation) server-side and returns them directly as JSON.
- This was a deliberate architectural choice: it collapses the "acquisition" and "processing" stages into a single step for this dataset, in contrast to every other raster dataset in the project (DS02, DS04, DS05), where acquisition and processing were necessarily separate due to the need to handle raw pixel data locally.
- Used the `byoc_collection` ID (`6303088f-3c19-4967-9038-119267c6d090`) discovered in the prior session's diagnostic investigation, confirming that the dataset-discovery work from that session was not

wasted, consistent with the prior methodology update's note (Point 4).

Step 4 — Evalscript Construction and Debugging

- Wrote a custom Sentinel Hub "evalscript" (the JavaScript-like function that defines what the API computes per pixel) requesting the `NDVI` and `dataMask` input bands, aggregated monthly (`P1M` interval) over each country's geometry.
- **Error 1**: `"Output dataMask requested but missing from function setup()"` — the evalscript's `setup()` function declared `dataMask` as an input but not as a corresponding output. Fixed by explicitly declaring both `ndvi` and `dataMask` as named outputs in `setup()`, matching Sentinel Hub's requirement that every input used in `evaluatePixel()` for masking purposes must have a matching output declaration when used with the Statistical API.
- After this fix, the request succeeded and returned valid statistics — the first successful NDVI data of any kind retrieved in the project.

Step 5 — Discovering and Correcting the Digital Number Encoding

- Initial successful results returned values in the range 0–250 (e.g., mean ≈ 160), which are not physically meaningful NDVI values (valid NDVI ranges from -1 to +1). Recognized this as a **digital number (DN) encoding** rather than raw physical NDVI — a common practice in satellite products to store continuous values as compact integers.
- Rather than guessing a conversion formula, retrieved and read the official CGLS NDVI 300m V3 Product User Manual (PDF, via `land.copernicus.eu` technical library) to find the documented scale factor and offset.
- Confirmed the official encoding: **real NDVI = (DN × 0.004) + (−0.08)**, and that DN values above 250 are reserved status flags (252 = unknown, 253 = snow, 254 = water, 255 = missing) rather than valid data, and must be excluded before applying the scaling formula.
- Verified the formula against the test output: a DN mean of 159.77 converts to (159.77 × 0.004) − 0.08 = 0.559, a physically realistic NDVI value for the Netherlands in January (moderate vegetation cover, consistent with winter conditions).
- Updated the evalscript to perform the DN-to-NDVI conversion and flag-value exclusion **server-side**, so that all data retrieved from this point forward is already in correct, physically meaningful units — avoiding the need for a separate post-processing conversion step, unlike DS06 (ERA5), which required a distinct unit-conversion stage after download.

Step 6 — Full Batch Execution and the France Anomaly

- Extended execution from the single-year, single-country test to the full scope: all 27 EU countries × 6 years (2019–2024) = 162 requests, iterating using the existing EU-27 country list (reused from DS09/DS07) and per-country NUTS geometries.
- **26 of 27 countries succeeded on the first full run.** France failed consistently across all six years with: `"Your request of 49944.87 meters per pixel exceeds the limit 26080.00 meters per pixel of the collection."`
- Diagnosed the cause: France's NUTS country geometry includes **overseas territories** (e.g., French Guiana in South America, Réunion in the Indian Ocean, Martinique in the Caribbean), which are geographically distant from mainland Europe. Because the Statistical API computes resolution based on the full extent of the requested geometry's bounding area, including these distant territories inflated

the effective area so much that the resulting per-pixel resolution exceeded the collection's maximum supported resolution.

- **Fix**: modified `load_country_geometry()` to intersect each country's NUTS geometry with the project's existing European bounding box (`config.py`'s `MIN_LON`/`MIN_LAT`/`MAX_LON`/`MAX_LAT`) using Shapely's `intersection()`, before submitting the request. This clips any overseas territory portions out of the geometry automatically.
- This fix was framed as consistent with the rest of the project rather than a one-off patch: every other dataset (NO_2, DEM, WorldCover) already operates exclusively within this same European bounding box, so restricting France's NDVI geometry to the same extent brings it into alignment with the project's existing spatial scope rather than introducing a special case.
- Re-tested France alone after the fix: succeeded, with realistic NDVI values (~0.51–0.57 for early 2019, consistent with the earlier Netherlands verification).
- Re-ran the full batch: **all 27 countries succeeded**, producing 162 total country-year records (27 × 6), confirmed by direct inspection of the output JSON (record count and country-code set).

Outcome Relative to the Prior Session's Open Constraint (Point 3)

- The PU (Processing Unit) quota constraint flagged in the prior session's methodology update was monitored throughout this execution; the full 162-request batch completed without any quota-exceeded error, indicating the request volume for this dataset's scope (country-level monthly statistics, not full-resolution raster tiles) remained well within the free-tier monthly allowance. No quota-management logic (e.g., request throttling or budget tracking) was ultimately required for this dataset's actual usage pattern.

Final Pipeline Summary (DS03)

```
Sentinel Hub OAuth Client (new credential type)
    ↓ (Client Credentials auth)
Per-country NUTS geometry, clipped to European bounding box
    ↓ (Statistical API request, evalscript with server-side DN→NDVI conversion)
Monthly NDVI statistics per country, 2019–2024
    ↓
ndvi_stats_test.json (162 records) — acquisition and processing complete in one
step
```

Note on File Naming

- The output file is currently named `ndvi_stats_test.json`, a holdover from the original single-country test script. Since this file now contains the complete, final 27-country batch output (not a test), it should be renamed to something reflecting its final status (e.g., `ndvi_stats_by_country_monthly.json`, matching the naming convention used for DS06's `era5_stats_by_country_monthly.json`) before this dataset is considered fully finalized in the project's file structure.

Project Journal — Day 6 (continued): Cross-Dataset Consistency Fix (EU-27 Scope Alignment)

Status

Complete. All four processed datasets (Climate, Land Cover, GDP, NDVI) now share a consistent EU-27, country-level scope, verified via direct record-count checks.

Motivation

- Rather than proceeding directly to further dataset acquisition/processing while download-dependent work (NO_2, DEM, Population) was blocked on connectivity, used the available time to validate that already-processed datasets could actually be merged together — a "dry run" of the eventual analysis-integration step, intended to surface structural mismatches early rather than discovering them during final causal-inference analysis.

Step 1 — Merge Compatibility Test

- Implemented `merge_test.py` to load all four processed datasets (Climate, Land Cover, GDP, NDVI) and inspect their shapes and `NUTS_ID`/`geo` code samples side by side.
- This immediately surfaced two structural inconsistencies that had not been visible when each dataset was processed independently:
    1. **Climate (ERA5) and Land Cover (WorldCover) outputs included non-EU entities** (e.g., Turkey `TR`, Ukraine `UA`, Kosovo `XK`) — because their zonal-statistics processing scripts had iterated over the *entire* NUTS boundary file (39 countries, including EFTA and candidate states), rather than the project's specifically-defined EU-27 list already used elsewhere (DS07 Population, DS03 NDVI).
    2. **GDP (Eurostat) output mixed multiple NUTS levels** (country-level `AL`, region-level `AL0`, `AL01`, etc.) in a single unfiltered column, and additionally — discovered in a later step — mixed multiple measurement units within the same country-year rows.
- Confirmed one apparent discrepancy (Liechtenstein `LI` present in Land Cover but absent from Climate) was **not a bug**: ERA5's coarse (~31km) native grid resolution means very small territories like Liechtenstein may not contain any grid-cell center, causing the zonal mask to be empty and the country to be legitimately skipped during DS06 processing — consistent with a limitation already documented in that dataset's journal entry.

Step 2 — Building a Reusable EU-27 Filter Utility

- Implemented `filter_eu27.py`, exposing `get_eu27_iso2_list()`, which reuses the existing `get_eu_country_codes()` function (originally built for DS09/DS07) and converts its ISO3 codes to the ISO2 format used by NUTS-based datasets, via the same mapping table already established in DS03/DS07.
- Built as a single shared utility specifically so that any future dataset requiring EU-27 filtering (rather than duplicating the country list and mapping across multiple scripts) can import and reuse it, consistent with the project's general pattern of reusing established components (e.g., DS07's dependency on DS09's country list) rather than re-deriving them per module.

Step 3 — Applying the Filter with Backups

- Implemented `apply_eu27_filter.py` to filter the Climate and Land Cover JSON outputs and the GDP CSV output down to EU-27-only records.
- Deliberately created a backup of each original (unfiltered) file (`_full.json` / `_full.csv` suffix) before overwriting, preserving the option to use broader NUTS-level or non-EU data in future analysis if

needed, while establishing the EU-27, country-level version as the default working dataset — consistent with the project's general principle (established during DS04 processing) of keeping intermediate/reference artifacts rather than discarding data that might later prove useful.
- Initial filtering run for GDP produced 1,134 records instead of the expected 162 (27 × 6), indicating an unresolved second issue beyond country scope.

Step 4 — Diagnosing and Fixing the GDP Unit-Mixing Issue

- Investigated the unexpected GDP record count by inspecting all rows for a single country-year combination (`DE`, 2019), which returned **seven different values** for the same country and year — ranging from 123 to over 3.5 million — immediately indicating that multiple measurement units were being conflated as if they were duplicate or comparable records.
- Traced the root cause back to `process_eurostat.py`'s original JSON-stat decoding logic (Day 6, DS08 session): the `unit` dimension had been decoded as part of the flat-index reconstruction but was never included in the output records, silently discarding information that turned out to be essential for correct filtering.
- Updated `decode_jsonstat()` to include the decoded `unit` label in each output record, and updated `save_csv()`'s hardcoded `fieldnames` list to match the new record structure — the initial fix attempt failed with a `ValueError` because only the record structure had been updated, not the CSV writer's expected field list, illustrating the importance of keeping both in sync when modifying a structured output format.
- Re-ran the corrected decoding process and inspected the resulting unit codes, identifying seven Eurostat unit variants: per-capita Euro (`EUR_HAB`), Million Euro (`MIO_EUR`), Million National Currency (`MIO_NAC`), and several Purchasing-Power-Standard (PPS) variants (absolute and per-capita, EU27-2020-referenced).
- Selected **`MIO_EUR`** (Million Euro, absolute/total GDP at current prices) as the project's standard GDP unit, on the basis that it is the most directly interpretable and universally comparable metric for the planned policy-impact analysis, rather than a per-capita or purchasing-power-adjusted variant.
- Updated `filter_gdp()` to filter on both EU-27 membership *and* `unit == "MIO_EUR"` simultaneously, and to rename/clean the output columns (`value` → `gdp_million_eur`, dropping the now-redundant `unit` column) so the final file's schema matches its original intended structure.

Debugging Note — Backup Restoration Error

- While attempting to re-apply the unit fix, mistakenly restored the GDP file from its `_full.csv` backup **before** re-running the fixed decoding script — this backup had been created *prior* to the `unit` column fix, so it did not contain the column needed for the new filtering logic, causing a `KeyError: 'unit'`.
- Resolved by recognizing that the correct fix was not to restore from any existing backup, but to simply re-run `process_eurostat.py` from the raw source JSON (which regenerates the full 18,470-record file with the corrected schema including `unit`), then re-run the GDP-specific filter against that freshly regenerated file.
- This is noted explicitly because it reflects a general lesson relevant to the project's backup practice: a backup only preserves data as it existed *at backup time* — if the processing logic itself changes afterward (as it did here, adding a new field), restoring an old backup can reintroduce the very problem the fix was meant to resolve, rather than simply undoing an unwanted filter.

Final Verification

- Re-ran the full filtering pipeline in corrected order: `process_eurostat.py` (regenerate from source) → `refilter_gdp.py` (apply EU-27 + `MIO_EUR` filter) → confirmed exactly 162 records (27 countries × 6 years), matching the expected scope.
- Final consistent dataset scope across all four processed modules:

Dataset	Records	Scope
Climate (DS06)	3,888	EU-27 × ~12 months × 6 years (Liechtenstein-equivalent small-territory gaps expected)
Land Cover (DS04)	27	EU-27, single 2021 snapshot
GDP (DS08)	162	EU-27 × 6 years, `MIO_EUR` only
NDVI (DS03)	162	EU-27 × 6 years

Design Principle Reinforced

- This session reinforced that **processing a dataset correctly in isolation does not guarantee it is usable in combination with other datasets** — scope (which countries), granularity (which NUTS level), and units (which measurement convention) must be explicitly aligned across all datasets before any cross-dataset analysis is attempted, rather than assumed to already match. Discovering and fixing this now, while only four datasets were involved, was substantially cheaper than discovering it later during full causal-inference analysis with all nine datasets combined.

Project Journal — Day 07: DS02 NO_2 — Methodology Switch to Sentinel Hub Statistical API

Status

Complete. Full acquisition finished for all 27 EU countries × 6 years (2019–2024) via the Sentinel Hub Statistical API. This supersedes the previously locked RPRO/HARP-based acquisition and processing pipeline for full-scale execution, while that original pipeline remains implemented and functional at small scale.

Context — Why the Switch Was Needed

- The originally locked DS02 methodology specified direct acquisition of Sentinel-5P Level-2 RPRO orbital products via the CDSE OData catalogue, followed by local HARP-based preprocessing (QA filtering, spatial binning) and month-wise batch orchestration (`run_pipeline.py`).
- This pipeline was fully built, hardened (retry logic, differential downloading, fault isolation), and verified correct at small scale (January 2019 partial test).
- At full scale, however, the raw acquisition volume proved impractical under real-world field conditions: approximately 55–60 orbital files per month × 72 months, each requiring separate download and HARP processing. Even at reasonable connection speeds, verification/skip-checking alone across previously-downloaded files consumed several minutes per run, and full-scale completion within the available time and bandwidth was not realistic.

- Rather than continuing to force the raw-acquisition approach, the decision was made to switch to the same **Sentinel Hub Statistical API** pattern already validated and proven successful for DS03 (NDVI) in the prior session — applying server-side quality filtering and returning aggregated statistics directly, without requiring bulk raw-file transfer.

Methodology & Data Source

- **Dataset**: Sentinel-5P TROPOMI Nitrogen Dioxide (NO_2), tropospheric column density
- **Access Method**: Sentinel Hub Statistical API, via the `sentinel-5p-l2` collection type
- **Authentication**: Sentinel Hub OAuth Client Credentials flow (`auth_sentinelhub.py`), reusing the same OAuth Client already created for DS03 (NDVI) — no new credential setup required
- **Output**: Monthly aggregated statistics (mean, min, max, standard deviation) per NUTS country, returned directly as JSON
- **Spatial Scope**: Each country's NUTS geometry intersected with the project's standardized European bounding box (reusing the clipping logic built for the DS03 France fix) before submission
- **Temporal Scope**: 27 EU countries × 6 years (2019–2024) = 162 requests, one per country-year, with monthly aggregation intervals (`P1M`) returned within each request

Implementation and Debugging

- Implemented `download_no2_sentinelhub.py`, structurally modeled on the DS03 NDVI Sentinel Hub implementation (`load_country_geometry()`, per-country/per-year request loop, evalscript-based server-side processing).
- **Error 1**: `"Collection 'S5PL2' has no band 'qa_value'"` — an initial evalscript attempted to reference quality-assurance values as an input band named `qa_value`, based on an unverified assumption. Verified against official Sentinel Hub Sentinel-5P L2 documentation that quality filtering for this collection is not a per-pixel band at all, but a **request-level parameter**: `processing.minQa`.
- **Key finding**: Sentinel Hub's documented default value for `minQa` on this collection is **75** — numerically identical to the project's independently-derived, ESA-recommended locked threshold (`qa_value ≥ 0.75`) from the original DS02 methodology. This confirmed that adopting the platform's standard default aligns exactly with the project's own scientific quality standard, requiring no compromise on the locked QA threshold.
- Corrected the evalscript to remove the invalid `qa_value` band reference and added `"processing": {"minQa": 75}` at the request level, alongside correcting the collection type identifier to `"sentinel-5p-l2"` (lowercase-hyphenated, per official documentation) rather than an earlier incorrect `"S5PL2"` guess.
- Re-tested against a single country (Netherlands, 2019): succeeded, returning physically realistic tropospheric NO_2 column density values (mean ≈ 1.9×10^{-4} mol/m^2, within the documented expected range for this parameter). Noted a high `noDataCount` relative to `sampleCount` in the response, confirmed as expected behavior given Sentinel-5P's orbital revisit pattern combined with strict QA filtering, not indicative of a processing error.

Full Batch Execution

- Extended execution to the full scope: all 27 EU countries × 6 years, iterating over the existing EU-27 country list (reused from DS09/DS07/DS03).
- **All 162 requests completed successfully** on execution, with no country-specific anomaly of the kind encountered for France in the DS03 NDVI batch (the same European-bounding-box geometry clipping,

already built into `load_country_geometry()` from the DS03 fix, was reused here and prevented the issue from recurring).
- Output saved as `no2_stats_by_country_monthly.json`.

Relationship to the Original Locked DS02 Methodology

- The original RPRO/HARP-based pipeline (`auth.py`, `search_products.py`, `download_no2.py`, `extract_no2.py`/`preprocess_file()`, `run_pipeline.py`) remains fully implemented, tested, and functionally correct at small scale. It is not deleted or invalidated — it represents a legitimate, higher-control acquisition path that remains available if a future need arises for raw Level-2 access (e.g., custom spatial binning at finer resolution than country-level statistics).
- The Sentinel Hub Statistical API route is understood to reflect the collection's standard processing level (OFFL-equivalent) rather than the specifically-locked RPRO reprocessed tier, since RPRO is not independently selectable via this API pathway. This is a deliberate, documented trade-off, consistent with the equivalent trade-off already accepted for DS03.
- This switch mirrors the DS03 precedent exactly: both datasets moved from a raw-acquisition-plus-local-processing model to a server-side statistical aggregation model, for the same underlying reason (full-scale raw acquisition impractical under real-world bandwidth/time constraints), using the same authentication infrastructure and the same spatial-clipping fix.

Final Pipeline Summary (DS02, Updated)

```
Sentinel Hub OAuth Client (shared with DS03)
    ↓ (Client Credentials auth)
Per-country NUTS geometry, clipped to European bounding box
    ↓ (Statistical API request, minQa=75 server-side filtering)
Monthly NO₂ statistics per country, 2019–2024
    ↓
no2_stats_by_country_monthly.json (162 records)
```

Project Journal — DS02 NO_2 EU-27 Filter Execution & Population Dataset Scope Decision

Status

Complete (for the parts that were runnable). EU-27 consistency filtering executed for the four remaining unfiltered/newly-relevant datasets (GDP, NO_2), with DEM filtering deferred pending download completion. Population's role in the project was also formally scoped down during this session.

Context — Starting Point

At the start of this session, the `filter_no2()` function had already been written (during a prior session, while the terminal was occupied running the DEM and Population downloads) but had never been executed. The objective was to finally run the full `apply_eu27_filter.py` script and bring NO_2 into EU-27-consistent scope alongside the four datasets (Climate, Land Cover, GDP, NDVI) filtered in earlier sessions.

Step 1 — Data Structure Verification

Before running the filter, the NO_2 output file (`no2_stats_by_country_monthly.json`) was inspected to confirm its field-naming convention matched the filter function's assumptions. This confirmed:

- 162 records present, matching the expected count (27 EU countries × 6 years).
- The country-identifier field is named `NUTS_ID`, consistent with the convention used across NDVI and other Sentinel Hub-derived datasets, validating that `filter_no2()` could be run without modification on this point.
- The NO_2 record structure was also observed to be deeply nested (raw Sentinel Hub Statistical API response format — monthly statistics nested within `data.data[].outputs.no2.bands.B0.stats`), unlike the flat structure of other processed datasets. This was noted as a required flattening step to be addressed later, ahead of the master dataset merge, but was not addressed in this session.

Step 2 — GDP Filter Failure and Correction

Running `apply_eu27_filter.py` surfaced two sequential errors in the existing `filter_gdp()` function, both caused by the function being out of sync with the current state of the GDP data file:

- **Error 1**: `KeyError: 'unit'` when filtering `df["unit"] == "MIO_EUR"`. Investigation (inspecting the CSV's actual columns) revealed that the GDP dataset had, in an earlier session, already been reprocessed from raw JSON-stat source data with the unit-mixing defect fixed at the source — meaning the output file no longer contained multiple units or a `unit` column at all, as it had already been standardized during processing rather than requiring a post-hoc filter.
- **Fix 1**: The unit-based filter condition was removed, retaining only the EU-27 country filter (`df["geo"].isin(EU27)`).
- **Error 2**: A second, related `KeyError: 'unit'` occurred on a subsequent `.drop(columns=["unit"])` line further down in the same function, which was still attempting to remove a column that no longer existed.
- **Fix 2**: This drop line was removed entirely.
- **Result**: GDP filter executed successfully — 162 → 162 records (no records dropped, confirming the dataset was already fully within EU-27 scope).

Step 3 — DEM Filter Deferred

Running the filter script surfaced a `FileNotFoundError` for `dem_stats_by_country.json`, confirming that DEM processing has not yet been executed, consistent with DEM acquisition (tile download) still being incomplete. The `filter_dem()` call was temporarily commented out of the script's execution block, to allow the remaining filter calls to run without being blocked by this dependency. This is a temporary, reversible change — the call will be uncommented once DEM acquisition and processing are complete.

Step 4 — NO_2 Filter Executed

With the GDP fix in place and DEM filtering deferred, the script was re-run successfully:

- **Climate**: 3,888 → 3,888 records (no change; already EU-27 scoped from a prior session).
- **Land Cover**: 27 → 27 records (no change; already EU-27 scoped).
- **GDP**: 162 → 162 records (fix applied this session, confirmed working).

- **NO_2**: 162 → 162 records (no records dropped — confirming that the underlying acquisition, which queried by EU-27 country list directly, had produced EU-27-scoped data from the outset; the filter step serves as a formal consistency confirmation rather than an active correction here).

NO_2 is now included in the project's EU-27-consistent dataset group, alongside Climate, Land Cover, and GDP.

Step 5 — Population Dataset: Scope Decision

Separately from the filtering work, the Population dataset's role in the project was formally reconsidered and scoped down:

- **Status confirmed**: Only 2019–2020 data has been successfully downloaded. The 2021–2024 range remains unresolved, as it was previously determined that the standard WorldPop access method used for this project does not serve those years, and completing that range would require integrating a different data source (WorldPop "Global 2" or HDX), which was deferred as out of scope for the time available.
- **Decision made**: Given that all other datasets in the project span the full 2019–2024 study period, and Population is limited to only 2 of those 6 years, Population will not be used as a control variable in the core Difference-in-Differences causal inference analysis (Module 8). Including a variable with 4 years of missing data in that model would risk distorting results rather than strengthening them.
- **Revised role**: Population is reclassified as a **supporting/descriptive dataset** — available for contextual reporting or supplementary description, but excluded from the core causal-inference model. The core DiD analysis will rely on NO_2 as the outcome variable, with Climate, Land Cover, and GDP as control variables, all of which have complete 2019–2024 coverage.
- This decision will be reflected in the Module Architecture document's description of Module 8 and of the Population dataset's status, to be updated at the next full module-status review.

Outstanding Items Carried Forward

- **DEM**: Acquisition (tile download) still incomplete; left downloading in the background. Once complete, `process_dem.py` must be executed, followed by uncommenting and re-running `filter_dem()`.
- **NO_2 nested structure**: Still requires flattening into a per-country-year-month flat format before it can participate in the master dataset merge; not addressed this session.
- **Population processing script**: Still not written. Given Population's revised supporting-dataset role, this remains lower priority than DEM completion.
- **Master merge script**: Not yet started; structure planning remains a pending next step once DEM is complete.

Development Log — NO_2 Flattening & December Data-Gap Fix

Context — Starting Point

With DEM downloading in the background and Population processing blocked on corrupted files, the NO_2 dataset (`no2_stats_by_country_monthly.json`) remained in its raw, deeply-nested Sentinel Hub Statistical API response format from the prior session's methodology switch. This structure — monthly statistics nested

within `data.data[].outputs.no2.bands.B0.stats` per country-year record — had been explicitly flagged as requiring flattening before it could participate in any future master dataset merge, since every other processed dataset in the project (Climate, Land Cover, GDP, NDVI) already existed in a flat, one-record-per-observation format. This session addressed that outstanding item.

Step 1 — Writing the Flattening Script

Implemented `flatten_no2.py` with a single function, `flatten_no2()`, which:

- Loads the raw nested JSON (162 country-year records).
- Iterates through each country-year record's list of monthly entries.
- Extracts the month number from each entry's `interval.from` ISO timestamp using `datetime.fromisoformat()`.
- Extracts the five statistical fields (`mean`, `min`, `max`, `stDev`, `sampleCount`, `noDataCount`) from the deeply nested `outputs.no2.bands.B0.stats` path.
- Writes out a new flat JSON file (`no2_stats_by_country_monthly_flat.json`), with one record per country-year-month combination, structured consistently with the flat format already used by ERA5 (Climate) and other processed datasets.

Step 2 — First Execution and Discrepancy Discovery

Ran `flatten_no2.py` for the first time. The script completed without error, but produced **1,782 flattened records** rather than the expected **1,944** (27 EU countries × 6 years × 12 months). This 162-record shortfall — coincidentally equal to the total number of country-year records — was immediately treated as a signal worth investigating rather than dismissed, consistent with the project's established practice of verifying record counts against expected totals before proceeding (as previously applied to GDP and NO_2 EU-27 filtering in earlier sessions).

Step 3 — Diagnosing the Pattern

Rather than guessing at the cause, wrote a diagnostic one-liner to group the flattened records by `(NUTS_ID, year)` and count how many of the 162 country-year combinations had fewer than the expected 12 monthly records.

Result: All 162 out of 162 country-year combinations had exactly 11 months, not 12 — a uniform, 100%-consistent shortfall rather than a scattered or random one. This ruled out an isolated data-quality issue (e.g., a few missing months for specific countries) and pointed instead toward a systematic, structural cause affecting every single request identically.

A second diagnostic — counting flattened records by month number across the entire dataset (1 through 12) — confirmed the exact nature of the pattern: months 1 through 11 each had exactly 162 records (one per country-year, as expected), while **month 12 (December) had zero records** across all 162 country-years. This eliminated any ambiguity about which month was affected and confirmed the issue originated at the acquisition stage, not in the flattening logic itself.

Step 4 — Root Cause Identification

Inspected the original acquisition script (`download_no2_sentinelhub.py`) and identified the source of the problem in the request payload's time-range parameters. Both the `dataFilter.timeRange` (which scopes the

raw satellite observations included in the request) and the `aggregation.timeRange` (which scopes the `P1M` monthly aggregation windows returned) were set as:

```
from: f"{year}-01-01T00:00:00Z"
to:   f"{year}-12-31T23:59:59Z"
```

This end-boundary — `December 31st, 23:59:59` — falls one second short of a complete December monthly interval under Sentinel Hub's `P1M` aggregation logic, which generates monthly intervals as half-open windows (`[start, end)`). A complete December interval requires an end-boundary of `January 1st, 00:00:00` of the following year; ending at `December 31st 23:59:59` caused the API to treat the December window as incomplete and omit it entirely from the response, rather than returning a partial or truncated December record. This behavior was consistent and deterministic across all 162 requests, explaining the perfectly uniform 11-of-12 pattern observed.

Step 5 — Fix Implementation

Modified both time-range blocks in `download_no2_sentinelhub.py`, changing the `to` boundary in each from:

```
"to": f"{year}-12-31T23:59:59Z"
```

to:

```
"to": f"{year + 1}-01-01T00:00:00Z"
```

This extends the requested window by one second in effect, but critically shifts the upper boundary to the start of the following year, ensuring the December `P1M` interval (`[December 1, January 1 next year)`) is fully enclosed within the requested range and therefore returned as a complete monthly aggregation.

Step 6 — Re-Acquisition and Verification

Re-ran the corrected `download_no2_sentinelhub.py` in full. All 162 requests (27 countries × 6 years) completed successfully, overwriting the previous `no2_stats_by_country_monthly.json` with the corrected dataset. Record count was confirmed at 162 country-year records, consistent with the original acquisition.

Re-ran `flatten_no2.py` against the corrected raw file. The flattening logic itself required no changes, since the fix was applied entirely at the acquisition stage — the flattening script simply processed whatever monthly entries were present in each country-year record, and with December now correctly included in the source data, the output naturally reflected the corrected structure.

Outcome

The corrected flattening run is expected to produce the full 1,944 records (27 × 6 × 12), with all twelve months represented for every country-year combination, resolving the systematic December gap and bringing NO_2

into full temporal consistency with the project's other monthly-resolution datasets ahead of the planned master dataset merge.

Design Principle Reinforced

This session reinforced a pattern already established earlier in the project (the GDP unit-mixing discovery, the DS03 France anomaly): **a dataset that appears successfully acquired (no errors, plausible record count) can still contain a systematic, silent gap that only becomes visible when record counts are explicitly checked against the expected total** rather than assumed correct from the absence of errors. The 100%-uniform nature of the December gap — rather than a partial or randomly-distributed one — was itself the key diagnostic clue that pointed to a structural, request-level cause rather than a data-availability issue, reinforcing the value of characterizing the *pattern* of a discrepancy, not just its existence, before attempting a fix.

Development Log — DEM Processing Through Master Dataset Completion

Status

Complete. DEM processed and EU-27 filtered; NO_2 and NDVI December-gap bugs identified and fixed; a critical ERA5 duplicate-timestamp bug discovered and resolved; full master dataset successfully assembled and verified clean.

Part 1 — DEM Processing

Context

With DEM tile acquisition essentially complete (859 of an estimated ~860 land tiles downloaded, following resolution of a slow-verification bottleneck in earlier sessions), the existing but previously unexecuted `process_dem.py` script was run for the first time.

Execution

The script executed successfully on the first attempt, with no errors — a notable contrast to the DS04 Land Cover processing pipeline, which had required multiple failed attempts (disk-space exhaustion, a corrupted tile, out-of-memory errors) before arriving at a working approach. The DEM pipeline benefited directly from that prior experience, having been designed from the outset with the same 500m-resampling strategy that Land Cover had converged on only after failure.

Three stages ran cleanly:

1. **Mosaic construction**: 860 raw DEM tiles combined into a single VRT (Virtual Raster) index, avoiding physical duplication of pixel data, consistent with the Land Cover approach.
2. **Resampling**: The VRT was resampled to 500m resolution using **bilinear** resampling — deliberately different from Land Cover's nearest-neighbor approach, since elevation is a continuous variable (unlike land cover's categorical class codes), making interpolation between values scientifically valid here.

3. **Zonal statistics**: Mean, min, max, and standard deviation of elevation were computed per NUTS country boundary, producing `dem_stats_by_country.json` with 39 records (covering all NUTS countries, not yet EU-27-filtered).

A minor `NodataWarning` appeared during zonal statistics computation (rasterstats defaulting to `-999` as an assumed nodata value), but this was assessed as inconsequential to data quality and not requiring correction.

EU-27 Filtering

The previously-deferred `filter_dem()` call in `apply_eu27_filter.py` — commented out in an earlier session pending DEM completion — was uncommented and the full filter script re-run. DEM filtered correctly from 39 to 27 records, joining Climate, Land Cover, GDP, and NO_2 as EU-27-consistent datasets.

Part 2 — NO_2 Flattening and the December Data-Gap Bug

Flattening

A new script, `flatten_no2.py`, was written to convert the NO_2 dataset from its raw, deeply-nested Sentinel Hub Statistical API response structure into a flat, one-record-per-country-year-month format consistent with the rest of the project's processed datasets.

Bug Discovery

The first flattening run produced 1,782 records rather than the expected 1,944 (27 countries × 6 years × 12 months). Rather than dismissing this as a minor discrepancy, a diagnostic check grouped records by `(NUTS_ID, year)` and found that **all 162 country-year combinations** had exactly 11 months present, not 12 — a perfectly uniform shortfall. A second diagnostic, counting records by month number across the full dataset, confirmed that **month 12 (December) had zero records** across every single country-year, while months 1–11 each had the full expected 162 records.

Root Cause

Inspection of the acquisition script (`download_no2_sentinelhub.py`) revealed that both the `dataFilter.timeRange` and `aggregation.timeRange` request parameters used an end-boundary of `f"{year}-12-31T23:59:59Z"`. Because Sentinel Hub's `P1M` monthly aggregation intervals are half-open (`[start, end)`), a complete December interval requires an end-boundary of `January 1st, 00:00:00` of the following year. Ending at `December 31st 23:59:59` caused the API to treat the December window as incomplete and silently omit it, rather than returning a partial record — explaining the perfectly uniform, deterministic nature of the gap.

Fix and Re-Acquisition

Both time-range end-boundaries were changed from `f"{year}-12-31T23:59:59Z"` to `f"{year + 1}-01-01T00:00:00Z"`. The full NO_2 acquisition (162 requests) was re-run successfully, and `flatten_no2.py` was re-executed against the corrected raw data, this time producing the full 1,944 records with all twelve months present for every country-year.

Part 3 — NDVI Flattening: Same Bug, Same Fix

Discovery

Before flattening NDVI, its raw JSON structure was inspected and found to be architecturally identical to NO_2's pre-fix structure — same deep nesting, and critically, the same pattern of only 11 monthly intervals per country-year (December absent), confirming the identical December-boundary bug existed independently in `download_ndvi_sentinelhub.py`, since NDVI and NO_2 are acquired via separate scripts despite sharing the same Sentinel Hub Statistical API pattern.

Fix and Re-Acquisition

The identical fix (extending both time-range end-boundaries to `f"{year + 1}-01-01T00:00:00Z"`) was applied to the NDVI acquisition script. Re-running acquisition encountered a transient `NameResolutionError` (DNS failure to reach `identity.dataspace.copernicus.eu`), unrelated to the code itself; a simple retry succeeded once the network/DNS issue resolved.

The re-acquired NDVI data was verified to contain 12 months per country-year before proceeding. A minor file-naming issue was also resolved: the corrected output had been saved as `ndvi_stats_test.json` (a leftover name from the original single-country test script, as previously flagged in an earlier session's journal), while a stale, December-missing version still existed under the intended final filename. The stale file was deleted and the corrected file renamed to `ndvi_stats_by_country_monthly.json`.

A new script, `flatten_ndvi.py` — structurally identical to `flatten_no2.py` but targeting the `ndvi` output key and field names — was written and executed, producing the full 1,944 flat records.

Part 4 — Master Dataset Assembly

Initial Merge Script

`master_merge.py` was written to join all processed datasets into a single analysis-ready table, keyed on `(NUTS_ID, year, month)`, using NO_2's flattened output as the base table (the most granular dataset, driving the merge). Climate and NDVI were joined at the same country-year-month granularity; GDP was joined at country-year granularity (repeating across all twelve months of a given year); Land Cover and DEM, being static single-snapshot datasets, were joined at country granularity alone (repeating across every row for that country).

Bug 1 — Land Cover Nested Dictionary

The first successful merge run produced 1,944 rows as expected, but inspection revealed that the land cover data had been written into a single column (`landcover_land_cover_percent`) containing an entire nested dictionary as a string (e.g., `{'Tree cover': 53.79, 'Cropland': 15.93, ...}`), rather than being split into individual numeric columns per land cover class. This was traced to the merge script's land-cover-handling logic not accounting for the fact that the source JSON stored class percentages as a nested sub-dictionary rather than flat key-value pairs. The merge logic was corrected to detect nested dictionary values and flatten each class into its own column (e.g., `landcover_tree_cover`, `landcover_cropland`), with class names cleaned (lowercased, spaces and slashes replaced with underscores) for valid column naming.

Bug 2 — Inconsistent Fieldnames Across Countries

Re-running the merge after the land-cover fix produced a `ValueError: dict contains fields not in fieldnames` during CSV writing. Root cause: different countries have different sets of land cover classes present (e.g., not every country has "Shrubland"), so the original approach of deriving CSV fieldnames from only the first row's keys failed whenever a later row introduced a column not present in the first. Fixed by collecting the union of all keys across every row before writing the CSV, with a stable column ordering (core fields first, remaining fields alphabetical).

Bug 3 — Systematic Temperature Data Loss (Most Significant)

After a clean merge run (1,944 rows, all expected columns present), a data-quality check via `.isnull().sum()` revealed `avg_temp_c` was missing in **100% of rows (1,944/1,944)**, while `avg_precip_mm` — sourced from the identical `climate_row` lookup dictionary in the same code path — was **fully populated**. This asymmetry was immediately recognized as logically impossible given the merge code (both fields are read from the same dictionary object), ruling out a merge-logic bug and pointing to a problem either upstream in the climate data itself or in some intermediate step.

Diagnostic sequence:

1. Confirmed the raw `era5_stats_by_country_monthly.json` file contained valid, non-null temperature values when sampled directly (e.g., `-0.78, -3.41...`), ruling out a total-corruption scenario.
2. Inspected the specific record for Austria, January 2019 directly by filtering the JSON, and found **two separate records** for the identical `(NUTS_ID, month)` combination: one with valid temperature (`-3.41`) and zero precipitation, and a second with `NaN` temperature and populated precipitation (`4.72`).
3. Counted total records in the climate file and found **3,888** — exactly double the expected 1,944 — confirming systematic duplication, not an isolated anomaly.
4. Traced duplication to its source by inspecting the underlying processed NetCDF file's time dimension directly, revealing **24 time steps per year instead of 12**, with each month appearing twice: once at `00:00:00` and once at `06:00:00`.

Root cause: In `process_era5.py`, the temperature source file (`avgua`, an "instantaneous"-type GRIB field) and precipitation source file (`avgad`, an "accumulated"-type field) carried slightly different internal timestamps for the same calendar month, despite representing the same monthly period. When both were combined into a single `xr.Dataset({...})` constructor without explicit time-alignment, xarray's automatic coordinate-alignment behavior treated the two differing timestamps as genuinely distinct time steps, producing two records per month — each populated with only one of the two variables and `NaN` for the other, since no actual observation existed for that variable at the mismatched timestamp.

This explains why `avg_precip_mm` appeared fully populated in the master dataset despite the underlying duplication: whichever of the two duplicate records the Python dictionary-based `climate_lookup` retained last (dictionaries silently overwrite on duplicate keys) happened to be the precipitation-populated, temperature-null variant for the specific test case inspected, though the broader pattern affected all 1,944 duplicated month-pairs identically.

Fix: `process_era5.py` was modified to explicitly force the precipitation dataset's time coordinate onto the temperature dataset's time coordinate (`ds_precip.assign_coords(valid_time=ds_temp["valid_time"].values)`) immediately after loading both datasets and before combining them, since both time values represent the same monthly period despite differing by a few hours. This eliminates the coordinate mismatch that caused xarray to treat them as separate

time steps. A redundant duplicate line (`ds_temp = xr.open_dataset(temp_file)` appearing twice) was also removed during this edit, though it had no functional effect on the bug.

Verification and Re-Processing

Following the fix:

1. `process_era5.py` was re-run for all six years, regenerating the processed NetCDF files with corrected, non-duplicated time steps.
2. `era5_regional_stats.py` was re-run, initially producing 2,736 records — this was correctly identified as expected, not a new bug, since this stage operates on the full NUTS boundary file (all ~38+ European countries, not yet EU-27-filtered) rather than the EU-27 subset, consistent with the pattern already established for other datasets requiring a separate downstream filtering step.
3. `apply_eu27_filter.py` was re-run, correctly filtering Climate from 2,736 to the expected 1,944 EU-27 records, alongside Land Cover (27), GDP (162), DEM (27), and NO_2 (162) all confirmed unchanged and correct.
4. `master_merge.py` was re-run, producing the final 1,944-row master dataset.
5. A final `.isnull().sum()` check confirmed `avg_temp_c` and `avg_precip_mm` both fully populated (zero missing values), resolving the bug completely.

Final Master Dataset State

`data/master_dataset.csv` — 1,944 rows (one per EU-27 country-year-month, 2019–2024), 21 columns spanning NO_2, NDVI, temperature, precipitation, GDP, elevation statistics, and eleven individual land cover class percentages.

Remaining missing-value pattern, fully explained:

- `mean_no2` (196 missing) and `mean_ndvi` (12 missing): legitimate data gaps from low sample counts or cloud-cover-affected satellite observations, consistent with expected Sentinel-5P/Sentinel-2 retrieval behavior.
- Select land cover class columns (Shrubland, Snow and ice, Moss and lichen, Herbaceous wetland: 144–1,440 missing): expected, reflecting countries where that particular land cover class is genuinely absent (e.g., flat, non-alpine countries having no "Snow and ice" coverage) rather than a data quality defect. These will require explicit zero-filling before use in downstream statistical modeling, since a missing value in this context represents "0% of this class," not "unknown."
- All other columns: zero missing values.

Design Principle Reinforced

This session surfaced two independent instances of the same underlying lesson already established earlier in the project (the GDP unit-mixing discovery, the DS03 France geometry anomaly): **a dataset that produces no errors and a plausible-looking record count can still contain a systematic, silent defect that only becomes visible through explicit verification** — checking record counts against expected totals, and checking for logically-impossible asymmetries between fields that should behave identically (as with `avg_temp_c` versus `avg_precip_mm`). In both the December-gap and the ERA5-duplication cases, the defect was **perfectly uniform** across the entire dataset rather than randomly scattered — and recognizing that

uniformity was itself the key diagnostic signal pointing toward a structural, request- or processing-level cause rather than a data-availability issue requiring separate investigation for each affected record.

Development Log — Module 8: Causal Inference Model Design, Implementation, and Critical Environment Debugging

Status

Complete. The project's core Difference-in-Differences causal inference model is now implemented and producing statistically valid results, following resolution of a significant environment-level numerical computing failure and two rounds of model-design correction.

Part 1 — Pre-Model Cleanup: Land Cover Zero-Fill

Before beginning causal inference work, a data-quality gap identified in the master dataset's missing-value audit was addressed: several land cover class columns (Shrubland, Snow and ice, Moss and lichen, Herbaceous wetland) contained missing values for countries where that particular land cover class is genuinely absent, rather than unknown. `master_merge.py` was modified to explicitly zero-fill these columns after CSV fieldname collection but before writing output, ensuring the master dataset correctly represents "0% of this class" rather than leaving a null value that downstream statistical models could misinterpret or silently drop. Re-running the merge confirmed all columns fully populated except the two columns with legitimate satellite-retrieval gaps (`mean_no2`, `mean_ndvi`).

Part 2 — Causal Inference Design: Confronting the No-Control-Group Problem

Initial Concept

Difference-in-Differences (DiD) was introduced as the intended methodology, illustrated via a treatment-group/control-group analogy: comparing a before-after change in a treated group against the same change in an untreated group isolates the treatment effect from confounding trends (e.g., seasonal patterns) that would otherwise bias a simple before-after comparison.

The Core Design Problem

Before implementation, it was recognized that GPIE's actual setting does not fit classic DiD cleanly: the European Green Deal and its associated legislation apply to all 27 EU member states simultaneously, meaning no natural control group (an EU country *not* subject to the policy) exists within the dataset.

To determine whether country-level policy-intensity variation could substitute for a control group, the Module 1 policy database (`documents.json`) was inspected directly. Two findings resulted:

1. Only 10 policy records existed in the database (noted as a potential incompleteness in scraping scope, though not investigated further as it did not block the immediate causal-inference design decision).

2. The sampled policy record (Regulation (EU) 2024/795) contained no country-specific field or tag — consistent with the general legal nature of EU Regulations, which apply uniformly and directly to all member states without requiring national transposition. This confirmed that a policy-intensity-based control group (Option 1, comparing "high-implementation" vs "low-implementation" countries) was not supported by the available data.

Design Decision: Generalized DiD via Two-Way Fixed Effects

Given the absence of a viable control group, the model was redesigned around a **timing-based approach**: using the European Climate Law (Regulation (EU) 2021/1119, effective 30 June 2021 — identified as the most significant single legislative milestone in the policy database, and the legally-binding centerpiece establishing the EU's climate-neutrality target) as a single treatment date applied uniformly across all countries, combined with:

- **Country fixed effects**, to absorb time-invariant country-specific characteristics (economic structure, geography, baseline pollution levels)
- **Time-related controls**, to absorb temporal trends independent of the treatment

This is a well-established variant of causal panel-data methodology (sometimes termed "generalized DiD" or a two-way fixed-effects design), rather than a compromise or deviation from rigorous practice — it was assessed as directly reusing the panel structure and control variables already present in the master dataset (GDP, climate, elevation, land cover).

Part 3 — First Implementation Attempt (linearmodels) and Design-Level Bugs

Initial Build

`causal_inference.py` was implemented using `linearmodels.panel.PanelOLS`, with the master dataset restructured into a panel format indexed by `(NUTS_ID, time)`. A binary `treatment` variable was constructed based on whether each observation's year-month fell after 30 June 2021. The model was specified with `entity_effects=True` (country fixed effects), `time_effects=True` (full time fixed effects), and a control set including elevation and land cover variables.

Bug 1 — Collinearity Between Entity Effects and Static Controls

The first execution ran without visible error but also produced no regression output — a silent failure. Diagnosis identified that `entity_effects=True` already fully absorbs any time-invariant, country-specific variable (such as elevation and land cover, which do not change across the study period for a given country) through its internal demeaning transformation. Including these same variables explicitly as regressors alongside entity effects created perfect multicollinearity, which was hypothesized to be causing a silent numerical failure in the underlying solver. Static controls (elevation, land cover) were removed from the explicit control set, retained instead as implicitly-controlled-for via the entity fixed effects — the econometrically correct approach.

Bug 2 — Fundamental Collinearity Between Treatment and Time Effects

After removing static controls, the same silent-failure pattern persisted. Further diagnosis revealed a more fundamental design flaw: because `treatment` varies only by time (identical value across all 27 countries for any given month) and `time_effects=True` creates a separate fixed effect for every unique month in the panel, `treatment` is mathematically 100% collinear with the time fixed effects — the model literally cannot distinguish "the treatment effect" from "this particular month's fixed effect," since they are the same information. This is the direct mathematical consequence of the no-control-group problem identified during design: full time fixed effects and a uniformly-applied treatment variable cannot coexist in the same model.

Fix: `time_effects=True` was replaced with a coarser seasonality control — monthly dummy variables based on calendar month (1–12) rather than each unique year-month period — allowing the treatment variable's before/after variation to remain estimable while still controlling for seasonal patterns in NO_2 (e.g., winter vs. summer atmospheric conditions). This was explicitly documented as a trade-off: common time-varying shocks unrelated to seasonality (e.g., COVID-19-era economic disruption) are no longer controlled for, a limitation inherent to the lack of a control group rather than a flaw in this specific implementation choice.

Part 4 — The Silent Crash: Diagnosing a Native Library Failure

Persistent Silent Failure

Even after both design corrections, the model continued to terminate with no output and no Python-level exception — the script would print progress statements up to immediately before the model-fitting call, then terminate silently. Wrapping the fitting call in a `try/except` block caught nothing, immediately suggesting the failure was not a normal Python exception.

Isolating the Crash

A systematic elimination process was carried out across several sessions:

1. **Exit code inspection**: Capturing `$LASTEXITCODE` after the crash revealed a large negative value (`-1066598273`), characteristic of a Windows-level process termination (analogous to a segmentation fault) rather than a Python-level error — confirming the crash originated below the Python interpreter, in native compiled code.

2. **Library substitution**: The model was reimplemented using `statsmodels` (`smf.ols` with a categorical formula for country and month fixed effects) in place of `linearmodels`, on the hypothesis that the issue might be specific to `linearmodels`'s internal implementation. The identical silent-crash pattern occurred, ruling out a `linearmodels`-specific bug and implicating a shared lower-level dependency (numpy/scipy's underlying linear algebra routines).

3. **Component isolation within statsmodels**: The clustered standard-error calculation (`cov_type="cluster"`) was removed in favor of plain OLS fitting, to test whether the crash was specific to the more complex covariance calculation. The crash persisted unchanged, ruling out clustering as the cause.

4. **Threading hypothesis**: Based on a known class of Windows-conda environment issues, Intel MKL's multi-threaded execution was suspected as a possible cause of internal thread-synchronization failure on large matrix operations. `OMP_NUM_THREADS` and `MKL_NUM_THREADS` environment variables were both set to `1` to force single-threaded execution before running the script. This did not resolve the crash.

5. **Boolean dtype hypothesis**: A further hypothesis was tested — that newer versions of `pandas.get_dummies()` returning boolean-dtype columns (rather than the historical float/integer dtype) might be incompatible with the numpy/patsy linear algebra pathway used internally by the formula API. The model was rewritten to bypass the formula API entirely, manually constructing the design matrix via direct `pd.concat()` of dummy variables with explicit `.astype(float)` casting throughout, and fitting via `statsmodels.api.OLS` directly on the numeric matrix. The crash persisted identically, ruling out the boolean-dtype hypothesis.

6. **Reducing to pure NumPy**: To isolate whether any statsmodels-specific code path was responsible at all, the diagnostic was reduced to bypass statsmodels entirely: a random matrix of the same dimensions as the actual design matrix (1748 × 42) was constructed, and `numpy.linalg.lstsq()` was called directly. This crashed identically, definitively ruling out statsmodels, linearmodels, and patsy as the source, and implicating numpy's own underlying linear algebra backend.

7. **Further NumPy isolation**: To narrow the fault further, an alternative solution method (`numpy.linalg.solve()` on the normal-equations form, which invokes a different LAPACK routine than SVD-based `lstsq`) was tested — also crashed. Finally, isolating to the matrix multiplication operation alone (`A.T @ A`, with no solving step at all) also crashed, confirming the fault existed at the most basic linear algebra operation level, below any solving or fitting logic entirely.

Root Cause

This elimination sequence conclusively identified the fault as a **broken or corrupted Intel MKL (Math Kernel Library) installation** within the project's specific conda environment — the compiled backend that NumPy and SciPy rely on for BLAS/LAPACK operations on Windows. This was not a bug in any Python-level code written for this project, but an environment-level numerical computing infrastructure failure, invisible to and unfixable through any amount of Python-level code adjustment.

Fix

An initial attempt to force-reinstall `numpy` and `scipy` via `conda install --force-reinstall` completed successfully but did not resolve the crash, indicating the corruption was specific to the MKL backend binaries themselves rather than the numpy/scipy Python packages. The environment was then reinstalled with the explicit `nomkl` package, which forces conda to install `numpy`/`scipy` linked against **OpenBLAS** instead of Intel MKL as the underlying linear algebra backend:

```
conda install -n gpie -c conda-forge numpy scipy nomkl --force-reinstall -y
```

A minimal matrix-multiplication test (`A.T @ A` on a small random matrix) succeeded immediately following this reinstall, confirming the OpenBLAS backend resolved the underlying numerical computing fault.

Part 5 — Successful Model Execution and Results

With the environment fixed, the full causal inference model (statsmodels OLS, explicit float-cast design matrix, country and month-of-year fixed effects, treatment variable, and climate/GDP controls) executed successfully to completion for the first time.

Model Specification

- **Outcome**: `mean_no2` (mean tropospheric NO_2 column density per country-month)
- **Treatment**: binary indicator for observations after 30 June 2021 (European Climate Law effective date)
- **Fixed effects**: country (26 dummy variables) and calendar month (11 dummy variables), absorbing time-invariant country characteristics and seasonal patterns respectively
- **Controls**: average temperature, average precipitation, GDP
- **Sample**: 1,748 observations (of 1,944 total, after dropping rows with missing NO_2 or control values), 27 countries

Results

- **Treatment coefficient**: -2.285×10^{-6}
- **P-value**: 0.026 (statistically significant at the 95% confidence level)
- **95% confidence interval**: $[-4.301 \times 10^{-6}, -2.692 \times 10^{-7}]$ (entirely negative, not crossing zero)
- **R-squared**: 0.3905
- **N**: 1,748

Interpretation

The negative, statistically significant treatment coefficient indicates that tropospheric NO_2 concentrations across EU-27 countries were measurably lower following the European Climate Law's entry into force (30 June 2021), after controlling for country-specific baseline differences, seasonal atmospheric patterns, temperature, precipitation, and GDP. This constitutes GPIE's first independently-derived, satellite-verified evidence that a core European Green Deal policy instrument corresponds with a statistically significant reduction in an independently observed environmental indicator — directly fulfilling the project's original "Trust, But Verify" research design.

Design Principle Reinforced

This session's dominant lesson extends a pattern established repeatedly earlier in the project (rasterio/NumPy incompatibility during Population processing, GDAL tile-read failures during Land Cover and Population processing): **environment-level numerical or I/O library failures can present as silent, seemingly-inexplicable failures indistinguishable at first from logic bugs**, and require a fundamentally different debugging strategy — systematic elimination of *entire libraries and abstraction layers* (linearmodels → statsmodels → pure NumPy → basic matrix multiplication) rather than inspecting the originally-written code for a logical error, since the originally-written code was, in this case, never the actual fault.

Development Log — Module 8 Robustness Testing: NDVI Validation, Placebo Test, and Identification of a Fundamental Design Limitation

Status

Complete for this phase. A second outcome variable (NDVI) was tested against the same model, followed by a placebo test that revealed a fundamental identification limitation in the single-cohort treatment design. This

limitation has been diagnosed precisely, and a methodological correction (introducing a genuine control group) has been decided upon for the next phase of work.

Part 1 — NDVI Cross-Validation

Objective

Having obtained a statistically significant treatment effect on NO_2, the same causal model specification was applied to a second, independent outcome variable — NDVI (vegetation health) — to test whether the Green Deal's environmental effect was detectable across multiple indicator types, not just the one already tested.

Implementation

A parallel script, `causal_inference_ndvi.py`, was created by adapting the existing NO_2 model: identical fixed-effects structure (country and calendar-month dummies), identical controls (temperature, precipitation, GDP), identical treatment definition (post–30 June 2021), with only the outcome variable changed from `mean_no2` to `mean_ndvi`.

Result

The treatment coefficient on NDVI was small and negative (−0.0059), with a p-value of 0.128 and a 95% confidence interval spanning zero ([−0.0136, +0.0017]). This result is **not statistically significant**.

Interpretation

This was assessed as a scientifically plausible and honest finding rather than a failure: the European Climate Law's immediate policy instruments (emissions trading, industrial regulation) are more directly and immediately connected to atmospheric NO_2 than to vegetation health, which responds to land-use and forestry policy on substantially longer timescales. A non-uniform pattern of results across outcome variables — a significant effect on one indicator and no detectable effect on another — was noted as more credible than uniformly positive results across every outcome tested would have been, since it suggests the model is genuinely differentiating between effects rather than producing indiscriminately positive findings.

Part 2 — Placebo Test and Discovery of a Fundamental Design Limitation

Objective

To test the credibility of the significant NO_2 result, a placebo test was conducted: the same model was re-run with the treatment date artificially shifted to a date within the study period where no comparable major policy event occurred (30 June 2020), on the logic that a well-identified causal model should find no significant effect at a fake treatment date, while a model that finds "significant effects" regardless of the date chosen would suggest the original result is not attributable to the specific policy being tested.

Implementation

`causal_inference_placebo.py` was written as a direct adaptation of the main model, with the treatment date parameterized and set to the placebo date (30 June 2020) rather than the true treatment date (30 June 2021).

Result — Placebo Test Failed

The placebo model produced a treatment coefficient of -3.29×10^{-6} with a p-value of 0.002 — **more statistically significant than the original real-date result** (which had a p-value of 0.026). This is a clear placebo test failure: a model that finds a "significant policy effect" at an arbitrary, non-policy-relevant date cannot be trusted to have correctly isolated the effect of the actual policy being studied.

Diagnostic Follow-Up — Isolating the Cause

To investigate further, an explicit linear time trend variable (`time_trend`, measured in days since the start of the study period) was added to the control set of the real-date (June 2021) model, on the hypothesis that the original significant result might be capturing a general, ongoing decline in NO_2 across the entire 2019–2024 period (driven by factors such as fleet modernization and general decarbonization trends unrelated to any single legislative act) rather than an effect specific to the Climate Law's entry into force.

Result: With the time trend included, the treatment coefficient on the real June 2021 date dropped to -1.39×10^{-6} with a p-value of 0.408 — **no longer statistically significant**. This confirmed the hypothesis directly: the originally observed "significant" effect was being driven by the general secular decline in NO_2 occurring throughout the entire study period, not by a distinguishable effect specific to the Climate Law's June 2021 entry into force.

Root Cause — A Precise Statement of the Limitation

The limitation is **not a coding error, data quality defect, or numerical computing issue** (as several earlier obstacles in the project had been) — it is a **structural identification limitation inherent to the research design**.

In the model as originally specified, the `treatment` variable is a function of time alone: it takes the identical value for every one of the 27 EU countries in any given month, because the European Climate Law applies to all member states simultaneously, with no EU country serving as an untreated comparison case. This means the model has no mathematical basis for distinguishing between two competing explanations for any observed post-treatment change in NO_2:

1. A genuine effect specifically attributable to the Climate Law's entry into force, or
2. A pre-existing, ongoing decline in NO_2 that was already occurring for reasons entirely unrelated to this specific piece of legislation (broader European decarbonization trends, vehicle fleet turnover, prior policy momentum, etc.), which happens to continue across the treatment date without being caused by it.

Without an untreated comparison group — a country or set of countries *not* subject to the EU Climate Law, against which the "what would have happened anyway" counterfactual trend could be estimated — these two explanations are **mathematically inseparable** within a single-cohort, time-only treatment design. Adding the explicit linear time trend as a diagnostic control demonstrated this directly: once the general trend was explicitly accounted for, no distinguishable treatment-specific effect remained detectable, exactly as the underlying identification problem predicts.

This is a well-recognized and long-documented limitation in causal policy evaluation research generally — any study of a policy applied uniformly and simultaneously to an entire study population, without an external comparison group, faces this same structural inability to separate policy effects from concurrent secular

trends. It is not unique to this project's implementation, and it was not detectable prior to explicit robustness testing (the initial model, without a placebo test or time-trend diagnostic, would have reported the spurious result as a genuine finding).

Part 3 — Methodological Correction: Introducing an External Control Group

Decision

Rather than proceeding with a design known to be unable to distinguish policy effects from concurrent trends, the causal inference methodology will be revised to introduce a genuine, non-EU control group — restoring the classic Difference-in-Differences structure that the original single-cohort design could not achieve.

Selected Control Countries

Three non-EU European countries have been selected as the control group: **the United Kingdom**, **Norway**, and **Switzerland**. These were chosen because:

- None are subject to the EU Green Deal or the European Climate Law, since none are EU member states (the UK specifically having exited the EU in 2020, placing its post-2021 period unambiguously outside EU regulatory scope).
- All three are geographically proximate, economically developed, and structurally comparable to EU-27 countries in terms of industrial base, climate, and general emissions-driving activity, making them credible counterfactual comparators — closer in relevant respects than a globally-selected control group would be.

Revised Design

With this control group in place, the model will move from a single-cohort "before/after" design to a genuine two-group Difference-in-Differences structure:

```
DiD Effect = (Change in EU-27 NO₂, pre- to post-treatment) − (Change in
UK/Norway/Switzerland NO₂, over the same period)
```

This structure allows any general, Europe-wide secular trend (the confound identified in Part 2) to be captured and subtracted out via the control group's own trend, isolating the portion of the EU-27's change that is specifically attributable to being subject to the Climate Law — which the untreated comparison countries were not.

Implementation Requirements Identified

- **Boundary data**: NUTS boundaries (the administrative boundary source used throughout the project to date) are an EU-specific classification system and do not cover the UK, Norway, or Switzerland. An alternative global boundary source (GADM or Natural Earth) will be required to acquire comparable country-level boundaries for the three control countries.
- **Earth Observation and auxiliary data**: NO_2, NDVI, and climate data for the three control countries will be acquired using the existing Sentinel Hub Statistical API pipelines already built and verified for the

EU-27 (Modules 2–3), extended to the new country geometries — no new acquisition methodology is required, only an expanded country list.
- **Master dataset extension**: An `is_eu` (or equivalently, `treatment_group`) indicator column will be added to the master dataset to distinguish EU-27 (treatment) from the three control countries, enabling a proper interaction-term DiD specification (`treatment_group × post_period`) in place of the current single treatment dummy.

This work has not yet begun; it is the defined next phase of Module 8.

Design Principle Reinforced

This session illustrates a distinct category of validation from the environment-level debugging documented in the prior session: here, the code executed correctly and produced a numerically valid result on the first attempt, yet that result was nonetheless **substantively wrong** in a way that only systematic robustness testing — not code review, not output inspection — could reveal. A statistically significant result with a clean p-value and correctly-executing code is not, by itself, evidence of a correctly identified causal effect. The placebo test and time-trend diagnostic applied here are standard, necessary components of credible causal inference practice specifically because a model can be fully correct in its mechanics while resting on a research design that cannot support the causal claim being made. Identifying this before proceeding to Modules 9–11 avoided building further analysis (economic efficiency rankings, policy-impact maps) on top of a result that would not have withstood scrutiny.

Development Log — Module 8 Extension: Control-Group Implementation and Final Difference-in-Differences Model

Status

Complete. A genuine external control group (United Kingdom, Norway, Switzerland) was acquired and integrated across all relevant datasets, resolving the fundamental identification limitation documented in the prior session. A proper two-group Difference-in-Differences model was implemented and executed, producing the project's final, honest causal-inference result for Module 8.

Part 1 — Boundary Acquisition for Control-Group Countries

Objective

Following the decision to introduce a genuine control group, country-level administrative boundaries were required for the United Kingdom, Norway, and Switzerland — none of which are covered by the project's existing NUTS boundary dataset, since NUTS is an EU-specific classification system.

Source and Acquisition

Boundaries were sourced from **GADM** (Database of Global Administrative Areas), a standard global boundary dataset, downloaded at Level 0 (country-outline only, no sub-national divisions) for all three countries. This

acquisition was conducted in a separate chat session due to attachment constraints in the main project session, with results verified there before being brought back: all three files (`gadm41_GBR_0.json`, `gadm41_NOR_0.json`, `gadm41_CHE_0.json`) were confirmed to have correct country-code properties (`GID_0`), valid closed-ring geometries, correct bounding boxes matching each country's real-world location, and a coordinate reference system (CRS84 / EPSG:4326) consistent with the project's existing NUTS file.

Files were placed in the project's existing boundary directory (`data/earth_observation/boundaries/raw/`) alongside the NUTS file. A minor file-discovery issue was noted during verification: the GADM files were saved with a `.json` extension rather than `.geojson`, causing them to be missed by an initial extension-filtered directory listing — resolved simply by listing the directory without a filter.

Unified Boundary-Loading Utility

A new shared module, `country_boundaries.py`, was created to provide a single, consistent interface for loading country geometry regardless of source (NUTS for EU-27, GADM for the three control countries), intended for import across all acquisition scripts (NO_2, NDVI, Climate) to avoid duplicating boundary-loading logic. Key design decisions:

- Country codes were standardized to a consistent 2-letter convention across both sources (`UK`, `NO`, `CH` for the control group), matching the NUTS convention already used throughout the project, even though GADM's native codes are 3-letter ISO3 (`GBR`, `NOR`, `CHE`).
- An optional `clip_to_bbox` parameter was included, reusing the project's existing European bounding box, to replicate the overseas-territory-clipping fix originally built for France's NDVI/NO_2 acquisition, since it was anticipated (correctly, as later confirmed) that this same fix would be needed for the UK's geographically dispersed island territories.
- `get_all_country_codes()` was added to return the full 30-country list (EU-27 + control group), for use as a drop-in replacement for the project's existing `get_eu27_iso2_list()` in acquisition loops.

Initial verification confirmed 30 total countries correctly resolvable, with both a control-group country (UK) and an EU country (Germany) successfully loading valid `MultiPolygon` geometries through the unified interface.

Part 2 — NDVI Acquisition for 30 Countries

Implementation

`download_ndvi_sentinelhub.py` was refactored to use the new shared `country_boundaries` utility in place of its original NUTS-only geometry loader, and to iterate over `get_all_country_codes()` (30 countries) rather than the EU-27-only list. Output was directed to a new file (`ndvi_stats_all_countries.json`) to preserve the existing verified EU-27-only NDVI dataset rather than overwriting it.

Execution and Two Distinct Failures

The first full run (180 requests: 30 countries × 6 years) completed with 168 successful records and two consistent failure patterns:

France failed with the same resolution-limit error encountered previously during the original EU-27 NDVI acquisition (`"Your request... exceeds the limit... of the collection"`), caused by France's NUTS

geometry including geographically distant overseas territories. This was expected, since the new shared `country_boundaries` utility's bounding-box clipping was not yet enabled by default in the calling script.

United Kingdom failed with a different, previously unseen error: `"COMMON_BAD_PAYLOAD"` on the geometry parameter. This was attributed to the sheer complexity of the UK's unclipped GADM geometry (approximately 2,562 polygon parts, reflecting the UK's many small islands), exceeding the Sentinel Hub Statistical API's payload complexity tolerance.

Fix and Retry

Bounding-box clipping was enabled explicitly for both failed countries by passing `clip_to_bbox` when calling `load_country_geometry()`. Before applying this at scale, the project's existing bounding box (`config.py`'s `MIN_LON`/`MAX_LON`/`MIN_LAT`/`MAX_LAT`) was verified against both countries' known coordinate extents, confirming neither Norway (northernmost point 71.18°N, comfortably within the bbox's 71.5°N limit) nor the UK would be inadvertently cut off by clipping.

A targeted retry script (`retry_failed_ndvi.py`) was used to reprocess only the two failed countries rather than re-running the full 180-request batch, appending results to the existing partial output under a new filename to avoid overwriting.

First retry: France succeeded immediately with bbox clipping applied. The UK failed again, but with a *new* error type: the clipping operation had produced a `GeometryCollection` (a mix of polygon and degenerate point/line geometries) rather than a clean `MultiPolygon`, which Sentinel Hub's API does not accept. This was diagnosed as an artifact of Shapely's `intersection()` operation: when a highly complex multi-part geometry (UK's islands) is intersected with a bounding box, some island fragments may touch the box boundary at only a single point or edge, producing degenerate zero-area geometry components alongside the valid polygon area, which Shapely bundles into a mixed-type `GeometryCollection`.

Second fix: `country_boundaries.py`'s clipping logic was extended to explicitly detect a `GeometryCollection` result and filter it down to only its `Polygon`/`MultiPolygon` components, discarding any degenerate points or lines, before returning the geometry. A second targeted retry for the UK alone succeeded across all six years.

Final Result

The complete 30-country, 6-year NDVI dataset was verified to contain exactly 180 records spanning all 30 expected country codes, with no remaining failures.

Part 3 — NO_2 Acquisition for 30 Countries

Implementation

`download_no2_sentinelhub.py` was refactored using the identical pattern established for NDVI: replacing the script's own NUTS-only geometry loader with the shared `country_boundaries` utility, switching the country-iteration source to `get_all_country_codes()`, and enabling bounding-box clipping from the outset (rather than discovering the need for it through failures, as had happened with NDVI). A new output file (`no2_stats_all_countries.json`) was used to preserve the existing EU-27-only dataset.

Execution

The full 180-request batch (30 countries × 6 years) completed successfully on the first attempt with zero failures, including France and the UK — confirming that the fixes developed and validated during the NDVI acquisition (bbox clipping plus the `GeometryCollection` filtering fix) transferred correctly to a second, independently-implemented acquisition script without requiring rediscovery of either issue.

Part 4 — Climate (ERA5) Extension to 30 Countries

Architectural Difference from NO_2/NDVI

Unlike NO_2 and NDVI, ERA5 climate data acquisition operates on a single Europe-wide bounding-box request per year (not a per-country request), with country-level statistics derived afterward via zonal statistics against a boundary file. Since the existing raw ERA5 data already covers the full European bounding box — inclusive of the UK, Norway, and Switzerland's geographic extent — no new raw data acquisition was required; only the downstream processing script (`era5_regional_stats.py`) needed modification to compute zonal statistics against the expanded 30-country boundary set.

Implementation

A new function, `load_all_country_geometries()`, was added to combine NUTS-derived (EU) and GADM-derived (control-group) geometries into a single unified list of `(country_code, geometry)` pairs, replacing the script's original NUTS-only iteration. The main zonal-statistics loop was updated to iterate over this combined list rather than directly over the NUTS GeoDataFrame. Output was directed to a new file (`era5_stats_all_countries_monthly.json`).

Bug — Duplicate Processing of EFTA Members

The first execution completed without error, loading 42 geometries (39 from NUTS, 3 from GADM) and producing 2,952 total records — noticeably more than the expected 2,160 (30 countries × 12 months × 6 years). Diagnostic grouping by country revealed that exactly two countries — Switzerland (`CH`) and Norway (`NO`) — had precisely double the expected record count (144 instead of 72), while all other countries were correct.

Root cause: The project's NUTS boundary file, as previously documented in an earlier session's journal entry, includes EFTA member states (Norway, Switzerland, Iceland, Liechtenstein) in addition to EU-27 countries, since NUTS is a broader European classification, not strictly an EU-membership list. Because Norway and Switzerland were being loaded once from NUTS (using their existing NUTS entries) and once again from GADM (as part of the newly-added control group), each was processed twice per month, producing duplicate records with (likely, though not deeply investigated further since the fix addressed the root cause directly) minor differences between the two boundary sources' exact geometry.

Fix

`load_all_country_geometries()` was modified to explicitly skip any NUTS entry whose country code matched one of the three control-group codes, ensuring Norway, Switzerland, and the UK are sourced exclusively from GADM, consistent with the rest of the control-group pipeline, rather than being loaded from both sources. Re-running the corrected script loaded 40 geometries (37 NUTS + 3 GADM, correctly excluding the two EFTA overlaps from NUTS) and produced 2,808 total records — still not yet exactly 2,160, but this remaining gap reflects legitimate scope broader than the intended 30 countries (the NUTS file includes non-

EU, non-control entities such as Turkey, Iceland, and Kosovo), addressed via filtering in the next step rather than a further processing bug.

EU-27 + Control-Group Filtering

A new function, `filter_climate_all_countries()`, was added to `apply_eu27_filter.py`, filtering the all-countries climate file down to the union of EU-27 codes and the three control-group codes, following the same backup-before-overwrite pattern established for all prior filtering functions in the project. This produced exactly 2,160 records, confirmed via direct inspection to span exactly the intended 30 countries with 72 records each (12 months × 6 years).

Part 5 — GDP Acquisition for Control-Group Countries

Motivation

A critical dependency was identified before proceeding to model construction: the existing causal-inference model design drops any row missing a control variable (via `dropna()`), and GDP was one of the model's controls. Without GDP data for the UK, Norway, and Switzerland, the entire control group would be silently dropped from the model at the `dropna()` step — with no error raised — potentially invalidating the control-group effort without any visible failure. Static variables like DEM and Land Cover, by contrast, were assessed as unnecessary to acquire for the control group, since they are fully absorbed by the model's country fixed effects and were not planned as explicit regressors going forward.

Data Source and Methodology

The project's existing GDP data (Eurostat) does not cover non-EU countries, so an alternative source was required. The **World Bank API** was used instead, providing free, unauthenticated access to GDP data (`NY.GDP.MKTP.CD` indicator, GDP in current USD) for any country via ISO3 code, including the UK (`GBR`), Norway (`NOR`), and Switzerland (`CHE`).

Currency conversion: Since World Bank data is reported in USD while the project's existing GDP data (Eurostat) is in EUR, an explicit currency conversion was applied using approximate annual average EUR/USD exchange rates for each study year (2019–2024), hardcoded as a lookup table. This was documented as a known approximation — annual averages rather than precise daily or monthly rates — assessed as an acceptable level of precision for a control variable in a regression model, rather than a primary variable requiring high accuracy.

Execution

`download_gdp_control_countries.py` was implemented and executed, successfully retrieving and converting 18 records (3 countries × 6 years) to a new file (`gdp_control_countries.csv`), kept separate from the existing EU-27 Eurostat-derived GDP file to preserve data provenance and allow independent verification of each source.

Part 6 — Control-Group Master Dataset and Final DiD Model

Master Dataset Construction

A new merge script, `master_merge_control.py`, was written specifically for the control-group analysis, combining:

- NO_2 and NDVI (both still in raw nested Sentinel Hub format for the 30-country files, requiring the same flattening logic developed earlier for the EU-27-only datasets, implemented generically here to handle both variables via a shared `flatten_nested_stats()` function)
- Climate (already flat, 30-country, EU-27-and-control-filtered)
- GDP (a combined lookup merging both the Eurostat EU-27 source and the World Bank control-group source into a single dictionary keyed by country and year)
- A new `treatment_group` column (1 for EU-27 countries, 0 for the three control countries), the key variable enabling a genuine DiD specification

DEM and Land Cover were deliberately excluded from this master dataset, consistent with the decision in Part 5, since they were not acquired for the control group and are unnecessary given the model's country fixed effects.

Execution produced 2,160 rows (30 countries × 6 years × 12 months) with zero missing values in `treatment_group`, `avg_temp_c`, `avg_precip_mm`, or `gdp_million_eur` — confirming the combined GDP lookup successfully resolved values for all 30 countries, avoiding the silent-data-loss risk identified in Part 5. `mean_no2` (230 missing) and `mean_ndvi` (18 missing) retained legitimate, expected gaps consistent with satellite-retrieval patterns observed throughout the project. A grouped check confirmed the expected group sizes: 27 countries with `treatment_group = 1` (EU-27) and 3 countries with `treatment_group = 0` (UK, Norway, Switzerland).

Final Difference-in-Differences Model

`causal_inference_final_did.py` implemented a proper two-group DiD specification, extending the single-cohort model design used previously:

- **`post`**: binary indicator for observations after 30 June 2021 (unchanged treatment date)
- **`did_interaction`**: the core causal estimator, calculated as `treatment_group × post` — equal to 1 only for EU-27 observations after the treatment date, and 0 for all control-group observations and all pre-treatment observations regardless of group
- **Fixed effects**: country and calendar-month dummies, as in the prior single-cohort model
- **Controls**: temperature, precipitation, GDP

This structure allows the model to distinguish, for the first time in this project's causal-inference work, between a Europe-wide secular trend (captured by the `post` main effect, common to both treatment and control groups) and an EU-specific additional effect attributable to the Climate Law (captured by the `did_interaction` term alone).

Result

The model executed successfully (1,930 observations after dropping missing-value rows, design matrix of 46 columns). The DiD interaction coefficient was:

- **Coefficient**: -1.40×10^{-6}
- **P-value**: 0.632
- **95% confidence interval**: $[-7.12 \times 10^{-6}, +4.32 \times 10^{-6}]$ (spans zero)
- **R-squared**: 0.3862

Interpretation

The interaction term is **not statistically significant**, and its confidence interval spans zero in both directions. This indicates that, once a genuine non-EU comparison group is introduced, **no statistically distinguishable additional reduction in NO_2 is detectable in EU-27 countries relative to the United Kingdom, Norway, and Switzerland following the European Climate Law's entry into force**. Whatever decline in NO_2 was observed within the EU-27 over this period appears to be part of a broader trend shared with comparable non-EU European countries, rather than a measurably distinct effect attributable specifically to this EU legislation.

This is assessed as a scientifically credible and methodologically well-supported finding, not a project failure. It directly reflects the rigorous validation sequence undertaken across this and the prior session: an initial result that appeared significant was subjected to a placebo test, which revealed it could not be trusted; a control group was then constructed specifically to address the identified structural weakness; and the resulting, properly-identified model produced a materially different, more conservative conclusion. This progression — from an apparently positive but ultimately unsupported result, to a rigorously validated null result — constitutes exactly the kind of "Trust, But Verify" scientific process the GPIE project was designed to demonstrate, applied here reflexively to its own causal claims rather than only to external government policy claims.

Design Principle Reinforced

This session's central lesson builds directly on the prior session's environment-debugging work but addresses a categorically different kind of problem: where the previous session dealt with an environment producing *incorrect execution* (silent crashes from a broken numerical library), this session dealt with a model that *executed correctly throughout* but was answering a subtly wrong question until a genuine control group was constructed. The willingness to treat an initially "successful," statistically significant result as provisional — and to actively try to break it via a placebo test — rather than treating a clean p-value as sufficient validation, was what surfaced the need for this entire phase of work. A model can be numerically correct, produce publishable-looking output, and still rest on an unsupported causal design; validating the design itself, not just the arithmetic, is a distinct and necessary step in credible causal inference work.

Development Log — Module 8 Final Validation: Event-Study Analysis

Status

Complete. An event-study extension of the control-group DiD model was implemented, providing a time-disaggregated robustness check on the overall null result reported in the prior session. This concludes the causal inference work for Module 8.

Objective

The overall DiD model (prior session) produced a single average treatment effect across the entire post-treatment period, which was non-significant. An event-study specification was implemented as a standard

robustness extension to this result, addressing two distinct questions that a single average effect cannot answer:

1. **Parallel-trends validation**: Does the EU-27 group show any pre-existing, statistically significant divergence from the control group *before* the treatment date? A DiD model's causal validity depends on the assumption that treatment and control groups would have followed similar trends absent treatment — this can be partially tested by checking whether they already diverged significantly in the pre-treatment period.
2. **Effect-timing**: Could a genuine but time-delayed policy effect have been masked by averaging across the entire post-treatment period? A policy effect might reasonably take one to two years to materialize as regulations are transposed into national implementation.

Methodology

`causal_inference_event_study.py` extended the master control-group dataset (`data/master_dataset_control.csv`) by binning the time variable into calendar quarters (2019Q1 through 2024Q4, 24 total periods) rather than a single binary pre/post indicator. For each quarter except a designated reference quarter (2021Q2, the quarter immediately preceding the treatment date of 30 June 2021), an interaction term was constructed: `treatment_group × (quarter == q)`, non-zero only for EU-27 observations falling within that specific quarter. This produces 23 separate coefficients, each representing the EU-27 group's estimated deviation from the control group in that specific quarter, relative to the reference quarter — the standard event-study specification for testing both pre-trends and effect dynamics simultaneously.

The model retained the same fixed-effects structure as the overall DiD model (country dummies) and controls (temperature, precipitation, GDP), replacing the previous single set of calendar-month dummies with quarter dummies to match the new temporal granularity.

Result

All 23 quarterly interaction coefficients were statistically non-significant, with p-values ranging from approximately 0.18 to 0.94 — none approaching the conventional 0.05 significance threshold, and coefficients fluctuating in sign (alternating positive and negative) without any discernible trend, either before or after the treatment quarter.

Interpretation

This result provides two reinforcing pieces of evidence:

Parallel-trends support: None of the seven pre-treatment quarters (2019Q1 through 2021Q1) showed a statistically significant difference between the EU-27 and control groups relative to the reference quarter, consistent with the assumption underlying the DiD design that the two groups were on comparable trajectories before the policy took effect. This strengthens, rather than merely assumes, the validity of the overall DiD estimate reported in the prior session.

No delayed effect detected: None of the fourteen post-treatment quarters (2021Q3 through 2024Q4) showed a significant effect either, ruling out the possibility that a genuine policy effect was present but obscured by averaging across the full post-treatment window in the overall DiD specification. The null result is therefore not an artifact of temporal aggregation — it holds consistently across every individual quarter

examined, providing a second, independent line of evidence supporting the overall conclusion reached in the prior session's control-group DiD model.

Design Principle Reinforced

This analysis exemplifies a standard but often-omitted step in credible causal inference: testing an aggregate result's robustness by disaggregating it along the dimension (here, time) that the aggregate estimate collapses. A single significant or non-significant coefficient can mask considerable underlying heterogeneity; demonstrating that a null result holds uniformly across 23 independent time periods — rather than resulting from a few offsetting significant effects that happen to average toward zero — provides meaningfully stronger evidence for the conclusion than the overall estimate alone. Combined with the prior session's placebo test and control-group construction, this completes a three-part validation sequence (placebo test → control-group DiD → event-study disaggregation) that collectively supports the project's final causal finding with a level of rigor substantially exceeding the project's original single-cohort model.

Module 8 — Final Status: COMPLETE

GPIE's core causal inference objective has been fulfilled: an independently-verified, satellite-derived assessment of whether the European Green Deal / European Climate Law produced a measurable, EU-specific reduction in NO_2 pollution, validated through a placebo test, a genuine external control group, and a full event-study robustness check. The finding — no statistically distinguishable EU-specific effect, distinct from a broader European trend shared with non-EU comparator countries — is a scientifically defensible, rigorously validated conclusion consistent with the project's "Trust, But Verify" research design, regardless of whether it matches the outcome that might have been hoped for at the project's outset.

Development Log — Event-Study Visualization and Environment Recovery

Status

Complete. A visual event-study plot was produced to accompany the prior session's quarterly regression results, following an unrelated but significant environment failure encountered during setup, and concluding with an important honest caveat regarding statistical power that has been added to the project's Module 8 documentation.

Part 1 — Environment Failure: Matplotlib Installation Corrupting the Conda Environment

Objective

To visualize the event-study coefficients (23 quarterly EU-vs-control interaction terms with 95% confidence intervals) produced in the prior session, a plotting script (`plot_event_study.py`) was written using `matplotlib`, which had not yet been installed in the project's `gpie` conda environment.

Failure

Running the script immediately failed with `ModuleNotFoundError: No module named 'matplotlib'`, as expected for a missing package. `matplotlib` was installed via `conda install -n gpie -c conda-forge matplotlib -y`, which completed without any reported error.

However, re-running the script afterward produced an entirely different and far more serious failure:

```
ImportError: DLL load failed while importing _ctypes: The specified module could
not be found.
```

This error occurred at the very first line of the script (`import pandas as pd`), and traced into `_ctypes` — a core built-in Python module, not a third-party package. This indicated that the `matplotlib` installation had somehow corrupted the Python interpreter installation itself within the `gpie` environment, not merely failed to install cleanly.

Diagnosis

The likely cause was conda's dependency resolver silently modifying or partially replacing the underlying Python interpreter binaries or its bundled C extension modules while resolving matplotlib's dependency tree — a known but uncommon failure mode when a conda environment's package set becomes sufficiently complex and inter-version-constrained (the `gpie` environment had, by this point, accumulated a large number of packages installed across many sessions: numpy, scipy, pandas, geopandas, rasterio, statsmodels, cdsapi, pyjwt, cryptography, and others, several with their own version-specific dependency requirements).

Fix Attempt 1 — Failed

An in-place fix was attempted first: force-reinstalling Python itself within the existing environment (`conda install -n gpie python=3.11 --force-reinstall -y`). This did not resolve the issue — the identical `_ctypes` DLL load failure persisted afterward, indicating the corruption was not limited to the Python binary alone, or that the reinstall did not fully replace the broken components.

Fix — Fresh Environment

Rather than continuing to debug an environment with an unknown extent of corruption, a new, clean conda environment (`gpie2`) was created from scratch, with all of the project's accumulated dependencies specified explicitly in a single creation command (numpy, scipy, pandas, nomkl, geopandas, rasterio, rasterstats, gdal, statsmodels, matplotlib, requests, python-dotenv, shapely, xarray, netcdf4, cdsapi, pyjwt, cryptography), including `nomkl` from the outset this time, given the prior session's discovery that Intel MKL was the root cause of an earlier, unrelated silent-crash issue in this same project.

This approach was chosen over further in-place repair because a fresh environment provides a clean, verifiable baseline rather than an unknown partially-fixed state, and because the full dependency list was already known and available from the project's accumulated history, making a from-scratch rebuild low-risk.

Verification

A minimal import test (`import pandas, numpy, matplotlib, geopandas, rasterio`) succeeded in the new `gpie2` environment, confirming the fix. A benign `GDAL_DATA` warning (missing configuration path for GDAL's XML schema files) was noted but assessed as non-blocking, since it does not affect the specific operations used in this project so far.

All subsequent commands in the project now reference `envs\gpie2\python.exe` rather than the original `gpie` environment. The original `gpie` environment was left untouched (not deleted) rather than repaired further, since the fresh environment fully resolved the immediate need.

Part 2 — Event-Study Plot Generation

Implementation

`plot_event_study.py` was written to reproduce the prior session's event-study regression (identical specification: EU × quarter interaction terms relative to a 2021Q2 reference quarter, country and quarter fixed effects, temperature/precipitation/GDP controls) and visualize the resulting 23 coefficients as a point-and-error-bar plot, with:

- Each quarter's estimated coefficient plotted as a point, with 95% confidence interval error bars
- A horizontal dashed reference line at zero
- A vertical dotted line marking the treatment date (30 June 2021), positioned between the 2021Q2 and 2021Q3 x-axis positions
- The reference quarter (2021Q2) included in the plot as an explicit zero point, for visual continuity across the full timeline

Execution and Verification

The script executed successfully in the new `gpie2` environment on the first attempt, reproducing coefficient values identical to the prior session's regression output (confirming consistency between the two runs) and saving the plot to `outputs/plots/event_study_plot.png`.

Since the current chat session had reached its attachment limit, the resulting image could not be directly reviewed within this conversation. Visual verification was instead conducted by the user opening the file directly via Windows' default image viewer (`Start-Process` on the file path), and reporting back on the plot's contents. This confirmed:

- All 23 quarterly data points present, correctly ordered from 2019Q1 through 2024Q4
- Error bars visible at every point
- The treatment-date reference line correctly positioned between 2021Q2 and 2021Q3
- The zero reference line correctly drawn
- Every single quarter's confidence interval spans zero, with no point statistically distinguishable from zero — visually confirming the regression's numerical result (no significant quarter, either pre- or post-treatment)

Part 3 — Honest Caveat: Null Result vs. Limited Statistical Power

Issue Raised

On reviewing the plot, a methodologically important distinction was raised: a non-significant result can arise from two different underlying situations — a genuine null effect (the true effect is close to zero), or an underpowered test (a real effect may exist but the study's sample is too small, or the confidence intervals too wide, to detect it reliably). These have different implications for how the result should be reported, and the plot itself does not by default distinguish between them.

Assessment

The relevant diagnostic is the width of the confidence intervals relative to the coefficient estimates. In this project's case, the overall DiD model's confidence interval — $[-7.12 \times 10^{-6}, +4.32 \times 10^{-6}]$ — was assessed as reasonably wide relative to the estimated effect sizes, rather than being tightly clustered around zero. A tightly-bounded interval close to zero would support a strong "genuine null effect" claim; a wide interval spanning a substantial range in both directions, as observed here, is also consistent with the study simply lacking sufficient statistical power to detect a real effect — a plausible concern given the control group consists of only three countries (UK, Norway, Switzerland), a comparatively small comparison group for a panel regression with many fixed-effect parameters.

Conclusion and Documentation Decision

This caveat was identified as an important, previously under-stated nuance in the project's Module 8 reporting. Rather than continuing to state the finding as an unqualified "no effect detected," the project's documentation will be revised to explicitly acknowledge this limitation: the honest conclusion is not simply "the policy had no effect," but rather "with this study's sample size and three-country control group, no statistically distinguishable EU-specific effect could be detected — a result consistent with either a genuinely negligible effect, or with the study's control group being too small to provide adequate statistical power to detect a real but modest effect." This is treated as a strengthening addition to the project's scientific rigor rather than a weakening of the prior conclusion, consistent with the project's established pattern of subjecting its own results to continued scrutiny rather than treating an initial finding as final.

Design Principle Reinforced

Two distinct lessons from this session extend patterns already established earlier in the project. First, the matplotlib/conda environment corruption is another instance — following the earlier MKL numerical-library corruption — of environment-level infrastructure failures being indistinguishable from code bugs at first appearance, reinforcing the value of isolating and rebuilding the environment itself as a debugging strategy of last resort, rather than exhaustively debugging code that is not actually at fault. Second, the null-versus-underpowered distinction reinforces a theme central to this project's approach to causal inference: a statistical result's face-value interpretation ("not significant" = "no effect") is often incomplete, and genuinely rigorous reporting requires actively interrogating what a result can and cannot support — the same spirit that motivated the placebo test and control-group construction in the prior module now applied to the interpretation of the null result itself, rather than stopping at the first non-significant p-value obtained.

Development Log — Module 10: Geospatial Output Generation (Choropleth Maps and Study-Design Visualization)

Status

Complete. Seven geospatial and statistical visualizations were produced covering the project's core datasets and causal-inference design, each independently verified for correctness before being finalized for use in the project's dashboard and reporting.

Approach Decision

QGIS was initially considered for this module, as originally planned in the project's module architecture. This was reconsidered and deliberately scoped out in favor of a fully Python-based mapping pipeline (`geopandas` + `matplotlib`), for two reasons: first, all of the project's boundary and statistical data were already in formats directly usable by `geopandas`, avoiding a separate export/import step into QGIS; second, a Python-based pipeline keeps map generation reproducible and version-controlled alongside the rest of the project's codebase, consistent with GPIE's broader automation-first design philosophy. QGIS proficiency is demonstrated through other portfolio work rather than being required here.

Verification Methodology

Given that generated map images could not be directly viewed within the main working session (attachment limits), a structured verification workflow was adopted: each map was opened locally via the operating system's default image viewer, then uploaded to a separate chat session with a detailed, checklist-style verification prompt specifying exactly what each element of the map was supposed to show. This produced explicit, itemized confirmation (or identification of defects) for each map before it was considered final, rather than relying on visual self-assessment alone.

Map 1 — NO_2 Choropleth (30 Countries, 2019–2024 Average)

The first map produced, establishing the visual template reused across subsequent maps: EU-27 boundaries from NUTS, control-group boundaries (UK, Norway, Switzerland) from GADM, combined into a single GeoDataFrame, with control-group countries visually distinguished via a thicker border rather than a separate color, so their NO_2 values remain directly comparable to EU countries on the same color scale.

Colormap iteration: An initial `YlOrRd` (yellow-to-red) colormap was used, chosen for intuitive association with pollution severity. Verification revealed several low-NO_2 countries rendering as visually indistinguishable from blank/white map background, since `YlOrRd`'s low end is very pale. This was corrected by switching to `plasma` (dark purple to bright yellow), whose low end remains clearly visible and distinguishable from missing data, resolving the issue.

Missing-data handling: `missing_kwds` was configured to render any genuinely missing country data as grey with diagonal hatching, rather than blending into the white background, established as a standard convention retained across all subsequent choropleth maps in this module.

Final verification confirmed all 30 countries correctly colored, correct legend, title, and attribution, and no rendering defects.

Map 2 — NO_2 Before vs. After (2019 vs. 2024, Side-by-Side)

A two-panel comparison map was built to visually complement the project's quantitative Module 8 finding, showing NO_2 distribution in 2019 and 2024 side by side. A single shared color scale (`vmin`/`vmax` computed across both years) was deliberately used rather than letting each panel auto-scale independently, since independent scaling would visually exaggerate or understate the actual magnitude of change between years — a shared scale ensures that any visible color shift between panels represents a genuine change in value, not an artifact of differing color normalization.

Verification confirmed a visible overall darkening/shift toward lower NO_2 in the 2024 panel relative to 2019 (particularly across Germany and Central Europe), a single shared colorbar beneath both panels, and correctly rendered control-group borders in both panels. Minor cosmetic polish (larger panel titles, larger figure size, reduced inter-panel spacing) was identified as optional refinement but not required for correctness.

Map 3 — Control-Group Study-Design Map

A simple categorical (non-data) map was produced specifically to visually communicate the project's causal-inference research design: EU-27 countries shaded one solid color (treatment group) and the three control-group countries shaded a distinctly different color with thicker borders, accompanied by a legend explicitly labeling each group's role in the Difference-in-Differences framework. This was designed as an introductory/explanatory visual for the dashboard, intended to orient a viewer to the study's comparison structure before presenting substantive data-driven maps.

Verification confirmed all three control-group countries were correctly and exclusively colored in the distinct color, all 27 EU countries correctly colored in the treatment color, correct legend labeling (no color/label swap), and no unexpected third colors or blank countries — assessed as fully correct on first generation.

Map 4 — Land Cover Dominant Class (EU-27)

A categorical map showing each EU-27 country's single largest ESA WorldCover land cover class, intended as environmental baseline context.

Bug encountered: The initial implementation crashed with `ValueError: Invalid RGBA argument: nan` when attempting to plot. Diagnosis initially suspected a missing country in the underlying land cover dataset (a pattern seen previously with the WorldPop population dataset), but a direct country-count check confirmed all 27 expected countries were present with no gaps. Further inspection of the raw JSON record structure revealed the actual cause: the land cover class percentages were nested one level deeper than the loading function assumed (`record["land_cover_percent"]`, a dictionary of class names to percentages, rather than the class names appearing as top-level keys directly on each record). The loading function had been written assuming a flat structure inconsistent with the dataset's actual, nested format.

Fix: The dominant-class extraction logic was corrected to read from the correctly-nested `land_cover_percent` key. A defensive `fillna("#cccccc")` grey fallback was also added to the color-mapping step, so that any future unmatched or missing land cover class would render as a clearly identifiable grey rather than crashing the script.

Verification confirmed exactly three distinct dominant classes present across EU-27 (Tree cover, Cropland, Grassland), correct solid categorical (non-gradient) rendering, a legend matching only the classes actually present, and correct exclusion of the three control-group countries (Land Cover data was intentionally

acquired for EU-27 only, consistent with the project's earlier design decision that static country-level variables are fully absorbed by fixed effects in the causal model and were therefore not required for the control group).

Map 5 — NDVI Choropleth (30 Countries, 2019–2024 Average)

Structurally identical to Map 1, using a `YlGn` (yellow-to-green) colormap appropriate for vegetation-health data, applied without requiring any of the colormap-related debugging encountered in Map 1, since the `plasma`-style low-visibility lesson had already been incorporated by choosing a colormap whose low end remains visually distinct from white.

Verification confirmed all 30 countries correctly colored with no missing/hatched countries, correct control-group border rendering, and a color pattern broadly consistent with known regional vegetation patterns (forested Western/Northern European countries appearing in richer greens than more agricultural or arid regions), with no anomalies requiring further investigation.

Map 6 — GDP Choropleth (30 Countries, 2019–2024 Average)

Initial Version — Visualization Problem Identified Pre-Verification

Before formal verification, a design flaw was identified directly by inspection: GDP's real-world distribution is highly right-skewed (a small number of very large economies — Germany, France, the UK — dwarf most other countries in the dataset). Combined with an initial `Blues` linear color scale, this compressed nearly all countries into a visually indistinguishable pale range, with only the largest economies standing out — a technically correct but poorly communicative visualization.

Fix — Colormap and Log Transformation

Two changes were made together: switching to the `viridis` colormap (whose low end remains dark and distinguishable, consistent with the fix applied in Map 1), and applying a `log10` transformation to the GDP values before color-mapping. The log transformation directly addresses the underlying skew (rather than merely working around it with a different color palette), spreading the color scale more evenly across the full range of country GDP values rather than compressing most countries into one end of the scale. The colorbar label was updated to explicitly state "$\log_{10}$ scale" to prevent misinterpretation of the transformed values as raw GDP figures.

Verification confirmed a substantially improved visual spread across countries (explicitly noted as a clear improvement over the initial version), correctly identified the largest visible economies (Germany, UK, France) as the brightest/highest values and smaller economies (Cyprus, Baltic states, Balkan countries) as the darkest/lowest, a correctly labeled log-scale colorbar showing appropriately small numeric values (approximately 4.5–6.5) rather than raw GDP figures in the hundreds of thousands, and correct control-group border rendering.

Map 7 — Event-Study Plot

Produced in a prior session (documented separately); included here for completeness of the module's full output inventory. A point-and-error-bar visualization of the 23 quarterly EU-vs-control-group NO_2 coefficients from the event-study regression, with a treatment-date reference line and zero-effect reference line, visually

reinforcing the Module 8 finding that no quarter — pre- or post-treatment — showed a statistically significant deviation from zero.

Module 10 — Final Status

All seven planned visualizations are complete, independently verified, and stylistically consistent (shared color-scheme conventions, consistent boundary sources, consistent control-group border treatment, consistent title/attribution formatting) across the full set. This provides the complete visual asset library required for Module 11 (dashboard construction).

Design Principle Reinforced

Two recurring patterns from earlier in the project reappeared in this module in new forms. First, the land cover `NaN` crash reinforced the value of inspecting a raw data record's actual structure directly (rather than assuming a structure based on how a similar dataset was formatted elsewhere in the project) before writing extraction logic against it — the same lesson underlying several earlier debugging sessions in this project, now applied to a nested-versus-flat JSON structure rather than an API response format. Second, the GDP log-transformation fix exemplifies a distinction relevant throughout this project's visualization work: a technically correct rendering (accurate colors mapped to accurate values) is not the same as an effective one, and recognizing when a visualization technically works but fails to communicate — here, due to an unaddressed property of the underlying data distribution — is a distinct and necessary check beyond confirming the absence of bugs or rendering errors.

Development Log — Module 11: Interactive Dashboard Construction and GitHub Deployment Setup

---

Status

Complete (dashboard construction + version control). An eight-page interactive Streamlit dashboard was built, styled, and populated with all project outputs, followed by first-time Git/GitHub setup and a successful initial push of the full codebase to a public repository.

Part 1 — Dashboard Framework and Styling Iteration

Initial Setup

Streamlit was installed into the `gpie2` environment. A multi-page app structure was adopted (`dashboard/app.py` as the home page, with a `dashboard/pages/` subfolder for additional pages), using Streamlit's built-in file-based page routing rather than manual navigation logic.

Bug — Missing Page Files Crash the App

On first run, the app crashed with `StreamlitAPIException: Unable to create Page. The file '5_Causal_Results.py' could not be found.` This was because Streamlit's multi-page routing

automatically scans the `pages/` folder and expects every referenced page file to exist; since only `app.py` had been created at that point, the app failed immediately rather than rendering a partial navigation. Resolved by creating placeholder files for all six planned pages before proceeding, each containing minimal valid content (`st.title("Coming soon...")`), allowing the app to run while pages were built out incrementally.

Styling — Two Full Iterations

A shared `dashboard/styles.py` module was created to centralize CSS styling via `st.markdown()` with `unsafe_allow_html=True`, applied consistently across every page through a shared `apply_custom_style()` function.

First version: a pastel color scheme (lavender/mint/peach gradient background, soft rounded cards) was implemented per initial styling preference.

Revision: this was explicitly rejected in favor of a dark, "professional/tech/advanced" aesthetic — a near-black gradient background, electric cyan/purple/green accent gradient for headings, glassmorphism-style metric cards, and monospace accents for numeric values (`Inter` and `JetBrains Mono` Google Fonts). This was a deliberate full rewrite of `styles.py` rather than an incremental adjustment, given the stylistic direction was completely different from the first version.

Minor Fix — Heading Alignment

The home page's main heading was left-aligned by default (Streamlit's default `st.markdown()` behavior), rather than centered as intended. Fixed by explicitly wrapping the heading HTML with an inline `style="text-align: center;"` attribute, rather than relying on any global CSS rule, since Streamlit's markdown rendering does not center-align headings by default even within custom CSS block styles applied elsewhere.

Bug — Duplicate Page Configuration

An early version of one content page (`1_Study_Design.py`) included its own `st.set_page_config()` call, duplicating the one already present in `app.py`. Since `set_page_config()` is only valid once per Streamlit session and must be the first Streamlit command executed, this produced unexpected duplicate rendering behavior. Fixed by removing `set_page_config()` from every subsequent page file — it is called exactly once, in `app.py`, for the entire multi-page app.

Part 2 — Content Pages

Eight pages were built in total, populated with the project's existing outputs:

1. **Home** (`app.py`) — project overview, key metrics, navigation guide, and author attribution ("Developed by Sakshi D. Maske, Independent Geospatial Researcher").
2. **Environmental Data** — tabbed display of the NO_2 and NDVI choropleth maps (Module 10 outputs).
3. **Study Design** — the control-group design map, explanation of the treatment/control architecture and the DiD logic, framed for a reader unfamiliar with the project's earlier iterations.
4. **Before vs. After** — the 2019-vs-2024 side-by-side NO_2 comparison map.
5. **Economic Context** — tabbed display of the GDP and Land Cover choropleth maps, with explanation of their role as control variables.
6. **Causal Results** — the project's core statistical findings: headline DiD coefficient/p-value/confidence-interval metrics, the event-study plot, and a narrative summary of the three-step validation sequence

(initial model → placebo test → control-group correction).

7. **Methodology & Limitations** — an expandable, detailed walkthrough of the full validation journey (placebo test failure, control-group construction, event-study robustness check) and explicit statement of the statistical-power limitation and the Module 9 scoping-out decision.
8. **About & Data** — downloadable master dataset (CSV export via `st.download_button()`), a data-preview table, a data-sources/citation table, and the GitHub repository link (initially a placeholder, updated once the repository existed).

Design Decision — Full-Effort Map Suite

Rather than a minimal set of visuals, all previously-verified Module 10 maps were deliberately integrated across the dashboard's relevant pages, since generating them had already been low-marginal-cost in Python and each map's inclusion strengthened a specific page's narrative (e.g., the GDP map on Economic Context, the study-design map on Study Design) rather than being presented as an undifferentiated gallery.

Part 3 — Interactivity Additions (Beyond Static Images)

Following a review that the dashboard, as initially built, was essentially a static image gallery, a set of interactivity enhancements was added:

Interactive Time-Series Explorer (`7_Explore_Trends.py`)

`plotly` was installed and used to build a country-selectable, variable-selectable line chart (NO_2, NDVI, temperature, or GDP over 2019–2024), with:

- A multi-select dropdown for choosing countries to compare, defaulting to a mix of EU-27 and control-group countries
- Control-group countries rendered with a distinct dotted line style, visually distinguishing them from solid EU-27 lines within the same chart
- A vertical reference line marking the treatment date (30 June 2021)
- `st.cache_data` applied to the data-loading function, avoiding repeated CSV reads on every user interaction

Regression Output Table and Comparative Bar Chart (added to Causal Results page)

A structured coefficients table was added summarizing the DiD model's key regression output (interaction term, main effects, controls) in tabular form, alongside an interactive Plotly grouped bar chart comparing EU-27 vs. control-group average NO_2 across pre- and post-treatment periods — providing a second, complementary visual representation of the same finding already shown via the event-study plot and headline metrics, aimed at readers who engage more readily with summary bar comparisons than with a 23-point regression coefficient plot.

Part 4 — Version Control Setup (First-Time Git/GitHub Configuration)

Git Installation

Git was not previously installed on the system. `git` commands failed with `CommandNotFoundException` in the VS Code terminal even immediately after installing Git for Windows (standalone x64 installer), because VS

Code's integrated terminal caches the system PATH at window-launch time and does not pick up newly-installed executables without a full application restart — closing and reopening only the terminal panel was insufficient; the entire VS Code application had to be closed and reopened before `git --version` resolved correctly.

Repository Initialization

`git init` was run from the project root (after an initial misstep of running it from the `dashboard/` subdirectory, corrected via `cd ..`), successfully creating a local repository.

`.gitignore` Audit and Expansion

The existing `.gitignore` was found to be significantly out of date — it only excluded NO_2 raw data and a couple of credential files, dating from early in the project before DEM, Land Cover, Population, and GADM-related large files had been introduced. It was rewritten to comprehensively exclude: all raw satellite/climate data directories across every dataset (NO_2, NDVI, DEM, Land Cover, Climate, Population), all `.nc` NetCDF files, intermediate processed raster files (VRT mosaics, resampled TIFFs), credentials (`.env`, CLMS service key), Python cache files, and Streamlit's local cache directory — while deliberately leaving all `final/` processed JSON/CSV outputs and the master datasets un-excluded, since these are small, directly reusable, and central to the project's reproducibility.

Bug — Dubious Ownership Error

`git add .` failed with `fatal: detected dubious ownership in repository`, a Git security feature that flags repositories on filesystems that don't record standard Unix-style file ownership (relevant here because the project resides on a D: drive rather than the default user profile location). Resolved by explicitly marking the directory as trusted via `git config --global --add safe.directory <path>`, as directed by Git's own error message.

Bug — Missing Git Identity

The first commit attempt failed with `Author identity unknown`, since Git requires a configured name and email before allowing any commit, and none had been set on this machine (a fresh Git installation). Resolved via `git config --global user.name` and `git config --global user.email`, a standard one-time setup step.

Initial Commit and Push

Following the above fixes, `git add .`, `git commit`, and `git push` completed successfully: 134 files, approximately 1.85 million lines of insertions (reflecting the project's accumulated JSON datasets and generated map images), pushed to a newly created public GitHub repository (`sakshimaske303-commits/GPIE`) via browser-based authentication. Two previously-existing incomplete repositories on the same GitHub account were deleted beforehand, to ensure the new repository represents a clean, complete, single source of truth for the project rather than coexisting alongside earlier abandoned attempts.

Design Principle Reinforced

This session's environment-level issues (Git PATH caching, dubious-ownership detection, missing Git identity) form a distinct but related category to the numerical-library failures debugged earlier in the project (MKL

corruption, matplotlib-triggered Python breakage): all are first-time-setup or tool-configuration failures rather than logic bugs, and all were resolved by directly following the diagnostic information the tool itself provided (Git's own suggested fix commands, in this case) rather than requiring independent root-cause investigation — a useful distinction from the earlier, harder-to-diagnose silent numerical crashes, where the tool provided no actionable guidance and isolation had to be performed manually across multiple library layers.

Development Log — Dashboard Deployment Fixes, Additional Visualizations, and Global Transferability Validation

Status

Complete. Resolved a Streamlit Cloud path-resolution bug affecting all dashboard pages, added three previously-missing visualizations (DEM, ERA5 climate, NDVI before/after), restructured dashboard navigation and documentation, and conducted a standalone transferability validation of the acquisition pipeline on a non-EU country (India).

Part 1 — Streamlit Cloud Deployment: Path Resolution Bug

Bug

Following initial Streamlit Community Cloud deployment, every dashboard page (except Home) failed with `MediaFileStorageError`, tracing to `st.image()` calls using relative paths of the form `"../outputs/plots/..."`. This pattern had worked correctly during local execution but failed under Streamlit Cloud's deployment structure.

Diagnosis

The relative path `../outputs/plots/...`, resolved from each page file's location (`dashboard/pages/`), was intended to navigate two directory levels up to the project root. However, the `PROJECT_ROOT` variable used to construct absolute paths was itself computed incorrectly:

```
PROJECT_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
```

Starting from `dashboard/pages/1_Environmental_Data.py`, two levels of `os.path.dirname()` resolves only to `dashboard/`, not the actual project root — one level short. This had gone undetected during local testing, plausibly because local execution's working directory or a different invocation context masked the discrepancy, but Streamlit Cloud's `/mount/src/gpie/` deployment structure surfaced it immediately, producing paths of the form `/mount/src/gpie/dashboard/outputs/plots/...` (an erroneous extra `dashboard/` segment) rather than the correct `/mount/src/gpie/outputs/plots/...`.

Fix

Every page file's `PROJECT_ROOT` calculation was corrected to include a third `os.path.dirname()` call:

```
PROJECT_ROOT =
os.path.dirname(os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
```

This was applied individually across all eight dashboard page files (`1_Environmental_Data.py` through `8_About_Data.py`), along with converting all remaining relative path references (`../data/...`) to `os.path.join(PROJECT_ROOT, ...)` construction, consistent with the fix already applied to image paths. Following this correction and redeployment, all pages loaded without error.

Part 2 — Documentation Discoverability

README Restructuring

A "Project Documentation" table was added to the very top of `README.md`, immediately following the project title and tagline, listing all three accompanying documents (`Project_Journal.md`, `Research_Paper.md`, `Devlopment_Log.md`) with one-line descriptions of each — positioned before the live dashboard link, ensuring documentation is the first thing visible to any repository visitor.

GitHub Auto-Render Investigation

A separate concern was raised regarding `README.md` not appearing to auto-render on the repository's main page, appearing only via direct search or direct URL access. Investigation confirmed via direct URL access (`github.com/.../blob/main/README.md`) that the file exists correctly at the expected root path with correct content — ruling out a missing-file or misnamed-file explanation. Further investigation clarified that GitHub's standard behavior places the repository's file/folder listing above the auto-rendered README on the main page (not below or instead of it), which had been misinterpreted as the README "not appearing." This was confirmed to be standard GitHub behavior common to virtually all repositories, not a defect specific to this project.

Dashboard Documentation Section

A parallel "Project Documentation" section, mirroring the README's table, was added to the dashboard's About & Data page, with three columns linking directly to each document's GitHub URL — ensuring documentation is discoverable both via the repository and via the live dashboard.

Part 3 — Missing Visualizations: DEM, ERA5 Climate, and NDVI Before/After

Motivation

A review of the dashboard's map coverage revealed that two datasets acquired with substantial acquisition and processing effort — Copernicus DEM (elevation, 860 tiles) and ERA5 (climate, involving the earlier duplicate-timestamp bug fix) — had never been visualized anywhere in the project, despite being fully processed and present in the master dataset as control variables. Additionally, the Before/After comparison module had only ever been built for NO_2, leaving NDVI — a co-equal secondary outcome variable in the causal model — without an equivalent visual.

DEM Elevation Map

`map_dem_choropleth.py` was implemented following the established choropleth template (EU-27 only, consistent with DEM's EU-27-only acquisition scope), using the `terrain` colormap — a standard cartographic convention for elevation data — to visualize mean elevation per country from `dem_stats_by_country.json`.

ERA5 Climate Map

`map_climate_choropleth.py` was implemented for all 30 countries (EU-27 + control group, since climate data was acquired for the full 30-country scope), using the `coolwarm` colormap (blue=cold to red=warm) to visualize mean 2019–2024 temperature, with control-group countries marked via the established thick-border convention.

NDVI Before/After Map

`map_ndvi_before_after.py` was implemented as a direct structural parallel to the existing NO_2 before/after map (shared vmin/vmax color scale across both year panels, same boundary-loading and control-group-border logic), producing a 2019-vs-2024 side-by-side NDVI comparison.

Colorbar Layout Bug

Initial versions of both before/after maps (NO_2 and NDVI) exhibited a colorbar positioned too close to and slightly overlapping the map panels above it, using `fig.colorbar(..., shrink=0.4)` attached loosely to both axes. This was corrected by switching to an explicit `fig.add_axes()`-defined colorbar position, reserving dedicated vertical space at the bottom of the figure (`fig.subplots_adjust(bottom=0.18, top=0.88, ...)`) rather than relying on matplotlib's automatic layout to avoid overlap — a more reliable approach given the specific two-panel-plus-suptitle-plus-colorbar layout being used. This fix was applied to the NO_2 before/after map first (verified working) and then replicated identically in the NDVI version.

Part 4 — Dashboard Restructuring

Page Reordering

The dashboard's page order was identified as logically inverted: "Environmental Data" (presenting NO_2/NDVI maps) appeared before "Study Design" (explaining the treatment/control comparison framework those maps exist to support), requiring a reader to view results before understanding what was being measured or why. Page files were renumbered (`1_Environmental_Data.py` ↔ `2_Study_Design.py` swapped) so Study Design now appears first, establishing context before data presentation.

Control Variables Page Expansion

The former "Economic & Land Context" page (containing only GDP and Land Cover tabs) was renamed to "Control Variables & Context" and expanded from two tabs to four, incorporating the newly-created DEM (Elevation) and ERA5 (Climate) maps alongside the existing GDP and Land Cover content — consolidating all of the causal model's control-variable visualizations onto a single, appropriately-named page rather than leaving two of them unvisualized anywhere in the project.

Part 5 — Global Transferability Validation

Motivation

The project's original stated design goal — a "globally transferable methodology," explicitly claimed in the earliest project overview — had never been empirically tested; all acquisition, processing, and modeling work had been conducted exclusively within the EU-27 + 3-country control-group scope. This represented a gap between stated claim and demonstrated evidence, worth addressing directly rather than leaving as an unverified assertion, or alternatively softening the claim's language — the former was chosen as achievable within available time.

Design Decision — Standalone Test, Not a New Comparative Study

It was explicitly clarified that this validation is a **standalone architectural test** — confirming the acquisition pipeline itself is portable to a non-EU country — rather than a new comparative causal analysis. Accordingly, no modification to any existing EU-27 project script was made; a new, independent, minimal script was written instead, avoiding any risk of introducing changes to the validated EU-27 pipeline.

Implementation

`test_india_transferability.py` was implemented as a self-contained script reusing only the existing Sentinel Hub authentication module (`auth_sentinelhub.py`), with a simple rectangular bounding-box geometry for India (rather than a precise administrative boundary, since exact boundary precision is unnecessary for an architectural portability test) and the identical evalscript logic used in the EU-27 NO_2 acquisition pipeline, with zero modification to the core request/aggregation structure.

Execution and Result

All 6 requested years (2019–2024) completed successfully with zero failures on first execution, saved to `data/global_transferability_test/india_no2_test.json`. A sample value inspection (January 2019: 2.64×10^{-5} mol/m²) confirmed the returned NO_2 concentration falls within the same physically realistic range observed throughout the EU-27 dataset (approximately $1–6 \times 10^{-5}$), providing evidence the pipeline is not merely executing without error but returning scientifically valid output on a new, previously untested country.

Visualization

`plot_india_trend.py` was implemented to flatten the nested Sentinel Hub response (using the same flattening pattern established for NO_2/NDVI elsewhere in the project) and produce a simple time-series line chart of India's NO_2 trend across the full 72-month period, providing a visual complement to the raw acquisition-success confirmation.

Documentation

A "Transferability Validation" section was added both to `README.md` (positioned after the Key Finding section) and to the dashboard's About & Data page, explicitly framing this as a standalone architectural proof-of-concept rather than a comparative study, to avoid any ambiguity about its scope or claims.

Design Principle Reinforced

The `PROJECT_ROOT` path bug reinforces a recurring theme across this project's debugging history: a piece of logic that "worked" in one execution context (local development) is not proof of correctness, only proof that the specific context happened not to expose the error — the underlying arithmetic error (one `dirname()` call

short of the actual project root) was present from the moment the code was written, simply undetected until a differently-structured deployment environment made it visible. Separately, the transferability-test design decision — building a new, isolated script rather than modifying validated production code to test a tangential claim — reflects a deliberate risk-management principle applied consistently throughout this project: validated, working code paths (here, the EU-27 causal-inference pipeline) should not be touched to accommodate an unrelated exploratory task, even when doing so might appear more code-efficient, since the risk of introducing a regression into already-correct code outweighs the marginal convenience of code reuse.

Part 6 — Portfolio-Finalization Review: Repository Cleanup, Cluster-Robust Standard Errors, and the NDVI Correction

Motivation

Ahead of Zenodo publication and professor outreach, the full repository (code, dashboard, and all three documentation files) was audited end-to-end for correctness, reproducibility, and internal consistency, with a specific goal of applying the same statistical rigor already used for the primary NO_2 result to every other part of the project rather than treating the NO_2 validation as a one-off exercise.

Repository and Documentation Fixes

Several concrete issues were found and corrected: `dashboard/app.py`'s PDF download buttons used relative paths (`open("Research_Paper.pdf")`) that only resolve correctly if the working directory happens to be the repository root, the same path-resolution bug already found and fixed in DOUBLE_JEOPARDY and STOLEN_STRATA — fixed here using the same `BASE_DIR`/`ROOT_DIR` pattern. `requirements.txt` listed only 4 of the ~15 third-party packages the codebase actually imports (missing `numpy`, `matplotlib`, `statsmodels`, `rasterio`, `rasterstats`, `xarray`, `shapely`, `requests`, `python-dotenv`, `PyJWT`, `beautifulsoup4`), meaning the README's own "Running Locally" instructions would fail — corrected by auditing actual imports across the codebase. Six duplicate `" - Copy"` image files in `outputs/plots/` were identified and removed. `Devlopment_Log.md` contained a literal unresolved Git conflict marker (`<<<<<<< HEAD` / `=======`) left over from an incomplete manual merge resolution, and its own title read "GREEN POLICY INTELLIGENCE SYSTEM (GPIS)" rather than "ENGINE (GPIE)" — both fixed. `Project_Journal.md` stated the project integrates "seven" datasets while its own table listed eight, and separately gave two different visualization counts (9 vs. 7) neither of which matched the actual final total of ten (after DEM/climate/NDVI-before-after were added in Part 3, above) — all reconciled to eight datasets and ten visualizations, consistently across `README.md`, `Project_Journal.md`, and the dashboard's homepage metric. Two references in `Research_Paper.md`'s bibliography were cited under their publisher's name (`Scientific Reports`, `Nature Publishing Group`) rather than actual authors — corrected by looking up the original papers and citing the real author lists.

Cluster-Robust Standard Errors

Every causal-inference script (`causal_inference.py`, `causal_inference_placebo.py`, `causal_inference_final_did.py`, `causal_inference_event_study.py`, `causal_inference_ndvi.py`) used `sm.OLS(y, X).fit()` with default (classical, i.i.d.) standard errors. For panel data with repeated monthly observations per country, this is a well-documented pitfall (Bertrand, Duflo & Mullainathan, 2004): within-country serial correlation means classical standard errors understate true uncertainty. All five scripts were updated to cluster standard errors by country (`cov_type="cluster", cov_kwds={"groups": ...}`). Re-running the final DiD model with clustering moved the headline result from p = 0.632 to p = 0.663 — the null

finding became *more* solid, not less, so this correction was safe to apply without threatening the paper's central claim. The event-study was also re-run clustered: 3 of 23 quarters became nominally significant (versus 0 under classical SEs) — close to the ~1 false positive expected by chance at this sample size, with no consistent directional pattern (one pre-treatment quarter plausibly explained by COVID-19 lockdown timing, two post-treatment quarters with opposite signs) — reported transparently rather than omitted, and `plot_event_study.py` was updated to fit with clustering and visually flag the three points, replacing its now-inaccurate hardcoded title claiming zero significant quarters.

Additional Robustness Checks

Five further checks were run against the corrected NO_2 model, all reinforcing the null result: (1) removing GDP entirely, addressing its potential status as a post-treatment "bad control" (Angrist & Pischke, 2009) — the estimate moved *closer* to zero without it (p = 0.880), meaning GDP was not driving the result; (2) a log-transformed outcome, after discovering 23 of 1,930 rows have non-positive mean NO_2 values (a known artifact of satellite trace-gas retrievals near the detection limit — 10 of the 23 cluster in December 2023 across multiple countries, consistent with the previously-documented December data-gap rather than a new issue) — excluding those rows, the log-transformed result (p = 0.669) confirmed the null finding is not a functional-form artifact; (3) treatment-date sensitivity, shifting the assumed date by ±6/±12 months — no shifted date reached significance; (4) a country-level heterogeneity split by baseline pollution level — neither the higher- nor lower-baseline subgroup was significant, though point estimates diverged in sign, flagged as a direction for future work rather than a finding; (5) a formal minimum-detectable-effect calculation, quantifying what had previously only been described qualitatively as "a wide confidence interval" — at 80% power this design can detect an effect of roughly 28% of baseline NO_2 or larger, while the observed coefficient is only ~4.4% of baseline. A SUTVA/spillover discussion was also added to the paper, noting that any real Climate Law effect leaking into the geographically-adjacent, trade-linked control countries would bias the estimate toward zero — meaning the null result is, if anything, conservative. A note was added stating explicitly that the staggered-adoption critique of two-way fixed-effects DiD (Goodman-Bacon and related work) does not apply here, since the Climate Law's simultaneous EU-27-wide rollout means there is no variation in treatment timing across units.

The NDVI Correction — A Genuine New Finding

While extending cluster-robust SEs to every model, it was discovered that `causal_inference_ndvi.py` had never received the same control-group correction the NO_2 analysis went through in Part 1 (Section 4, above) — it still ran the original single-cohort design (`master_dataset.csv`, EU-27 only, no external control group) that the NO_2 placebo test had already proven unreliable for this exact class of problem. Rewriting it to match `causal_inference_final_did.py`'s two-group design (`master_dataset_control.csv`, `did_interaction` term, cluster-robust SEs) produced a materially different result: where the original single-cohort NDVI model found no effect (coefficient = −0.0059, p = 0.128), the corrected model finds a statistically significant relative *decline* in EU-27 NDVI versus the control group (coefficient = −0.0210, p = 0.012, 95% CI [−0.0372, −0.0047]). This is reported as an honest, methodologically robust secondary finding — explicitly not as evidence the Climate Law harmed vegetation, since land-use change, drought, and agricultural-policy shifts are not controlled for in this design — but as a reminder that validation rigor applied only to a project's primary outcome can leave real findings sitting undetected in its secondary outcomes. A new script, `map_ndvi_eu_vs_control_bar.py`, was written (modeled on the existing NO_2 equivalent) to give this finding a visual companion, added to both `Research_Paper.md` (Figure 5, with subsequent figures renumbered) and the dashboard's Causal Results page.

Documentation and Dashboard Sync

All of the above — corrected p-values and confidence intervals, the new robustness checks, and the NDVI finding — were propagated consistently across `Research_Paper.md`, `Project_Journal.md`, this development log, and the dashboard's Causal Results and Methodology pages, so that no version of the project's numbers is stale relative to any other.

Design Principle Reinforced

Extending the NO_2 model's own validation standard (placebo test, genuine control group, cluster-robust SEs) to the project's secondary outcome — rather than treating that standard as satisfied once the primary result was validated — surfaced a real, previously invisible finding. This is the same lesson the NO_2 analysis itself already taught (Part 4 of the original development history): a result that looks settled after one round of scrutiny can still be hiding something that only a second, equally rigorous pass reveals. The corollary this session adds is that rigor is not a property of a *project* — it is a property of each individual analysis within it, and has to be applied to all of them individually, not assumed to transfer from whichever one was checked first.

Part 7 — Panel-Readiness Review and Final Polish

Motivation

With the project's documented issues fixed and its methodology strengthened (Part 6), the finished project was reviewed once more end-to-end, specifically against the kind of critique an Erasmus Mundus GEM/CDE scholarship panel would raise: control-group size, the SUTVA/spillover risk, the NDVI result's interpretation, and country-level aggregation as opposed to sub-national granularity. These are the same limitations already documented in Section 6 of `Research_Paper.md`, which is itself a useful check — the project's genuine limitations are the ones that stand out on a fresh read, not different or previously-missed ones.

Triage Rather Than Doing Everything

Rather than chasing every possible improvement, each candidate addition was triaged by cost versus actual value for a scholarship application, given that six further projects remain to receive the same treatment:

Implemented (cheap, high-value, and directly strengthen an already-identified limitation): a paragraph added to `Research_Paper.md` Section 3.1 making the UK's 2020 EU exit and 2020-end transition period an explicit part of the control-group justification (previously only implicit in the Development Log, not stated in the paper itself); a new Section 7, "Future Work," added to `Research_Paper.md` naming Synthetic Control Method, NUTS-2/grid-cell spatial resolution, formal spatial-autocorrelation diagnostics (Moran's I), and a delayed-effect test around the "Fit for 55" package as specific, well-defined next steps — this converts likely panel questions ("why didn't you do X") into a proactively answered point, without the multi-day data-engineering effort actually building any of them would require; a new "Architecture" (text-based pipeline diagram: data sources → preprocessing → modelling → dashboard) and "Reproducibility" section added to `README.md`; and a `CITATION.cff` file created for the eventual Zenodo release.

Deliberately deferred, not implemented: actually building the Synthetic Control Method, NUTS-2/pixel-level regional re-analysis, or formal spatial-econometric diagnostics — each is a substantial new data-acquisition and modelling effort in its own right, not a documentation fix, and is now explicitly named as future work

instead (see above) rather than left as a silent gap. Presentation-only additions (uncertainty/confidence-interval choropleth overlays, an animated project-timeline walkthrough on the dashboard, a live policy-type filter) were also deferred as polish with real but secondary value, consistent with this project's practice of not pursuing effort-intensive additions that do not change the substantive findings (the same principle already applied to scoping out Module 9 in Part 1).

Outcome

No new empirical claims were added in this pass — all changes are documentation and repository-presentation improvements layered on top of the analysis already validated in Part 6. The Research Paper's reference list was extended with the two Synthetic Control Method citations (Abadie, Diamond & Hainmüller, 2010; Ben-Michael, Feller & Rothstein, 2021) introduced by the new Future Work section.

Development Log — Deep Verify: Independent Recomputation of Every Reported Statistic (2026-08-03)

Status

Complete. Every quantitative claim in `Research_Paper.md` was independently recomputed from `data/master_dataset.csv` and `data/master_dataset_control.csv`, re-running `causal_inference.py`, `causal_inference_placebo.py`, `causal_inference_final_did.py`, `causal_inference_ndvi.py`, and `causal_inference_event_study.py` directly, plus hand-written reimplementations of the four robustness checks (GDP removal, log-transform, treatment-date sensitivity, baseline-pollution heterogeneity split) and the minimum-detectable-effect calculation, since no standalone scripts for those exist in the repository.

Result — one real inconsistency found and corrected

Every number checked out to the reported precision **except** the two "initial single-cohort model" figures (NO_2, Section 4.1; NDVI, Section 4.5 first paragraph). Both were computed with **classical (non-clustered) standard errors**, not the cluster-robust-by-country standard errors this project's own methodology section (3.3) and every other model in the paper explicitly use. This was traceable directly: the corrected two-group NO_2 model's own Methodology page already documents its p-value moving from 0.632 (classical) to 0.663 (cluster-robust) — proof cluster-robust SEs were correctly adopted as the project standard partway through the analysis — but the two earliest, already-superseded single-cohort models were apparently never revisited under that later standard once it was adopted.

Recomputed with cluster-robust SEs, by country:

- **NO_2 initial model** (coefficient -2.29×10^{-6} confirmed exact): p = 0.041, 95% CI $[-4.48 \times 10^{-6}, -9.04 \times 10^{-8}]$ — not 0.026/$[-4.30 \times 10^{-6}, -2.69 \times 10^{-7}]$ as originally reported. Still significant at 5%, so this does **not** change Section 4.1's conclusion or the paper's overall narrative arc for NO_2.
- **NDVI initial model** (coefficient −0.0059 confirmed exact): p = 0.0017 — not p = 0.128 ("no significant effect") as originally reported. This **does** change the accurate narrative: the initial single-cohort NDVI model was already statistically significant under this project's own stated standard-error methodology, not only after the two-group control-group correction. The corrected model (p = 0.012, confirmed exact) remains the trustworthy, reported result — its role is improved identification (isolating an EU-specific effect from a shared regional trend via a genuine control group), not first-time detection of significance.

Everything else — confirmed exact

Placebo test (coefficient -3.29×10^{-6}, p = 0.004); linear-time-trend-controlled model (p = 0.186); corrected two-group NO_2 model (coefficient -1.40×10^{-6}, p = 0.663, CI [-7.68×10^{-6}, $+4.88 \times 10^{-6}$], R^2 = 0.386, N = 1,930); corrected two-group NDVI model (coefficient −0.0210, p = 0.012, CI [−0.0372, −0.0047]); event-study result (exactly 3 of 23 quarters significant at p < 0.05 — 2020Q1 positive, 2023Q1 positive, 2023Q3 negative — matching "opposite signs" and the "close to 1 expected by chance" framing precisely); GDP-removed robustness check (coefficient -4.80×10^{-7}, p = 0.880); log-transformed model (23/1,930 = 1.2% non-positive NO_2 rows, 10 of 23 in December 2023, log coefficient 0.046/≈4.7% relative change, p = 0.669); treatment-date sensitivity (p = 0.764, 0.357, 0.151, 0.086 at the four shifted dates); baseline-pollution heterogeneity split (13 higher-baseline / 14 lower-baseline countries; coefficients -4.49×10^{-6} p = 0.245 and $+3.74 \times 10^{-6}$ p = 0.339); and the minimum-detectable-effect calculation (28.4% of pre-treatment baseline NO_2; observed effect ≈4.4% of that baseline). All matched the paper to the reported precision.

Citations

Spot-checked 3 of the paper's 13 references via independent web search (Tong et al. 2025, *npj Clean Air*; Riveros-Gavilanes 2023, *JORIT*; Mathew et al. 2024, *Scientific Reports* 14, 21624) — all confirmed real and correctly cited. The remaining 10 (Abadie, Diamond & Hainmüller 2010; Angrist & Pischke 2009; Bekes & Kezdi 2021; Ben-Michael, Feller & Rothstein 2021; Bertrand, Duflo & Mullainathan 2004; Bikbov et al. 2024; Callaway, Goodman-Bacon & Sant'Anna 2024; Roth, Sant'Anna, Bilinski & Poe 2023; Wang et al. 2020; Zeldow & Hatfield 2024) were not individually re-verified this round due to time — flagged here rather than silently treated as checked.

Fix applied

Corrected the two initial-model passages in `Research_Paper.md` (Sections 4.1, 4.5, and the Section 5 Discussion paragraph comparing NO_2 and NDVI), `Project_Journal.md` (Methodology Phase 4 and Final Findings #2/#3), and the dashboard's `5_Causal_Results.py` and `6_Methodology.py` pages, to state both the originally-reported classical-SE figures and the corrected cluster-robust figures, and to reframe the NDVI narrative accurately: the control-group correction improved identification, it did not newly create significance. No underlying data, model code, or headline conclusion changes — this is a standard-error consistency fix, not a data or specification error.